

INSTITUTO INFNET

NEUZA GIOVANNA DE OLIVEIRA

DISCIPLINA: Projeto de Bloco: Desenvolvimento Back-End

Documento Final do Projeto

MediCare — Plataforma de Agendamento e Gerenciamento de Consultas Médicas

---

Sumário

1. Introdução 4

2. Requisitos do Sistema 4

2.1 Descrição do sistema 4

2.2 Objetivo 5

2.3 Usuários e sistemas externos 5

2.4 Requisitos funcionais 6

2.5 Requisitos não funcionais 7

2.6 Regras de negócio 8

2.7 Mudanças de requisitos 8

3. Casos de Uso 9

3.1 Atores 9

3.2 Lista de casos de uso 9

3.3 Descrição dos casos de uso 9

3.4 Mapeamento entre atores e casos de uso 15

3.5 Diagrama UML de casos de uso 16

3.6 Casos de uso a explorar com os usuários 17

4. Modelagem de Classes 18

4.1 Classes candidatas 18

4.2 Matriz de associação 19

4.3 Dicionário de dados 19

4.4 Enumerações 22

4.5 Associações 22

4.6 Diagrama de classes 23

5. Modelagem Dinâmica 24

5.1 Diagramas de sequência 24

5.2 Diagramas de comunicação 38

5.3 Métodos das classes 46

6. Implementação 49

6.1 Arquitetura 49

6.2 Persistência 49

6.3 Interface com Razor Pages 51

7. Testes 57

8. Execução e Artefatos 58

9. Conclusão 59

Anexo — Código-fonte 60

1. Introdução

Este documento reúne, na versão final, todos os artefatos produzidos ao longo do Projeto de Bloco: requisitos, casos de uso, modelagem de classes, diagramas dinâmicos, implementação e testes do MediCare, uma plataforma de agendamento e gerenciamento de consultas médicas.

O projeto seguiu o RUP (Rational Unified Process), com cinco entregas. Cada entrega partiu da anterior e, na quarta, o modelo foi revisado para incorporar as mudanças de requisitos pedidas pelo cliente. As seções a seguir apresentam o resultado final dessas etapas, já com as mudanças incorporadas.

Entrega	Fase do RUP	Conteúdo
TP1	Concepção	Descrição do sistema, usuários e requisitos funcionais.
TP2	Concepção	Atores, casos de uso, mapeamento e diagrama UML de casos de uso.
TP3	Elaboração	Classes candidatas, matriz de associação, dicionário de dados, diagrama de classes e leitura e escrita de arquivos CSV em C#.
TP4	Construção	Mudanças de requisitos, casos de uso revisados e novos, diagramas de sequência e de comunicação, métodos das classes e sua implementação.
TP5	Construção	Aplicação Razor Pages, persistência com EF Core e testes automatizados.

O código-fonte e os documentos de cada etapa estão no repositório **https://github.com/CodeByGiovanna/MediCare**, e o código completo também está no anexo deste documento.

---

2. Requisitos do Sistema

2.1 Descrição do sistema

O MediCare é uma plataforma web criada para facilitar o acesso e o gerenciamento de consultas médicas. O paciente encontra profissionais de saúde pela especialidade, consulta os horários disponíveis de cada um e agenda a consulta de forma simples, sem depender de ligações ou de atendimento presencial.

A plataforma oferece duas modalidades de atendimento: consulta presencial e teleconsulta. A modalidade é definida pelo médico para cada horário que ele disponibiliza. Nas consultas presenciais, o sistema mostra o local do atendimento; nas teleconsultas, paciente e médico acessam a consulta pela plataforma no horário agendado. Depois de agendar, o paciente pode acompanhar suas consultas, cancelá-las ou reagendá-las para outro horário livre do mesmo médico.

Cada pessoa acessa a plataforma com a própria conta, e o que ela pode fazer depende do seu perfil. O paciente cria a conta informando seus dados de contato, a data de nascimento e o canal em que prefere receber os avisos. O médico se cadastra com o CRM e a especialidade. As senhas são guardadas de forma protegida, e uma conta

desativada pelo administrador deixa de ter acesso.

O sistema controla a disponibilidade para evitar conflitos na agenda. Um horário reservado deixa de aparecer para os outros pacientes e volta a ficar livre quando a consulta é cancelada ou reagendada. O médico não consegue cadastrar horários sobrepostos, em datas passadas ou presenciais sem um local de atendimento, e cada operação que não pode ser concluída é explicada ao usuário.

O médico gerencia a própria disponibilidade, cadastrando os dias, horários, modalidade e local de cada atendimento, e acompanha a agenda de consultas. Após cada atendimento, ele registra a consulta como realizada, o que forma o histórico do paciente, que pode ser consultado nos atendimentos seguintes.

Durante o projeto, ao revisar os dados que o sistema manipularia, o cliente pediu que a recepção da clínica também pudesse agendar, remarcar e cancelar consultas para os pacientes atendidos no balcão. Também pediu que os pacientes recebessem confirmações e lembretes das consultas por e-mail, SMS ou WhatsApp, por meio de um sistema de notificação externo, e que fosse possível entrar com contas Google, Microsoft ou Meta.

Um administrador é responsável pelo funcionamento geral da plataforma: acompanha e controla as contas de usuários, consulta e cancela agendamentos quando necessário e aciona a geração dos lembretes e o envio das notificações pendentes.

2.2 Objetivo

O objetivo do MediCare é centralizar em uma única plataforma o agendamento e o acompanhamento de consultas médicas presenciais e por teleconsulta, facilitando o acesso dos pacientes aos profissionais de saúde, dando aos médicos controle sobre sua disponibilidade e agenda, e mantendo pacientes informados sobre cada consulta.

2.3 Usuários e sistemas externos

Usuário / sistema	Papel
Paciente	Pesquisa médicos, consulta horários, agenda, cancela e reagenda consultas, acessa teleconsultas e recebe notificações.
Recepcionista	Funcionário da recepção que consulta horários e agenda, reagenda e cancela consultas em nome dos pacientes.
Médico	Disponibiliza horários, define a modalidade de atendimento, acompanha a agenda, registra atendimentos e consulta o histórico dos pacientes que atende.
Administrador	Gerencia usuários e agendamentos e aciona o envio das notificações.
Sistema de Notificação	Sistema externo que entrega as mensagens por e-mail, SMS ou WhatsApp.
Sistema de autenticação SSO	Sistema externo (Google, Microsoft, Meta) usado no login com conta externa.

2.4 Requisitos funcionais

Os requisitos definidos na TP1 foram mantidos, e os incluídos com as mudanças da TP4 estão indicados na coluna Origem. Todos estão implementados na versão final, exceto o RF28: o login já reconhece contas externas, mas a integração com os provedores ficou para uma próxima versão.

ID	O sistema deve permitir...	Origem
	Paciente	
RF01	Criar uma conta na plataforma.	TP1
RF02	Fazer login.	TP1
RF03	Pesquisar médicos por especialidade.	TP1
RF04	Visualizar as informações de um médico.	TP1
RF05	Consultar os horários disponíveis para atendimento.	TP1
RF06	Visualizar a modalidade de atendimento de cada horário.	TP1
RF07	Agendar uma consulta presencial ou uma teleconsulta.	TP1
RF08	Visualizar suas consultas agendadas.	TP1
RF09	Visualizar o local de uma consulta presencial.	TP1
RF10	Acessar uma teleconsulta no horário agendado.	TP1
RF11	Cancelar uma consulta.	TP1
RF12	Reagendar uma consulta.	TP1
RF13	Escolher o canal em que recebe os avisos (e-mail, SMS ou WhatsApp) e atualizar seu telefone.	TP4
	Recepcionista	
RF14	Consultar horários e agendar, reagendar e cancelar consultas de pacientes cadastrados.	TP4
	Médico	
RF15	Criar e gerenciar seu perfil profissional.	TP1
RF16	Informar seus horários disponíveis para atendimento.	TP1
RF17	Definir se um horário é para consulta presencial ou teleconsulta.	TP1
RF18	Visualizar sua agenda e as informações das consultas agendadas.	TP1
RF19	Acessar suas teleconsultas agendadas.	TP1
RF20	Remover ou alterar a disponibilidade de um horário.	TP1
RF21	Registrar uma consulta como realizada.	TP4
RF22	Consultar o histórico de consultas realizadas de um paciente que atende.	TP4
	Administrador	
RF23	Visualizar os usuários cadastrados.	TP1
RF24	Gerenciar pacientes, médicos e recepcionistas, ativando ou desativando suas contas.	TP1

ID	O sistema deve permitir...	Origem
RF25	Acompanhar e cancelar os agendamentos realizados na plataforma.	TP1
RF26	Gerar os lembretes das consultas do dia seguinte e enviar as notificações pendentes.	TP4
	Sistema	
RF27	Gerar notificações de confirmação, reagendamento e cancelamento no canal preferido do paciente.	TP4
RF28	Permitir o login com uma conta Google, Microsoft ou Meta.	TP4

2.5 Requisitos não funcionais

Os requisitos não funcionais descrevem as características de qualidade atendidas pela versão implementada.

ID	Requisito
RNF01	A aplicação deve ser acessada pelo navegador, sem instalação no computador do usuário (ASP.NET Core Razor Pages, .NET 8).
RNF02	Os dados devem ser gravados em banco de dados relacional (SQLite, acessado pelo EF Core), com chaves estrangeiras garantindo a integridade das associações.
RNF03	As senhas não podem ser armazenadas em texto; o sistema guarda apenas o hash SHA-256.
RNF04	As páginas devem exigir login e só podem ser acessadas pelos perfis autorizados.
RNF05	Os formulários devem ser protegidos contra requisições forjadas (token antiforgery).
RNF06	Duas reservas do mesmo horário não podem ser processadas ao mesmo tempo.
RNF07	Quando uma regra de negócio impedir uma operação, o usuário deve receber uma mensagem explicando o motivo.
RNF08	As regras de negócio devem ser verificadas por testes automatizados.

2.6 Regras de negócio

ID	Regra
RN01	O e-mail é único entre todos os usuários, de qualquer perfil.
RN02	Conta desativada não consegue entrar. Conta de provedor externo não usa senha da plataforma.
RN03	Um horário só pode estar reservado por uma consulta agendada de cada vez.
RN04	Paciente desativado não pode agendar consultas.
RN05	Só consultas agendadas podem ser canceladas, reagendadas ou registradas como realizadas.
RN06	O reagendamento só pode ser feito para um horário livre do mesmo médico.
RN07	Um horário não pode estar em data passada, deve terminar depois de começar e não pode conflitar com outro horário do mesmo médico.
RN08	Horário presencial exige um local de atendimento do próprio médico; teleconsulta não tem local.
RN09	Horário que já foi usado em alguma consulta não pode ser removido.
RN10	Cada agendamento, reagendamento e cancelamento gera uma notificação no canal preferido do paciente. Cada consulta recebe no máximo um lembrete.
RN11	A teleconsulta só pode ser acessada pelo paciente e pelo médico da consulta, dentro do horário agendado.
RN12	O médico só pode registrar como realizadas as consultas da própria agenda e só vê o histórico de pacientes que atende.

2.7 Mudanças de requisitos

As mudanças pedidas pelo cliente na quarta entrega e o seu impacto no projeto:

Mudança	O que o cliente pediu	Impacto no projeto
M1 — Recepcionista	Agendar, remarcar e cancelar consultas em nome dos pacientes atendidos no balcão.	Novo ator e nova classe Recepcionista; UC03 a UC07 passaram a ter a recepcionista como ator.
M2 — Notificações	Confirmações e lembretes das consultas por e-mail, SMS ou WhatsApp.	Novo ator externo, nova classe Notificacao, novo caso de uso UC14 e canal preferido no paciente.
M3 — Histórico	O médico consultar as consultas anteriores do paciente.	Novo caso de uso UC15; UC10 passou a registrar a consulta como realizada.
M4 — Login externo	Entrar com conta Google, Microsoft ou Meta.	Novo ator externo, atributo provedor em Usuario e fluxo alternativo em UC02.

---

3. Casos de Uso

3.1 Atores

Ator	Tipo	Participação
Paciente	Usuário	UC01 a UC08 e UC11.
Recepcionista	Usuário	UC02 a UC07, em nome do paciente atendido.
Médico	Usuário	UC01, UC02, UC08 a UC11 e UC15.
Administrador	Usuário	UC02, UC12, UC13 e UC14.
Sistema de Notificação	Sistema externo	Recebe as notificações geradas em UC05, UC06, UC07, UC13 e UC14.
Sistema de autenticação SSO	Sistema externo	Informa os dados do usuário no login externo (UC02).

3.2 Lista de casos de uso

ID	Caso de uso	Atores	Origem
UC01	Criar conta	Paciente, Médico	TP2
UC02	Realizar login	Todos os usuários, Sistema SSO	TP2
UC03	Pesquisar médicos	Paciente, Recepcionista	TP2
UC04	Consultar horários disponíveis	Paciente, Recepcionista	TP2
UC05	Agendar consulta	Paciente, Recepcionista	TP2
UC06	Cancelar consulta	Paciente, Recepcionista	TP2
UC07	Reagendar consulta	Paciente, Recepcionista	TP2
UC08	Gerenciar perfil	Paciente, Médico	TP2
UC09	Gerenciar disponibilidade	Médico	TP2
UC10	Gerenciar agenda	Médico	TP2
UC11	Realizar teleconsulta	Paciente, Médico	TP2
UC12	Gerenciar usuários	Administrador	TP2
UC13	Gerenciar agendamentos	Administrador	TP2
UC14	Enviar notificações	Administrador, Sistema de Notificação	TP4
UC15	Consultar histórico do paciente	Médico	TP4

Os casos de uso UC01 a UC13 foram definidos na TP2 e reescritos na TP4 para refletir as mudanças do cliente; UC14 e UC15 foram criados na TP4.

3.3 Descrição dos casos de uso

UC01 — Criar conta

Atores: Paciente, Médico

Objetivo: Permitir que um novo paciente ou médico seja cadastrado na plataforma.

Fluxo principal:

1. O usuário seleciona a opção de criação de conta.
2. O sistema solicita os dados de cadastro. O paciente informa nome, e-mail, senha, telefone, data de nascimento e o canal em que deseja receber avisos. O médico informa nome, e-mail, senha, CRM, telefone e especialidade.
3. O sistema verifica se o e-mail já está em uso por qualquer usuário.
4. O sistema registra a conta com a senha armazenada em forma de hash.
5. O sistema informa que o cadastro foi realizado.

Fluxos alternativos:

- 3a. E-mail já cadastrado: o sistema informa o erro e não cria a conta.

UC02 — Realizar login

Atores: Paciente, Recepcionista, Médico, Administrador; Sistema de autenticação SSO

Objetivo: Permitir que o usuário acesse as funcionalidades do seu perfil.

Fluxo principal:

1. O usuário informa e-mail e senha.
2. O sistema localiza a conta pelo e-mail.
3. O sistema verifica a senha e se a conta está ativa.
4. O sistema apresenta o menu correspondente ao tipo de usuário.

Fluxos alternativos:

- 1a. Login com conta externa: o usuário escolhe entrar com Google, Microsoft ou Meta; o sistema de autenticação SSO valida o usuário e devolve seus dados; o sistema localiza a conta vinculada e segue para o passo 4.
- 2a. A conta encontrada utiliza provedor externo: o sistema orienta o usuário a entrar pelo provedor.
- 3a. Senha incorreta, e-mail inexistente ou conta desativada: o sistema informa o erro.

UC03 — Pesquisar médicos

Atores: Paciente, Recepcionista

Objetivo: Permitir que o paciente, ou a recepcionista que o atende, encontre médicos na plataforma.

Fluxo principal:

1. O usuário acessa a pesquisa.
2. O usuário informa a especialidade desejada ou pesquisa todas.
3. O sistema apresenta os médicos ativos correspondentes.

UC04 — Consultar horários disponíveis

Atores: Paciente, Recepcionista

Objetivo: Permitir a consulta dos horários livres de um médico.

Fluxo principal:

1. O usuário seleciona um médico.
2. O sistema apresenta os horários livres, ordenados por data e hora, com a modalidade e, quando presencial, o local do atendimento.

UC05 — Agendar consulta

Atores: Paciente, Recepcionista; Sistema de Notificação

Objetivo: Registrar uma consulta em um horário disponível.

Fluxo principal:

1. Quando o ator é a recepcionista, ela escolhe na lista o paciente que está atendendo.
2. O usuário seleciona o médico e o horário.
3. O sistema verifica se o paciente está ativo e se o horário está livre.
4. O sistema reserva o horário e registra a consulta.
5. O sistema gera uma notificação de confirmação no canal preferido do paciente.
6. O sistema apresenta a confirmação do agendamento.

Fluxos alternativos:

- 3a. Horário já reservado ou paciente desativado: o sistema informa o erro e nada é registrado.

UC06 — Cancelar consulta

Atores: Paciente, Recepcionista; Sistema de Notificação

Objetivo: Cancelar uma consulta agendada.

Fluxo principal:

1. O sistema lista as consultas agendadas do paciente.
2. O usuário seleciona a consulta.
3. O sistema altera a consulta para cancelada e libera o horário.
4. O sistema gera uma notificação de cancelamento.

Fluxos alternativos:

- 2a. A consulta escolhida não pertence ao paciente ou não está agendada: o sistema informa o erro.

UC07 — Reagendar consulta

Atores: Paciente, Recepcionista; Sistema de Notificação

Objetivo: Alterar o horário de uma consulta agendada.

Fluxo principal:

1. O usuário seleciona uma consulta agendada.
2. O sistema apresenta os horários livres do mesmo médico.
3. O usuário escolhe o novo horário.
4. O sistema reserva o novo horário, libera o anterior e atualiza a consulta.
5. O sistema gera uma notificação de reagendamento.

Fluxos alternativos:

- 3a. Horário de outro médico ou já reservado: o sistema informa o erro e mantém a consulta como estava.

UC08 — Gerenciar perfil

Atores: Paciente, Médico

Objetivo: Permitir que o usuário mantenha seus dados atualizados.

O paciente pode alterar o telefone e o canal pelo qual recebe as notificações. O médico pode alterar o telefone profissional.

UC09 — Gerenciar disponibilidade

Atores: Médico

Objetivo: Permitir que o médico cadastre e remova seus horários de atendimento.

Fluxo principal:

1. O médico informa data, hora de início, hora de término e modalidade.
2. Se a modalidade for presencial, o médico escolhe um dos seus locais de atendimento.
3. O sistema verifica se a data não é passada, se o término é posterior ao início e se o horário não conflita com outro do mesmo médico.
4. O sistema registra o horário, que passa a ser oferecido aos pacientes.

Fluxos alternativos:

- 3a. Dados inválidos ou conflito de horário: o sistema informa o erro.
- Remoção: o médico pode remover um horário que nunca foi usado em uma consulta.

UC10 — Gerenciar agenda

Atores: Médico

Objetivo: Permitir que o médico acompanhe suas consultas e registre os atendimentos.

Fluxo principal:

1. O médico abre a agenda.
2. O sistema apresenta as consultas agendadas, ordenadas por data e hora.
3. Após o atendimento, o médico seleciona a consulta e a registra como realizada.

Fluxos alternativos:

- 3a. A consulta não pertence à agenda do médico: o sistema informa o erro.

UC11 — Realizar teleconsulta

Atores: Paciente, Médico

Objetivo: Permitir que paciente e médico acessem uma consulta por teleconsulta.

Fluxo principal:

1. O usuário escolhe acessar uma teleconsulta.
2. O sistema lista as teleconsultas agendadas em que o usuário participa.
3. O usuário seleciona a consulta.
4. O sistema verifica se o usuário é o paciente ou o médico da consulta, se a modalidade é teleconsulta e se a consulta está agendada.
5. O sistema verifica se o momento atual está dentro do horário agendado.
6. O sistema libera o acesso e a consulta é realizada de forma remota.

Fluxos alternativos:

- 4a. Usuário não participa, consulta presencial ou não agendada: o sistema informa o erro.
- 5a. Fora do horário agendado: o sistema informa o horário em que a teleconsulta poderá ser acessada.

UC12 — Gerenciar usuários

Atores: Administrador

Objetivo: Permitir que o administrador acompanhe e controle as contas da plataforma.

O administrador lista pacientes, médicos, recepcionistas e administradores e pode ativar ou desativar uma conta pelo e-mail. Uma conta desativada não consegue realizar login.

UC13 — Gerenciar agendamentos

Atores: Administrador; Sistema de Notificação

Objetivo: Permitir que o administrador acompanhe e cancele agendamentos.

Fluxo principal:

1. O administrador lista os agendamentos, podendo filtrar por situação.
2. O sistema apresenta as consultas ordenadas por data e hora.
3. Se necessário, o administrador seleciona uma consulta agendada e a cancela.
4. O sistema segue o mesmo processo de UC06: cancela a consulta, libera o horário e gera a notificação de cancelamento.

UC14 — Enviar notificações

Atores: Administrador, Sistema de Notificação

Objetivo: Entregar ao sistema de notificação as confirmações e os lembretes de consulta.

Fluxo principal:

1. O sistema gera notificações automaticamente ao agendar, reagendar ou cancelar (UC05 a UC07).
2. O administrador solicita a geração dos lembretes do dia seguinte.
3. O sistema cria um lembrete para cada consulta agendada para aquele dia que ainda não tenha lembrete.
4. O administrador solicita o envio das notificações pendentes.
5. O sistema entrega canal, destino e mensagem ao sistema de notificação e marca cada notificação como enviada.

UC15 — Consultar histórico do paciente

Atores: Médico

Objetivo: Permitir que o médico veja as consultas anteriores de um paciente.

Fluxo principal:

1. O médico escolhe o paciente na agenda ou informa o seu e-mail.
2. O sistema verifica se o paciente possui ao menos uma consulta com esse médico.
3. O sistema apresenta as consultas realizadas do paciente, da mais recente para a mais antiga.

Fluxos alternativos:

- 2a. O paciente não possui consultas com o médico: o sistema não exibe o histórico.

3.4 Mapeamento entre atores e casos de uso

Caso de uso	Paciente	Recep.	Médico	Admin.	Notif.	SSO
UC01 — Criar conta	X		X			
UC02 — Realizar login	X	X	X	X		X
UC03 — Pesquisar médicos	X	X				
UC04 — Consultar horários disponíveis	X	X				
UC05 — Agendar consulta	X	X			X	
UC06 — Cancelar consulta	X	X			X	
UC07 — Reagendar consulta	X	X			X	
UC08 — Gerenciar perfil	X		X			
UC09 — Gerenciar disponibilidade			X			
UC10 — Gerenciar agenda			X			
UC11 — Realizar teleconsulta	X		X			
UC12 — Gerenciar usuários				X		
UC13 — Gerenciar agendamentos				X	X	
UC14 — Enviar notificações				X	X	
UC15 — Consultar histórico do paciente			X			

Recep. = Recepcionista; Admin. = Administrador; Notif. = Sistema de Notificação; SSO = Sistema de autenticação SSO.

3.5 Diagrama UML de casos de uso

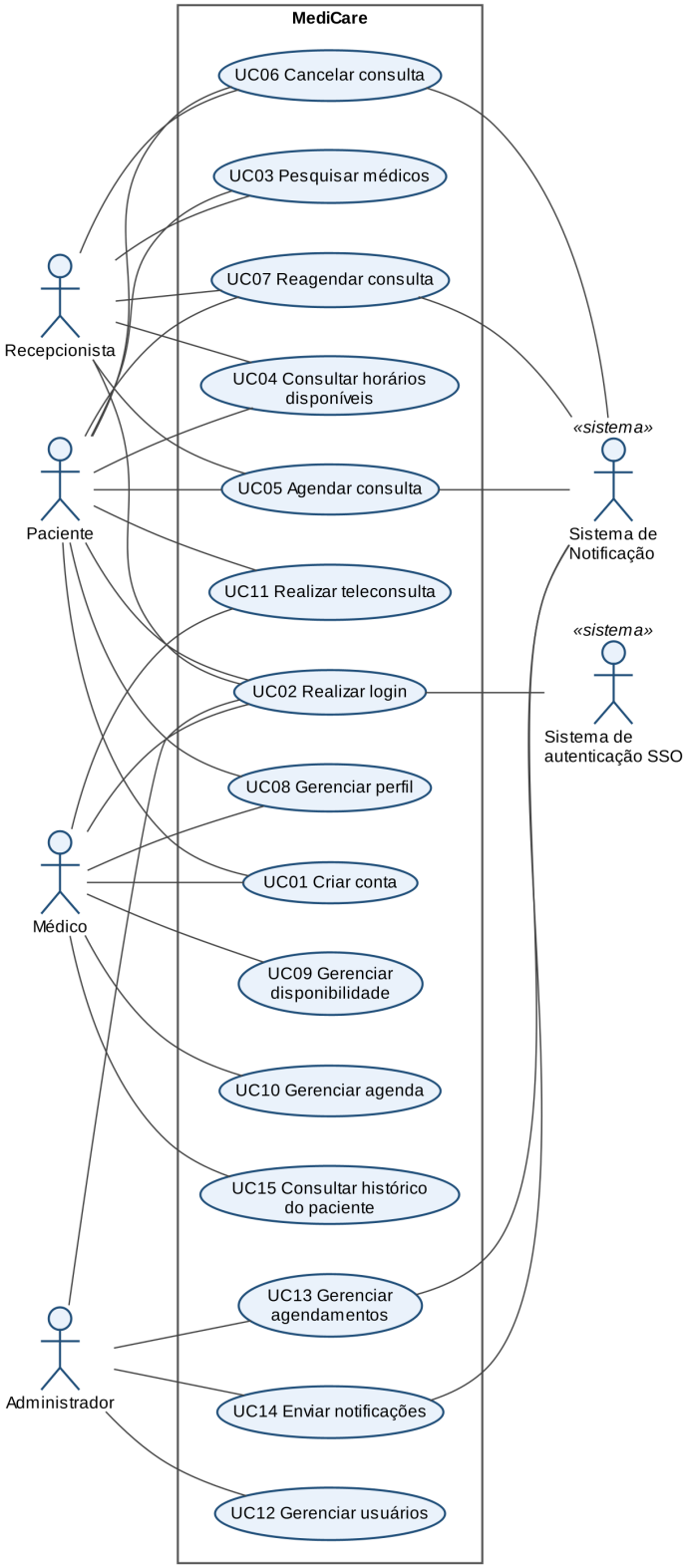


Figura 1 — Diagrama UML de casos de uso do MediCare.

3.6 Casos de uso a explorar com os usuários

Durante o projeto surgiram necessidades que ficaram fora do escopo desta versão. Elas são candidatas a novos casos de uso e devem ser validadas com pacientes, médicos e com a equipe da clínica antes de serem especificadas:

Possível caso de uso	Origem	Ponto a validar com os usuários
Prescrever exames	Pedido do cliente na revisão da TP4	Quais informações a prescrição precisa ter e como o paciente a recebe.
Cadastrar locais de atendimento	Implementação	Hoje os locais vêm dos dados iniciais; o médico precisa cadastrar e editar os seus.
Cadastrar paciente na recepção	Implementação	Pacientes sem conta que chegam ao balcão precisam ser cadastrados pela recepcionista.
Entrar com conta Google, Microsoft ou Meta	Mudança M4	O modelo e o login já reconhecem essas contas; falta a integração com os provedores.
Entregar notificações pelo serviço externo	Mudança M2	Escolha do serviço de e-mail, SMS e WhatsApp e confirmação de entrega.
Abrir a sala de vídeo da teleconsulta	UC11	Plataforma de vídeo a ser integrada após a liberação do acesso.

---

4. Modelagem de Classes

4.1 Classes candidatas

As classes foram identificadas a partir dos substantivos da descrição do sistema e dos casos de uso. Cada termo foi avaliado: os que representam informações a guardar viraram classes; os demais viraram atributos, operações ou foram descartados.

Termo identificado	Origem	Decisão	Justificativa
Usuário	UC01, UC02, UC12	Classe abstrata **Usuario**	Dados de acesso comuns a todos os perfis.
Paciente, Médico, Administrador	TP1, TP2	Classes **Paciente**, **Medico**, **Administrador**	Atores com dados e responsabilidades próprios.
Recepcionista	Mudança M1	Classe **Recepcionista**	Novo ator que agenda pelos pacientes.
Especialidade	TP1, UC03	Classe **Especialidade**	Critério de pesquisa de médicos.
Horário disponível	UC04, UC05, UC09	Classe **HorarioDisponivel**	Data, hora, modalidade e situação de cada horário oferecido.
Local do atendimento	TP1	Classe **LocalAtendimento**	Endereço das consultas presenciais.
Consulta / agendamento	UC05 a UC07	Classe **Consulta**	Liga um paciente a um horário.
Notificação	Mudança M2	Classe **Notificacao**	Mensagem a entregar ao paciente.
Modalidade, situação da consulta	TP1, UC06	Enumerações **Modalidade** e **StatusConsulta**	Valores fixos.
Canal, tipo e situação da notificação; provedor de login	Mudanças M2 e M4	Enumerações	Valores fixos: CanalNotificacao, TipoNotificacao, StatusNotificacao, ProvedorAutenticacao.
Conta, credenciais, perfil	UC01, UC02, UC08	Atributos de Usuario	Não têm identidade própria.
Agenda do médico, histórico	UC10, UC15	Consultas sobre Consulta e HorarioDisponivel	Não há dado novo a guardar.
Teleconsulta	UC11	Consulta + Modalidade	Consulta cujo horário é de teleconsulta.
Cancelamento, reagendamento	UC06, UC07	Operações de Consulta	Ações sobre a consulta.

4.2 Matriz de associação

Cada célula indica a multiplicidade da classe da coluna em relação a um objeto da classe da linha.

	Usuario	Paciente	Medico	Recep.	Admin.	Especial.	Local	Horário	Consulta	Notif.
Usuario	—	herança	herança	herança	herança	—	—	—	—	—
Paciente	herança	—	—	—	—	—	—	—	0..*	—
Medico	herança	—	—	—	—	1	0..*	0..*	—	—
Recepcionista	herança	—	—	—	—	—	—	—	—	—
Administrador	herança	—	—	—	—	—	—	—	—	—
Especialidade	—	—	0..*	—	—	—	—	—	—	—
LocalAtendimento	—	—	1	—	—	—	—	0..*	—	—
HorarioDisponivel	—	—	1	—	—	—	0..1	—	0..*	—
Consulta	—	1	—	—	—	—	—	1	—	0..*
Notificacao	—	—	—	—	—	—	—	—	1	—

Recepcionista e Administrador não têm associações estruturais: atuam por meio das operações do sistema.

4.3 Dicionário de dados

Atributos de *referência* representam as associações do diagrama; no banco, são chaves estrangeiras.

Usuario (abstrata) — dados comuns de todos que acessam a plataforma

Atributo	Tipo	Obrig.	Descrição
id	int	Sim	Identificador único dentro do tipo de usuário.
nome	string	Sim	Nome completo.
email	string	Sim	E-mail de acesso, único entre todos os usuários.
senhaHash	string	Não	Hash SHA-256 da senha; vazio em contas de provedor externo.
provedor	ProvedorAutenticacao	Sim	Senha própria (Local), Google, Microsoft ou Meta.
ativo	bool	Sim	Indica se a conta pode acessar a plataforma.
dataCadastro	DateTime	Sim	Data e hora de criação da conta.

Paciente (herda de Usuario) — usuário que agenda consultas

Atributo	Tipo	Obrig.	Descrição
telefone	string	Sim	Telefone de contato; destino de SMS e WhatsApp.
dataNascimento	Date	Sim	Data de nascimento.
canalPreferido	CanalNotificacao	Sim	Canal em que recebe as notificações.

Medico (herda de Usuario) — profissional que atende as consultas

Atributo	Tipo	Obrig.	Descrição
crm	string	Sim	Registro no Conselho Regional de Medicina, com a UF.
telefone	string	Sim	Telefone profissional.
especialidadeId	int (referência)	Sim	Especialidade do médico.

Recepcionista e **Administrador** (herdam de Usuario): não têm atributos próprios.

Especialidade — área de atuação usada na pesquisa

Atributo	Tipo	Obrig.	Descrição
id	int	Sim	Identificador único.
nome	string	Sim	Nome da especialidade.

LocalAtendimento — endereço de atendimento presencial de um médico

Atributo	Tipo	Obrig.	Descrição
id	int	Sim	Identificador único.
medicoId	int (referência)	Sim	Médico que atende no local.
nome	string	Sim	Nome da clínica ou consultório.
endereco	string	Sim	Logradouro, número e complemento.
cidade	string	Sim	Cidade.

HorarioDisponivel — intervalo oferecido pelo médico

Atributo	Tipo	Obrig.	Descrição
id	int	Sim	Identificador único.
medicoId	int (referência)	Sim	Médico dono do horário.
data	Date	Sim	Dia do atendimento.
horaInicio	Time	Sim	Hora de início.
horaFim	Time	Sim	Hora de término.
modalidade	Modalidade	Sim	Presencial ou Teleconsulta.
localId	int (referência)	Não	Local do atendimento; só nos horários presenciais.
disponivel	bool	Sim	Verdadeiro enquanto não houver consulta agendada no horário.

Consulta — agendamento de um paciente em um horário

Atributo	Tipo	Obrig.	Descrição
id	int	Sim	Identificador único.
pacienteId	int (referência)	Sim	Paciente que agendou.
horarioId	int (referência)	Sim	Horário reservado; muda no reagendamento.
status	StatusConsulta	Sim	Agendada, Cancelada ou Realizada.
dataAgendamento	DateTime	Sim	Momento do agendamento.

Notificacao — mensagem a ser entregue ao paciente

Atributo	Tipo	Obrig.	Descrição
id	int	Sim	Identificador único.
consultaId	int (referência)	Sim	Consulta que originou a notificação.
tipo	TipoNotificacao	Sim	Confirmação, reagendamento, cancelamento ou lembrete.
canal	CanalNotificacao	Sim	Canal usado no envio.
destino	string	Sim	E-mail ou telefone de destino.
mensagem	string	Sim	Texto com data, hora, médico e local ou modalidade.
dataCriacao	DateTime	Sim	Momento em que foi gerada.
status	StatusNotificacao	Sim	Pendente ou enviada.
dataEnvio	DateTime	Não	Momento da entrega ao sistema de notificação.

4.4 Enumerações

Enumeração	Valores	Descrição
Modalidade	Presencial, Teleconsulta	Forma de atendimento do horário.
StatusConsulta	Agendada, Cancelada, Realizada	Situação da consulta.
ProvedorAutenticacao	Local, Google, Microsoft, Meta	Forma de autenticação da conta.
CanalNotificacao	Email, Sms, WhatsApp	Canal de envio das notificações.
TipoNotificacao	Confirmacao, Reagendamento, Cancelamento, Lembrete	Motivo da notificação.
StatusNotificacao	Pendente, Enviada	Situação do envio.

4.5 Associações

Associação	Classes	Multiplicidade	Descrição
herança	Paciente, Recepcionista, Medico, Administrador → Usuario	—	Os tipos de usuário herdam os dados de acesso.
possui	Medico — Especialidade	0..* : 1	Cada médico tem uma especialidade.
disponibiliza	Medico □ HorarioDisponivel	1 : 0..*	Composição: o horário pertence ao médico.
atende em	Medico □ LocalAtendimento	1 : 0..*	Composição: locais de atendimento do médico.
ocorre em	HorarioDisponivel — LocalAtendimento	0..* : 0..1	Só horários presenciais têm local.
agenda	Paciente — Consulta	1 : 0..*	Consultas do paciente.
ocupa	Consulta — HorarioDisponivel	0..* : 1	Um horário pode aparecer em várias consultas ao longo do tempo, mas só uma agendada por vez.
gera	Consulta — Notificacao	1 : 0..*	Notificações geradas pela consulta.

4.6 Diagrama de classes

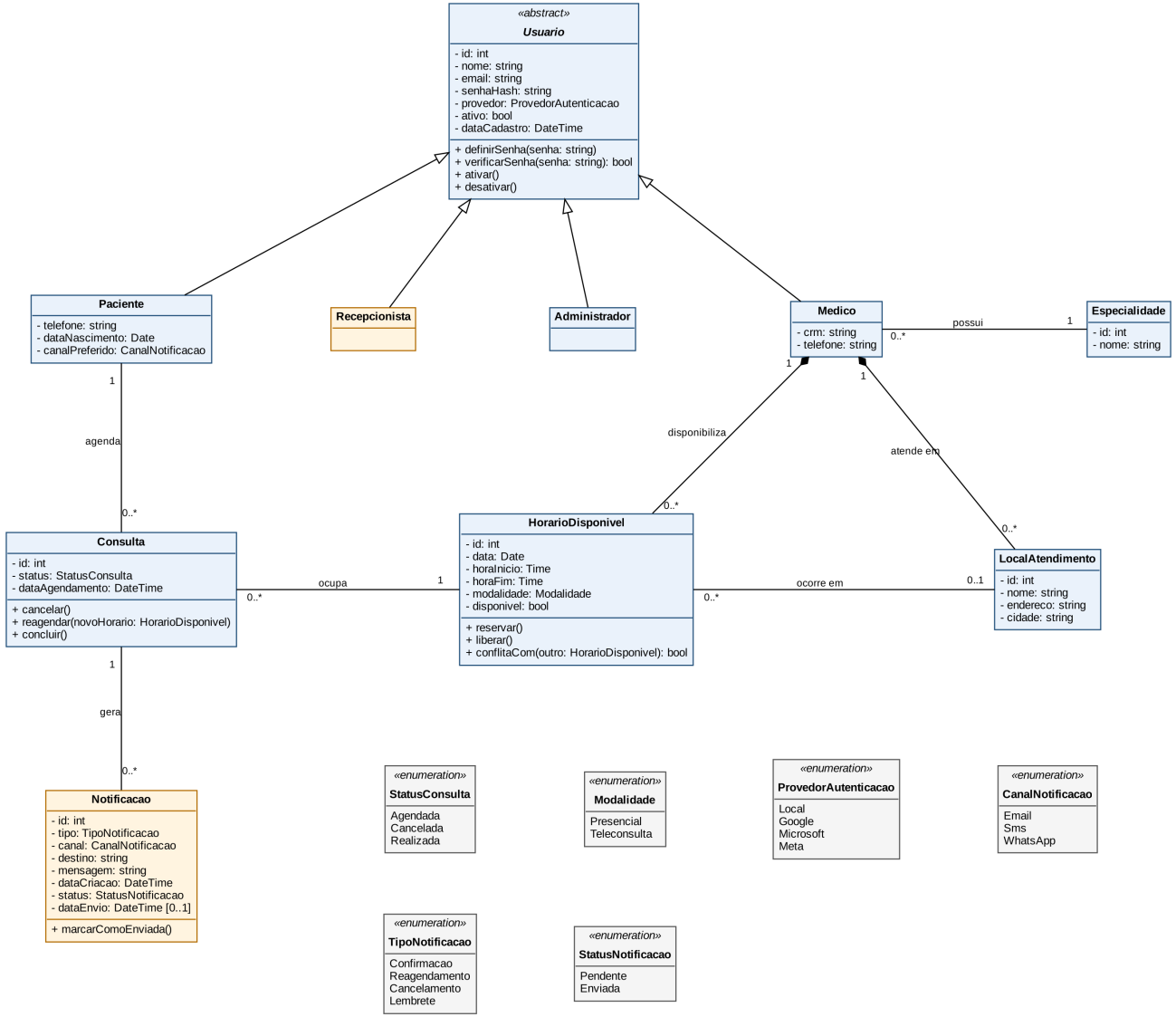


Figura 2 — 4.6 Diagrama de classes.

5. Modelagem Dinâmica

Os diagramas de sequência mostram a troca de mensagens entre atores e objetos em cada caso de uso; os diagramas de comunicação mostram as mesmas interações com foco nas ligações entre os objetos. Os nomes das mensagens correspondem aos métodos implementados.

5.1 Diagramas de sequência

UC01 — Criar conta

O cadastro só é concluído se o e-mail ainda não estiver em uso. A senha é convertida em hash pelo próprio objeto Paciente.

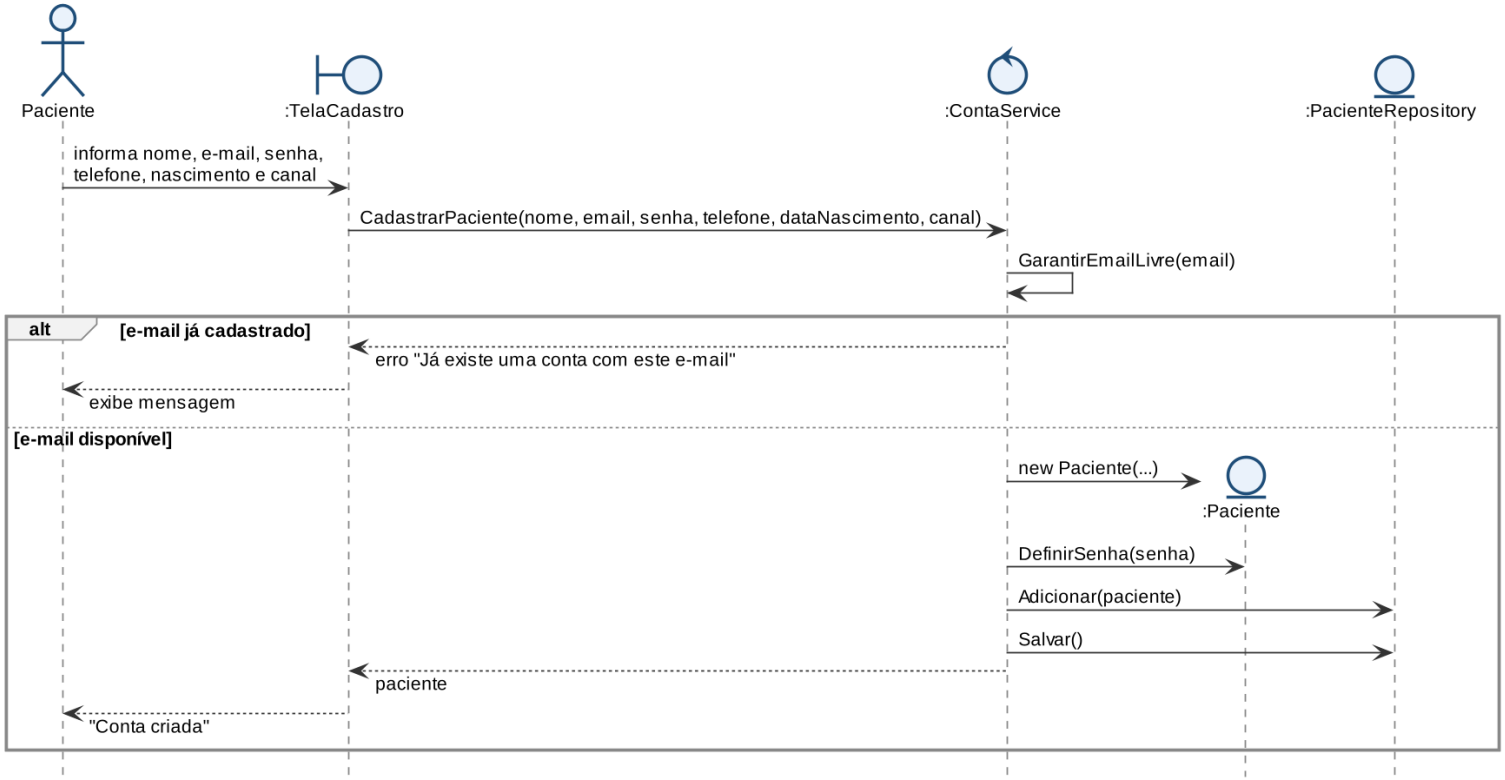


Figura 3 — UC01 — Criar conta.

UC02 — Realizar login

O fluxo alternativo com o sistema de autenticação SSO está previsto no modelo; nesta etapa o sistema identifica contas externas e orienta o usuário.

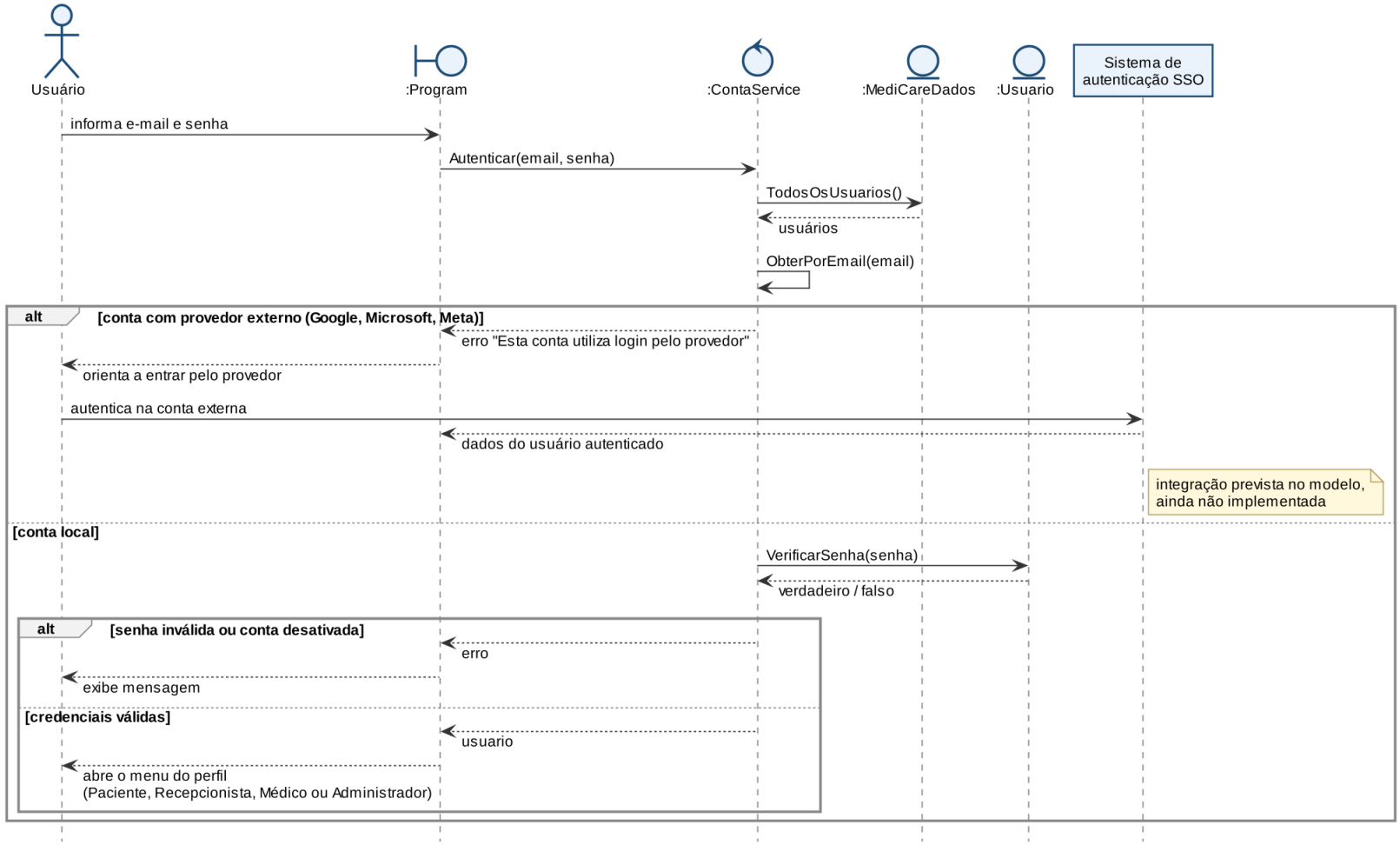


Figura 4 — UC02 — Realizar login.

UC03 e UC04 — Pesquisar médicos e consultar horários

Os dois casos de uso acontecem em sequência e usam o mesmo serviço.

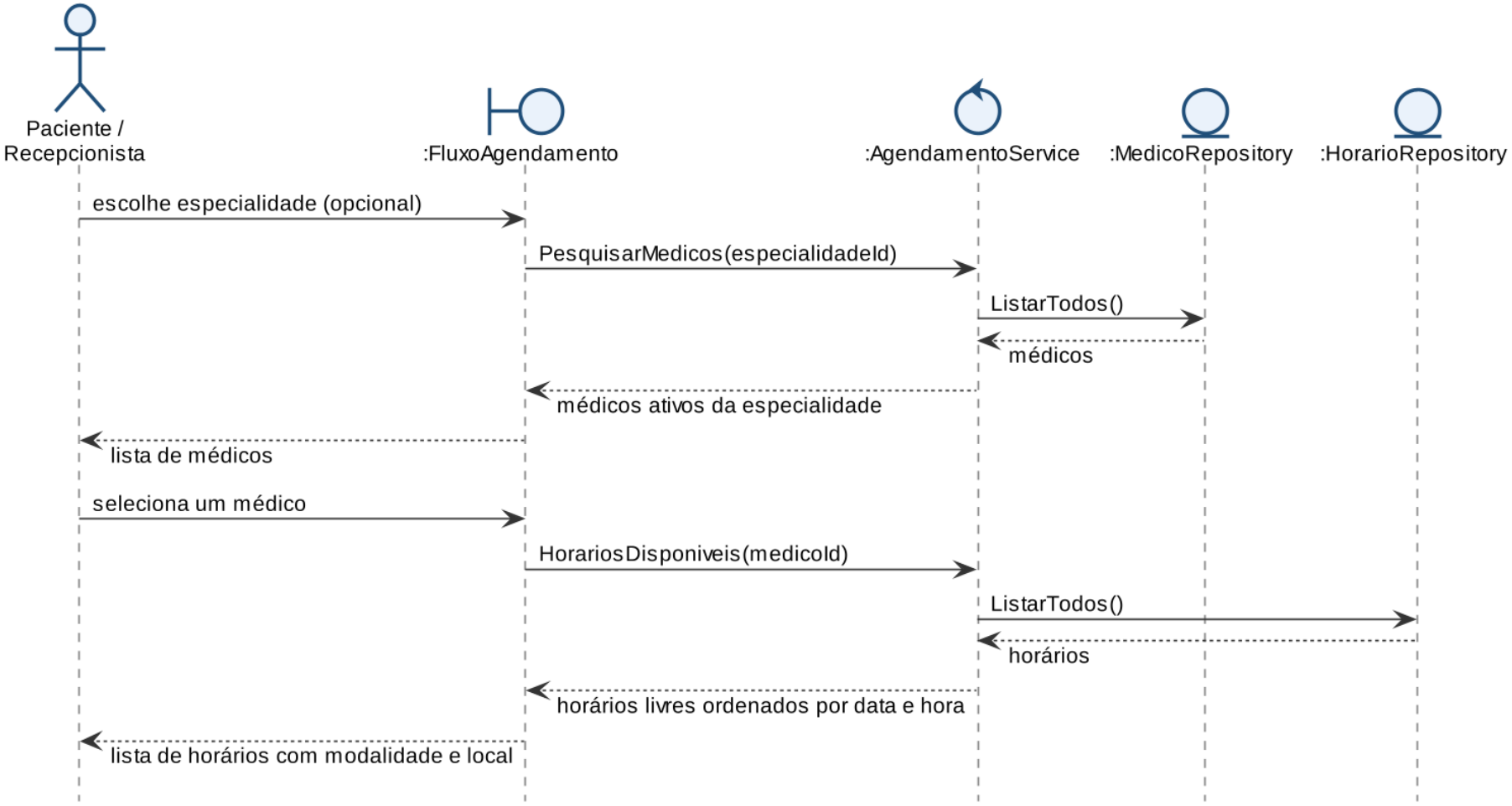


Figura 5 — UC03 e UC04 — Pesquisar médicos e consultar horários.

UC05 — Agendar consulta

A reserva do horário é feita antes de criar a consulta. Se o horário já estiver reservado, nada é gravado.

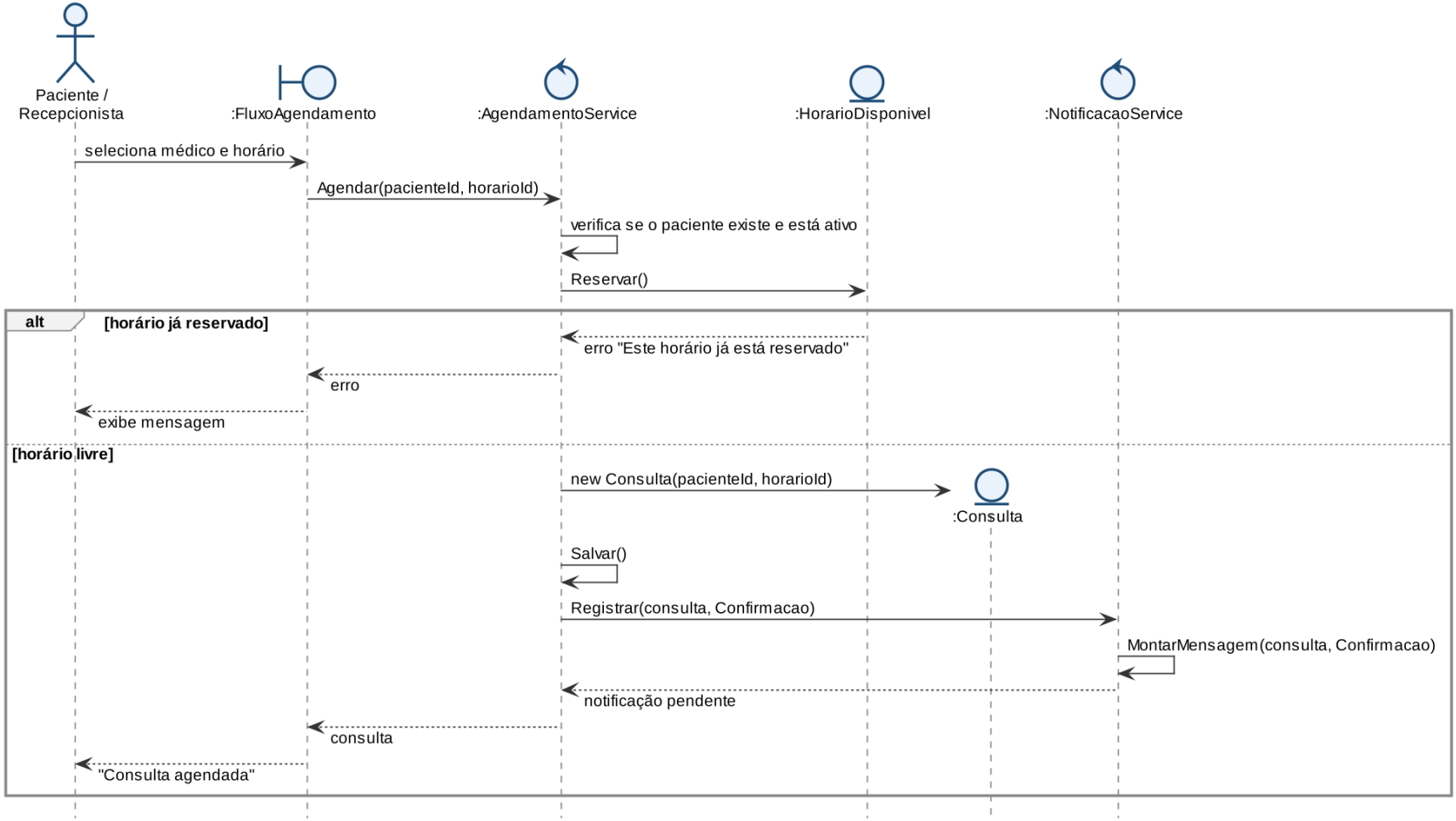


Figura 6 — UC05 — Agendar consulta.

UC06 — Cancelar consulta

A tela só permite escolher entre as consultas agendadas do próprio paciente.

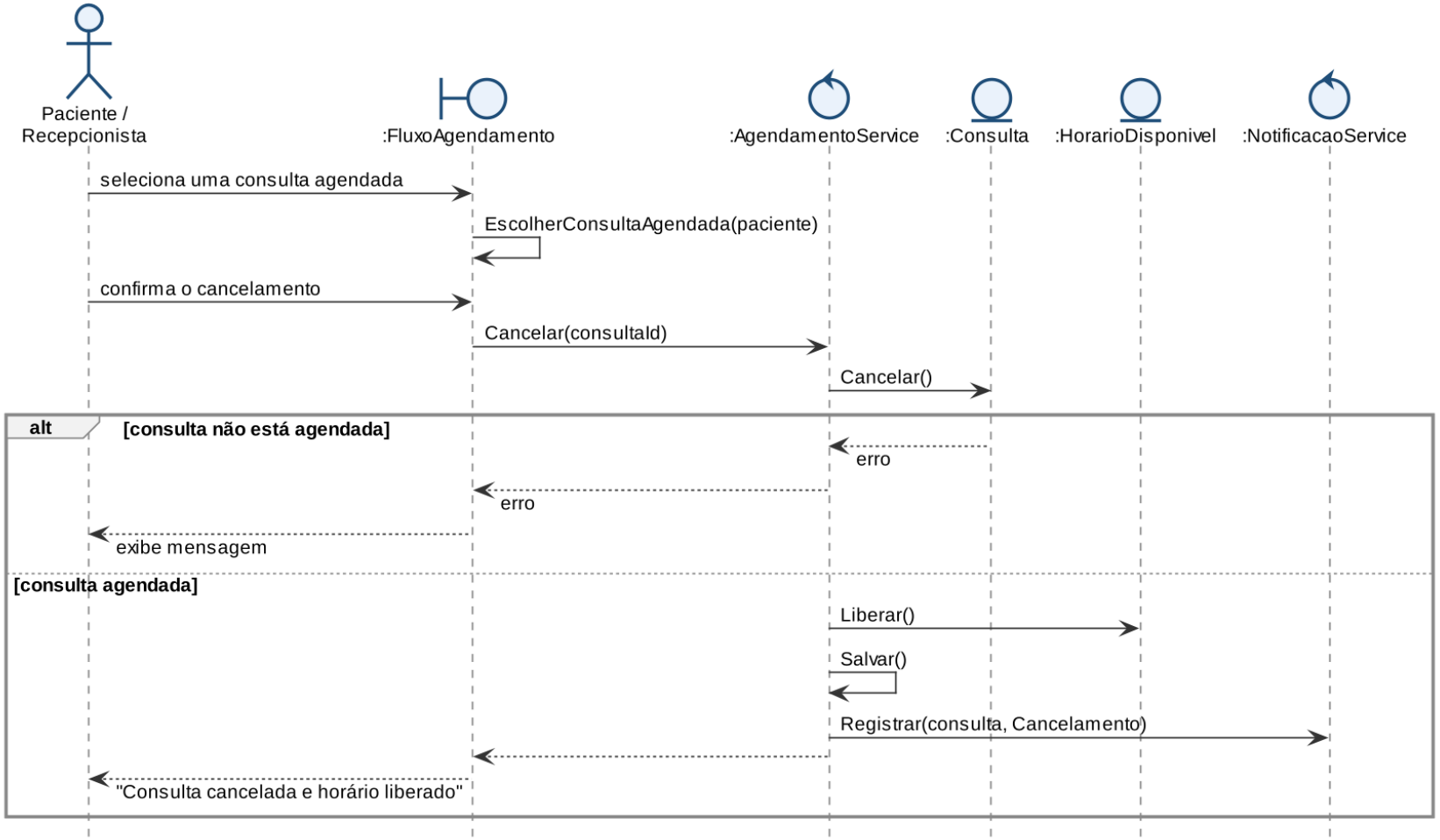


Figura 7 — UC06 — Cancelar consulta.

UC07 — Reagendar consulta

O novo horário é validado antes de qualquer alteração, para que a consulta não fique em estado inconsistente.

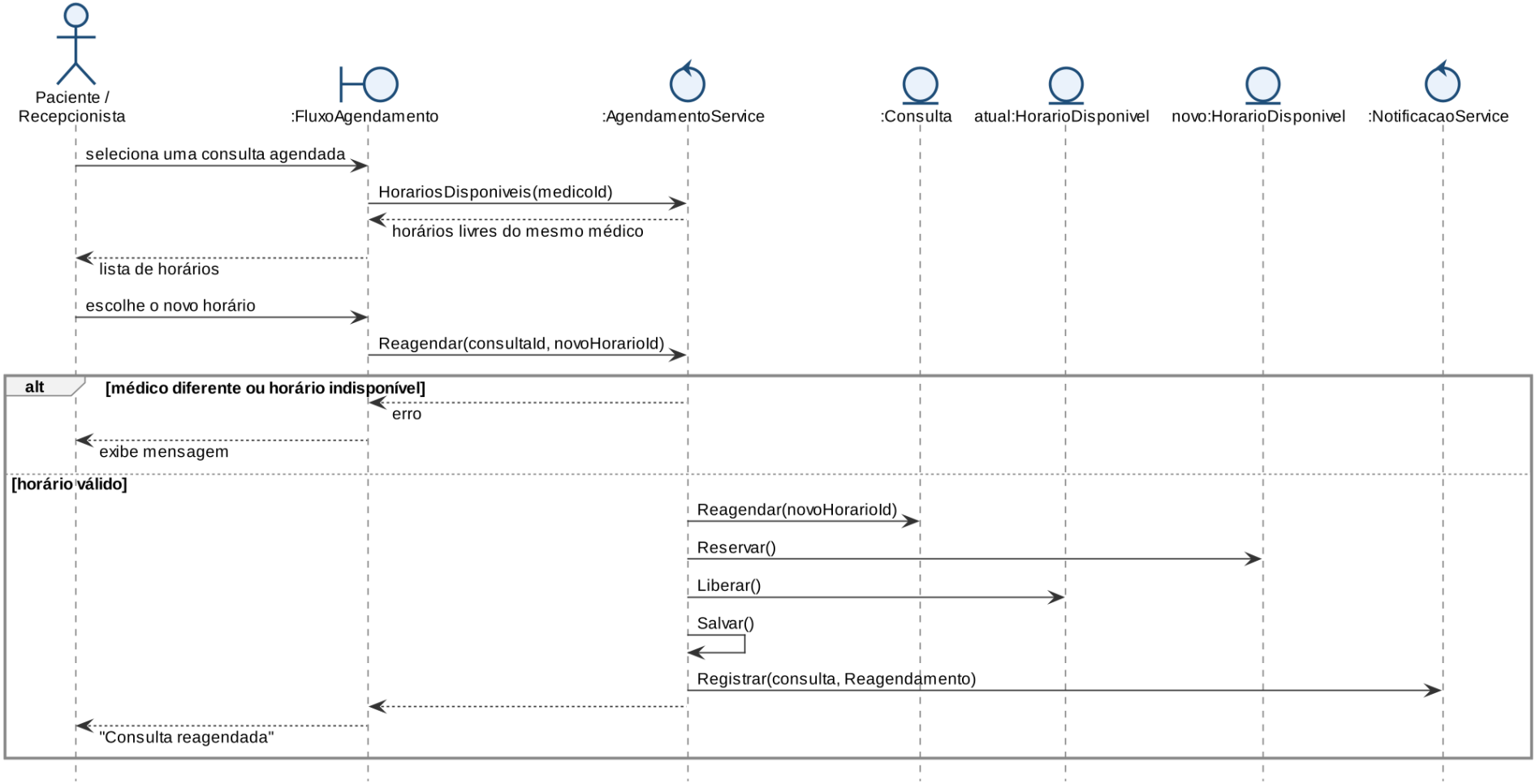


Figura 8 — UC07 — Reagendar consulta.

UC08 — Gerenciar perfil

O paciente atualiza telefone e canal de notificação; o médico atualiza o telefone pelo método equivalente.

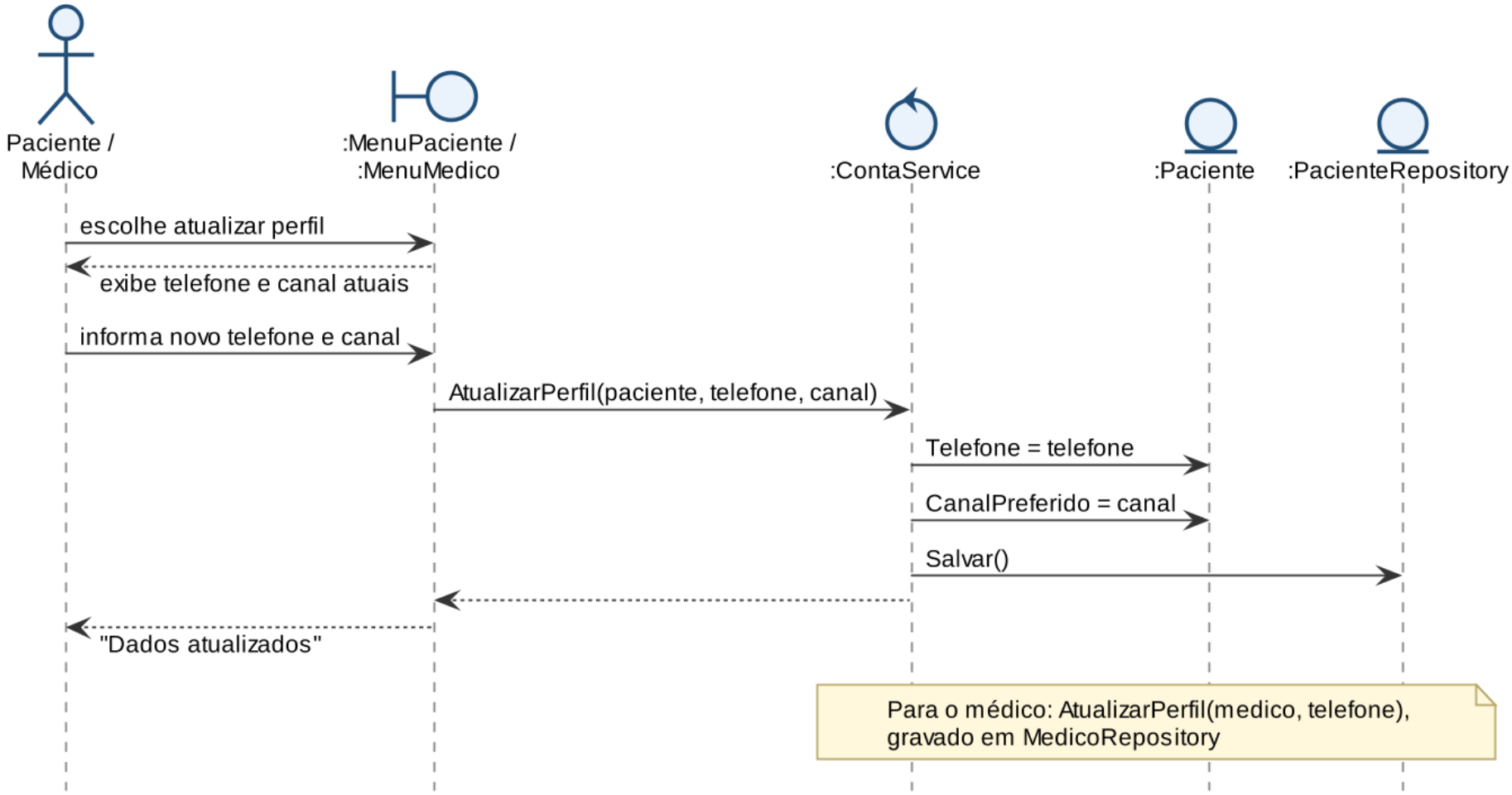


Figura 9 — UC08 — Gerenciar perfil.

UC09 — Gerenciar disponibilidade

Cada novo horário é comparado com os horários já cadastrados do médico pelo método ConflitaCom().

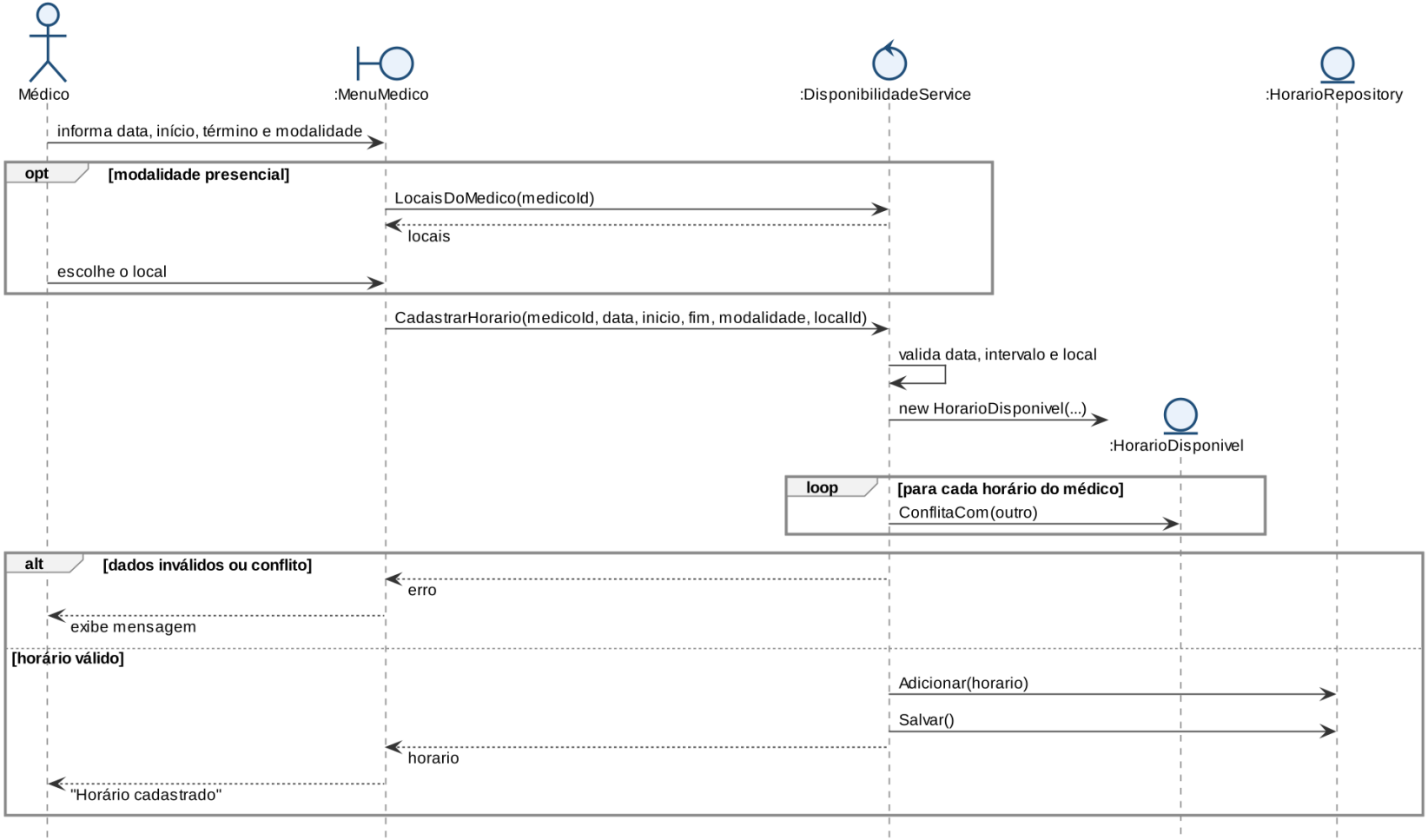


Figura 10 — UC09 — Gerenciar disponibilidade.

UC10 — Gerenciar agenda

Registrar a consulta como realizada é o que permite que ela apareça no histórico do paciente (UC15).

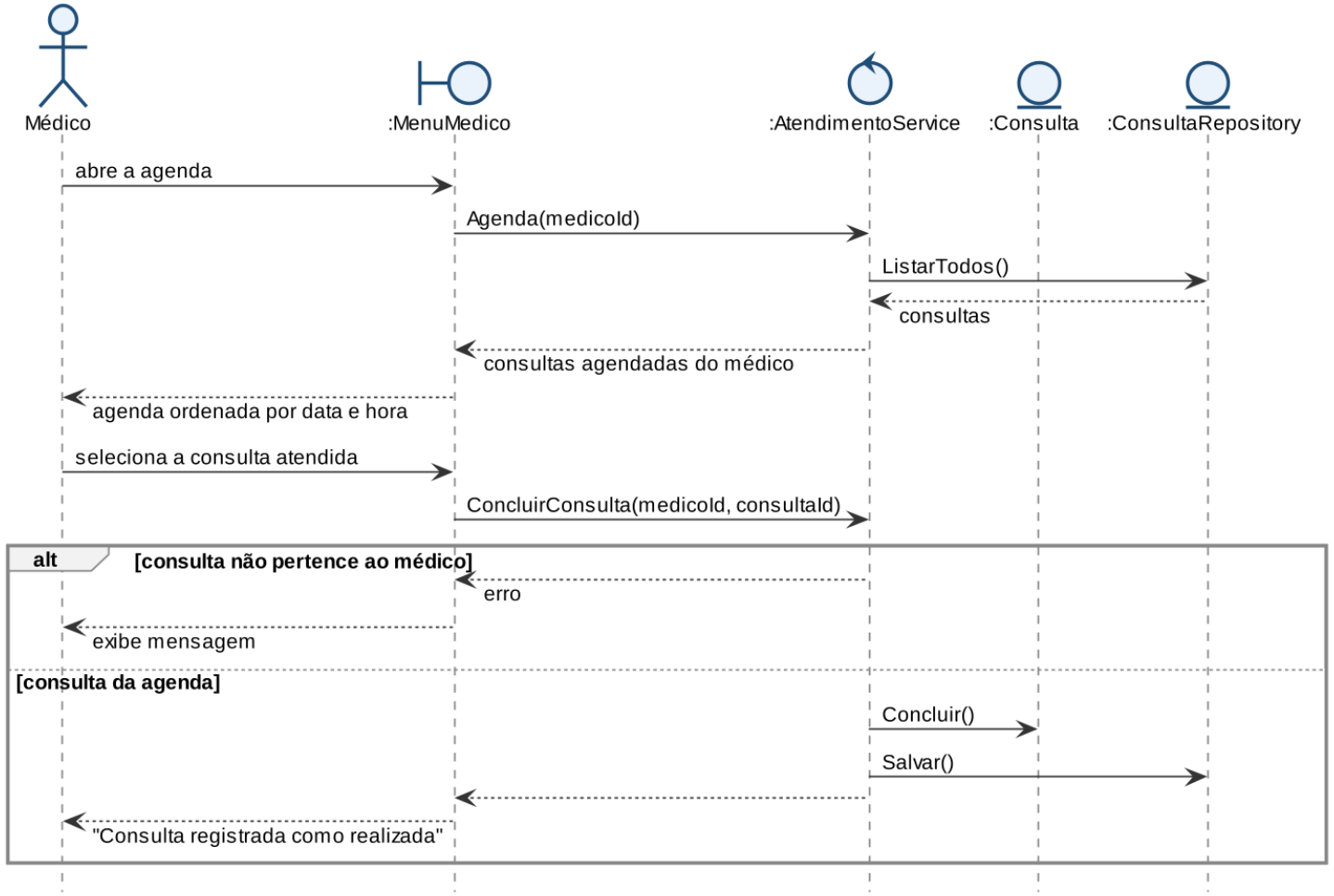


Figura 11 — UC10 — Gerenciar agenda.

UC11 — Realizar teleconsulta

O acesso é liberado somente ao paciente e ao médico da consulta, e somente dentro do horário agendado. A chamada de vídeo é feita pela plataforma de teleconsulta.

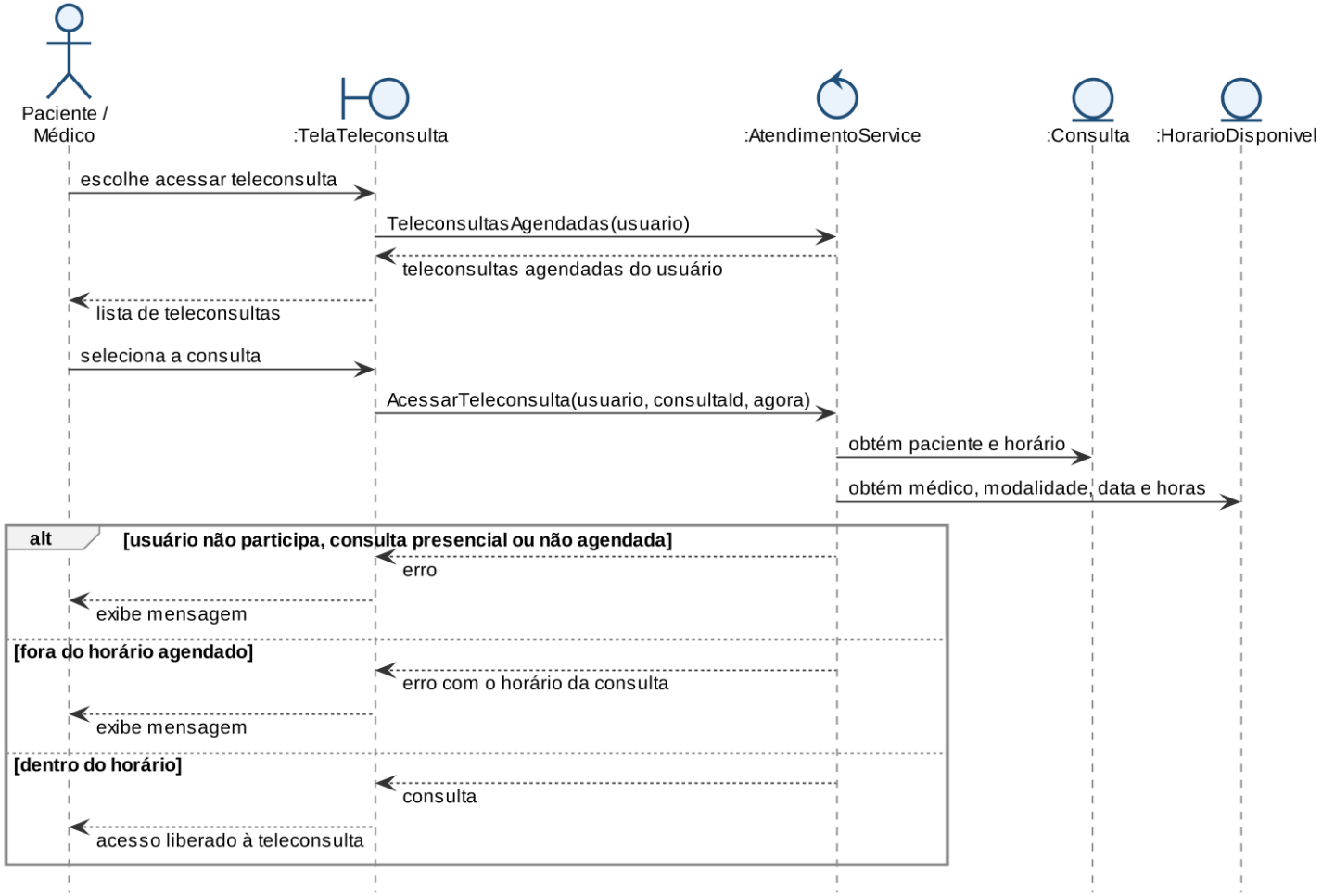


Figura 12 — UC11 — Realizar teleconsulta.

UC12 — Gerenciar usuários

A busca por e-mail percorre todos os tipos de usuário, e a gravação atualiza os arquivos de usuários.

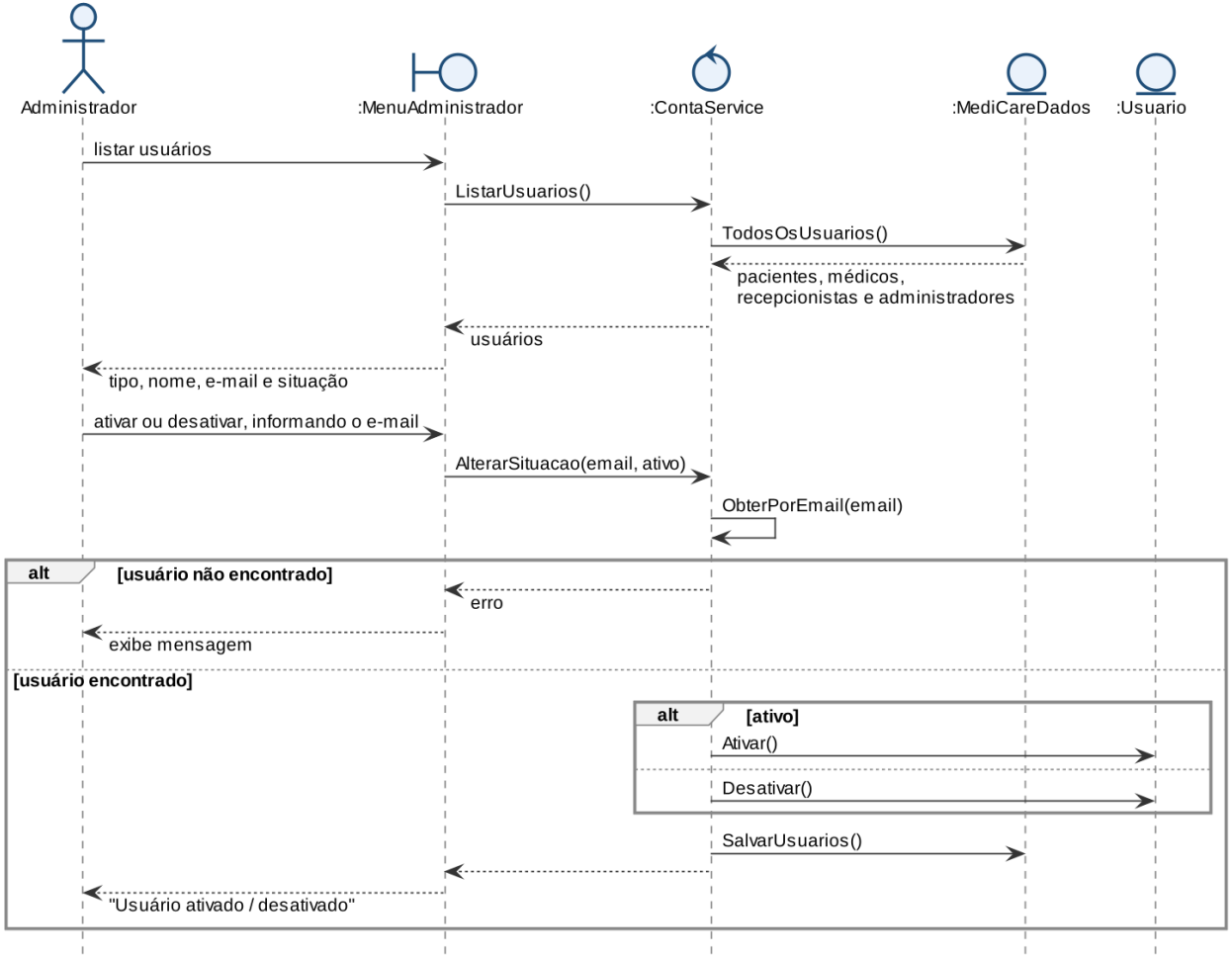


Figura 13 — UC12 — Gerenciar usuários.

UC13 — Gerenciar agendamentos

O cancelamento feito pelo administrador reaproveita o fluxo de UC06.

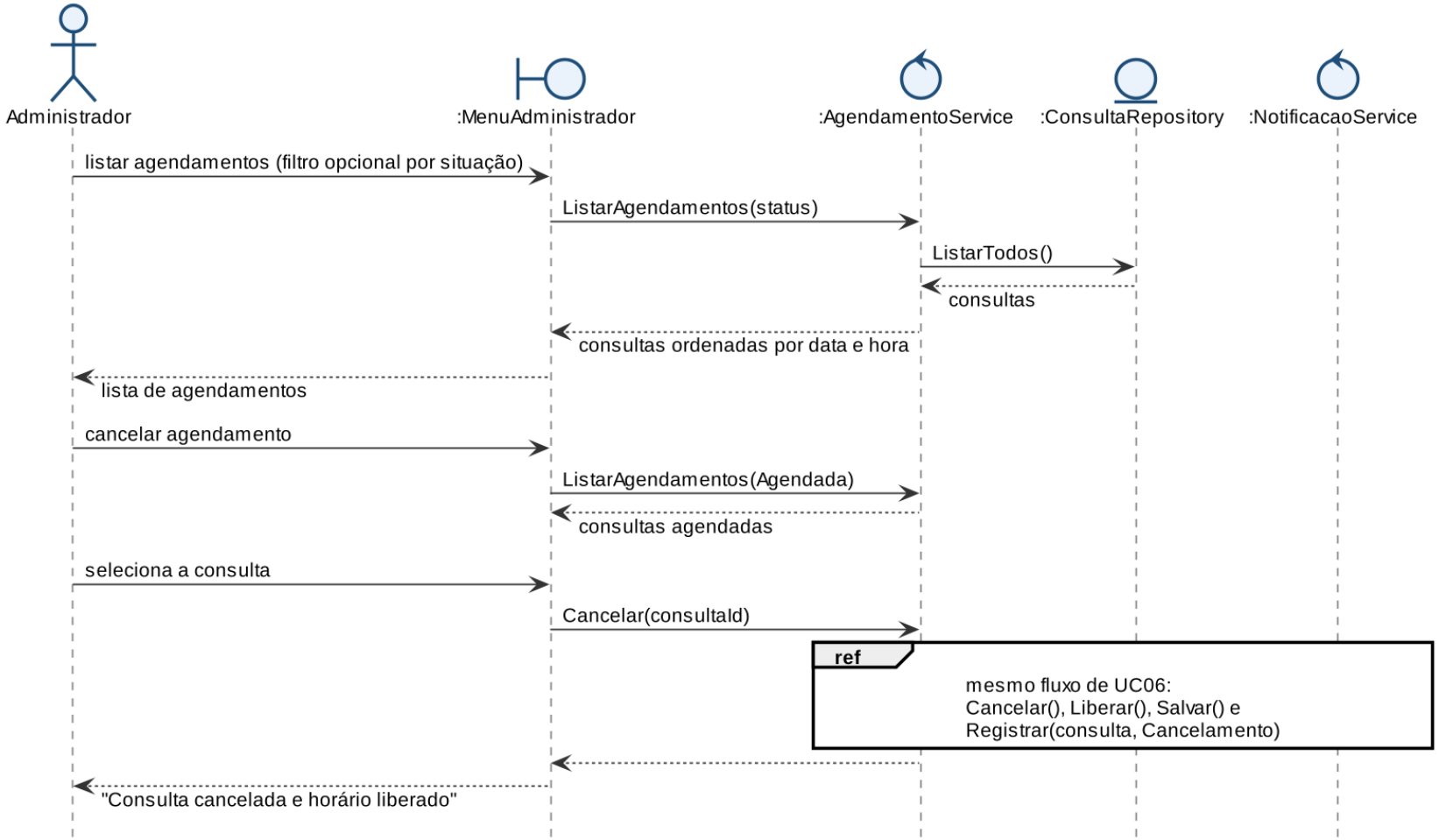


Figura 14 — UC13 — Gerenciar agendamentos.

UC14 — Enviar notificações

As notificações de confirmação, reagendamento e cancelamento são geradas nos UC05 a UC07; aqui são gerados os lembretes e feita a entrega ao sistema de notificação.

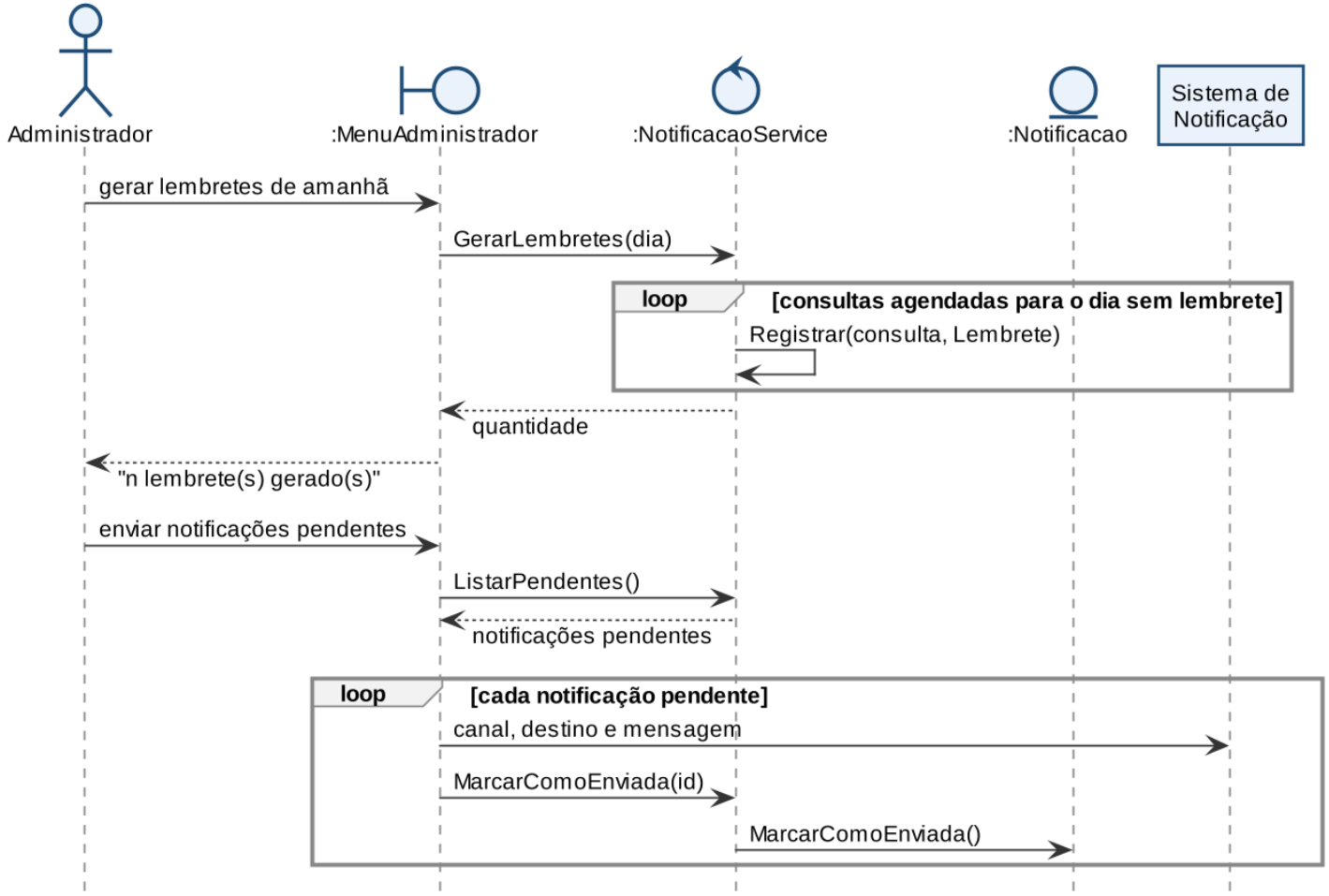


Figura 15 — UC14 — Enviar notificações.

UC15 — Consultar histórico do paciente

O médico só vê o histórico de pacientes que tenham ao menos uma consulta com ele.

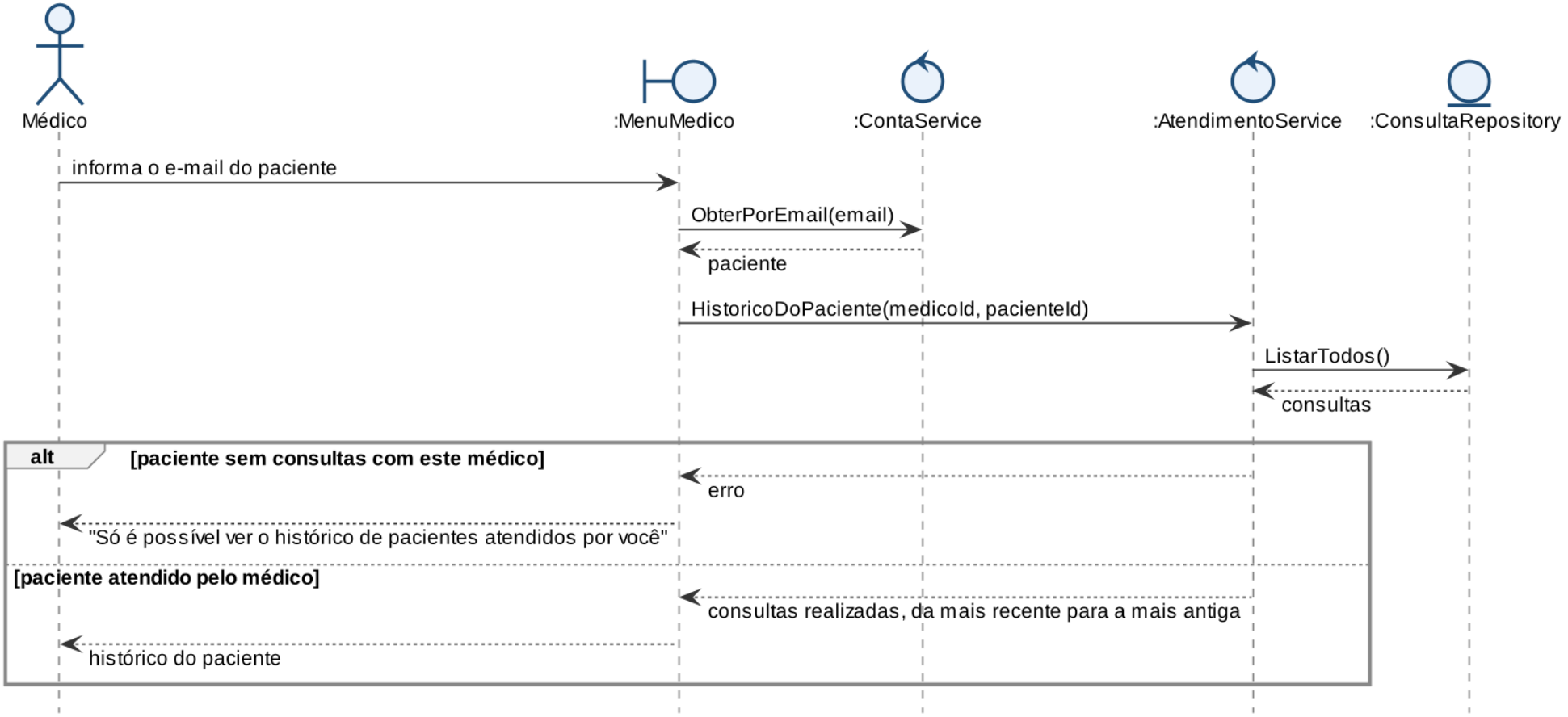


Figura 16 — UC15 — Consultar histórico do paciente.

5.2 Diagramas de comunicação

Cada linha liga dois objetos, e as mensagens trocadas por ela aparecem numeradas na ordem em que acontecem. “→” indica a mensagem do objeto de cima para o de baixo e “←” o retorno; “*” indica repetição.

UC02 — Realizar login (comunicação)

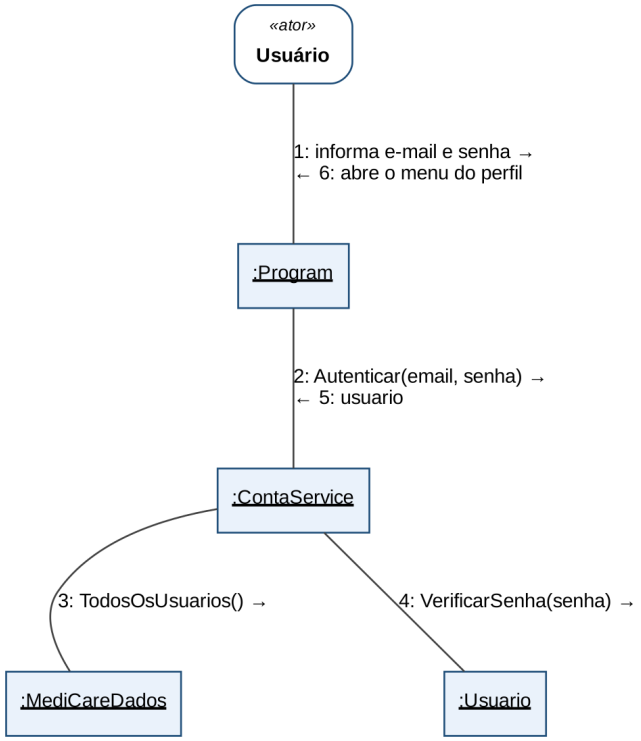


Figura 17 — UC02 — Realizar login (comunicação).

UC05 — Agendar consulta (comunicação)

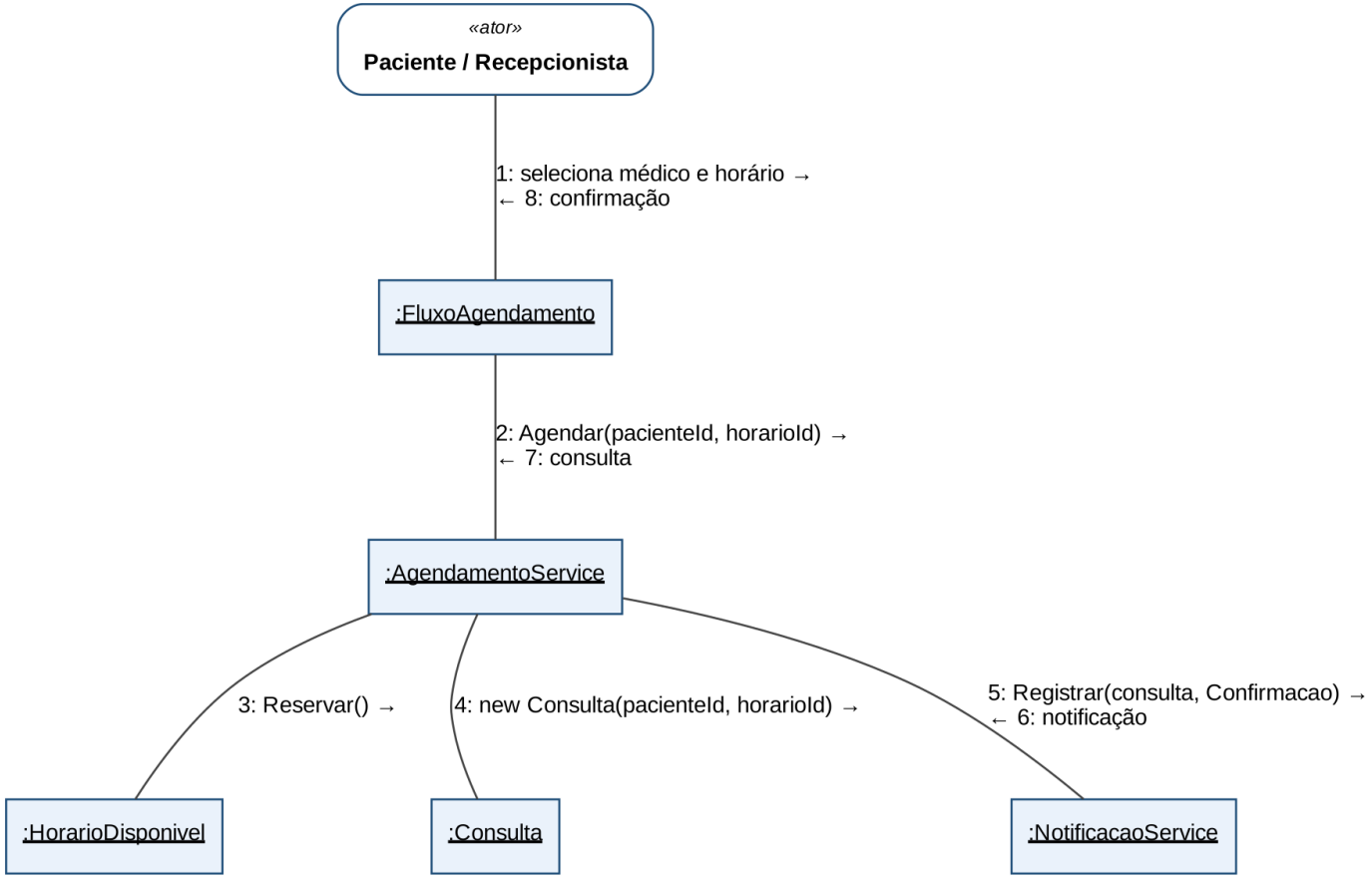


Figura 18 — UC05 — Agendar consulta (comunicação).

UC06 — Cancelar consulta (comunicação)

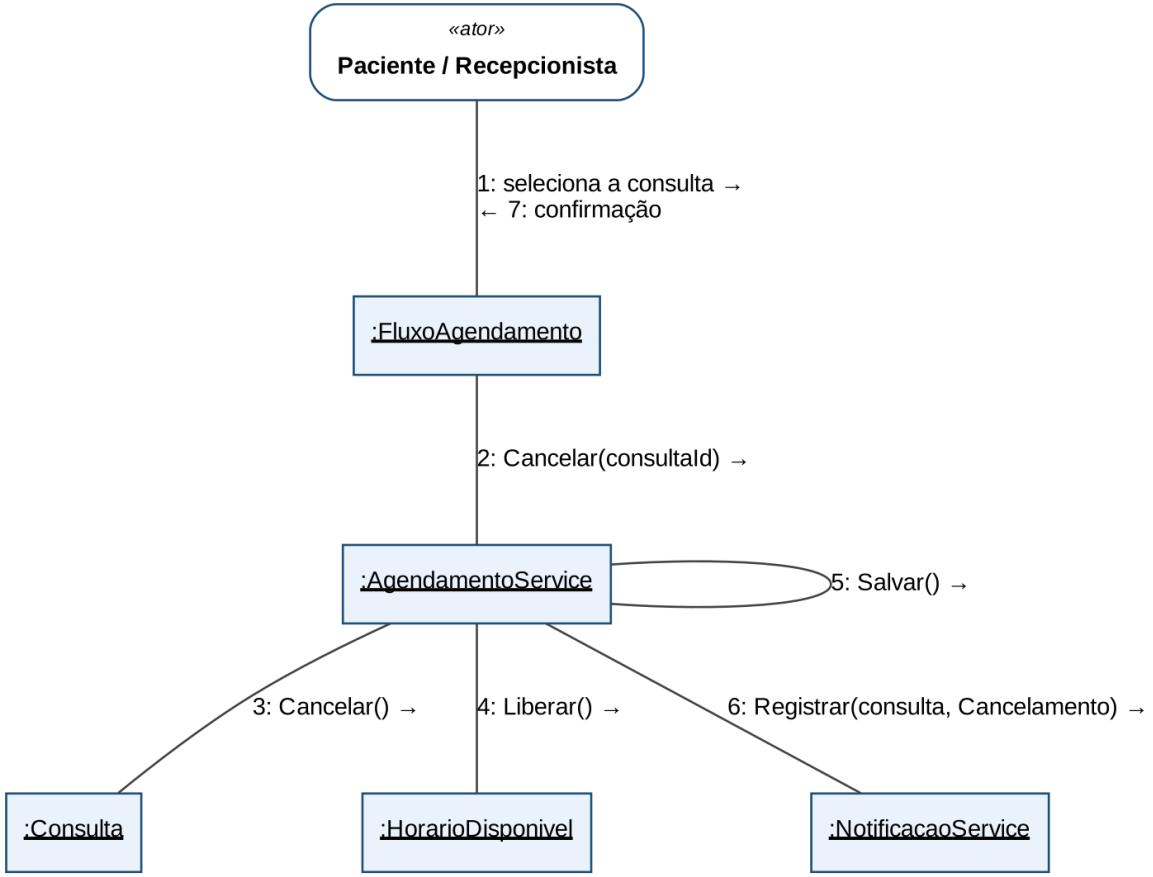


Figura 19 — UC06 — Cancelar consulta (comunicação).

UC07 — Reagendar consulta (comunicação)

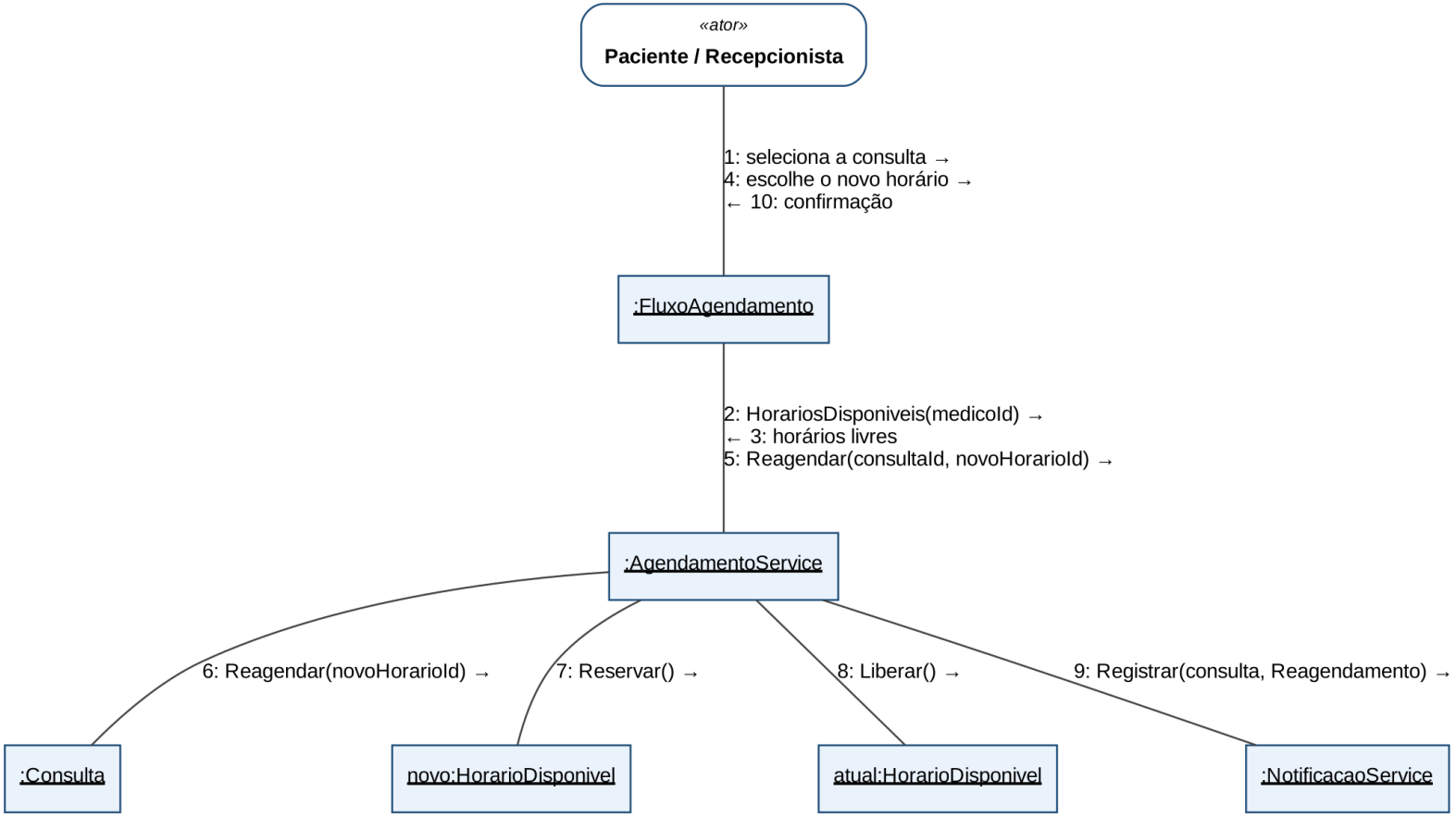


Figura 20 — UC07 — Reagendar consulta (comunicação).

UC09 — Gerenciar disponibilidade (comunicação)

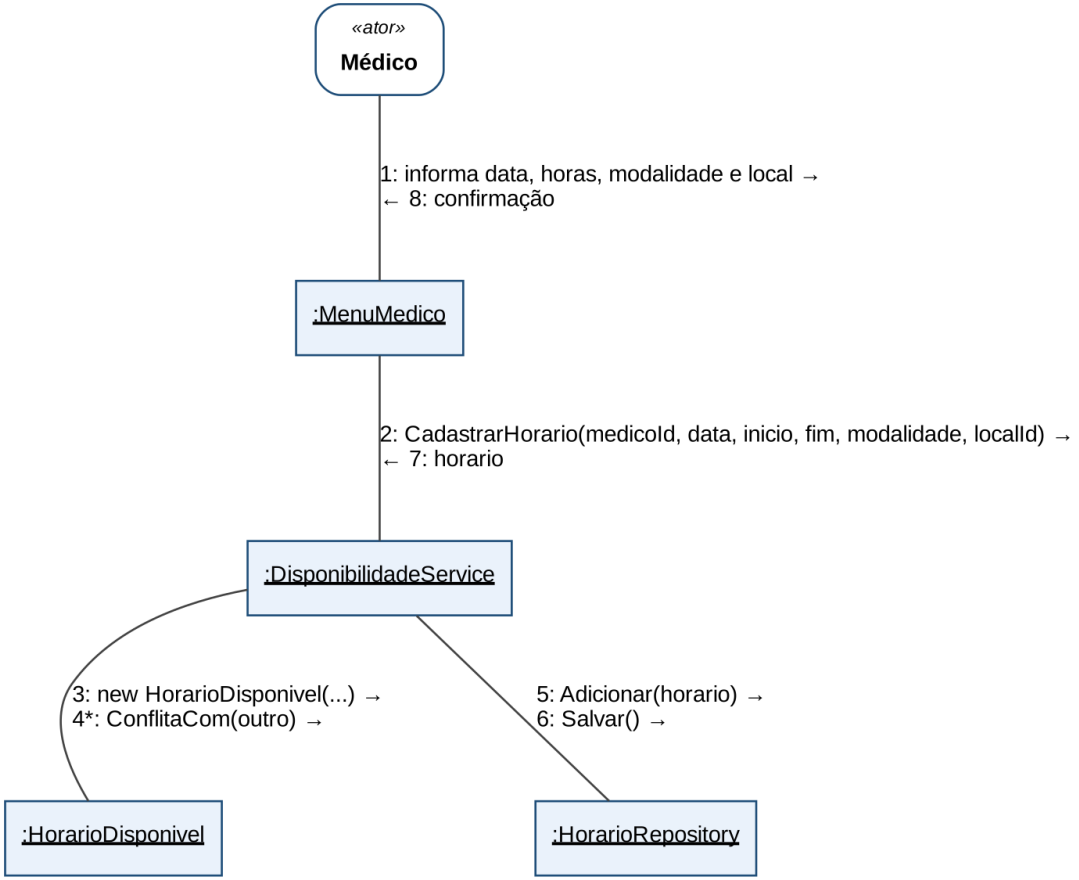


Figura 21 — UC09 — Gerenciar disponibilidade (comunicação).

UC11 — Realizar teleconsulta (comunicação)

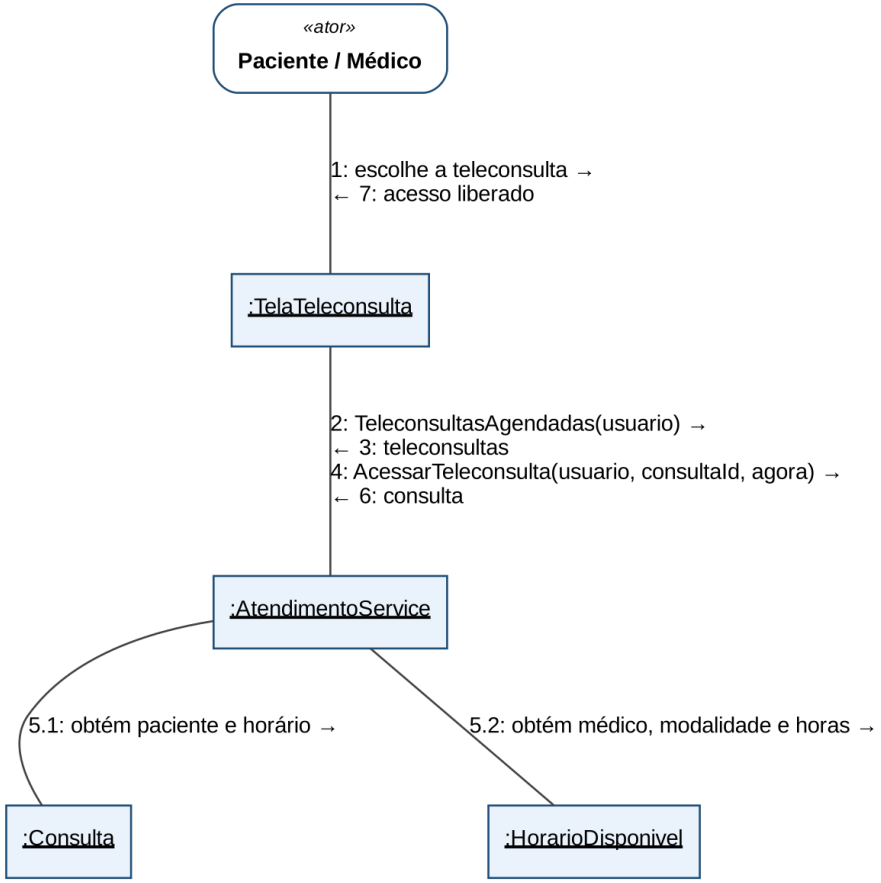


Figura 22 — UC11 — Realizar teleconsulta (comunicação).

UC14 — Enviar notificações (comunicação)

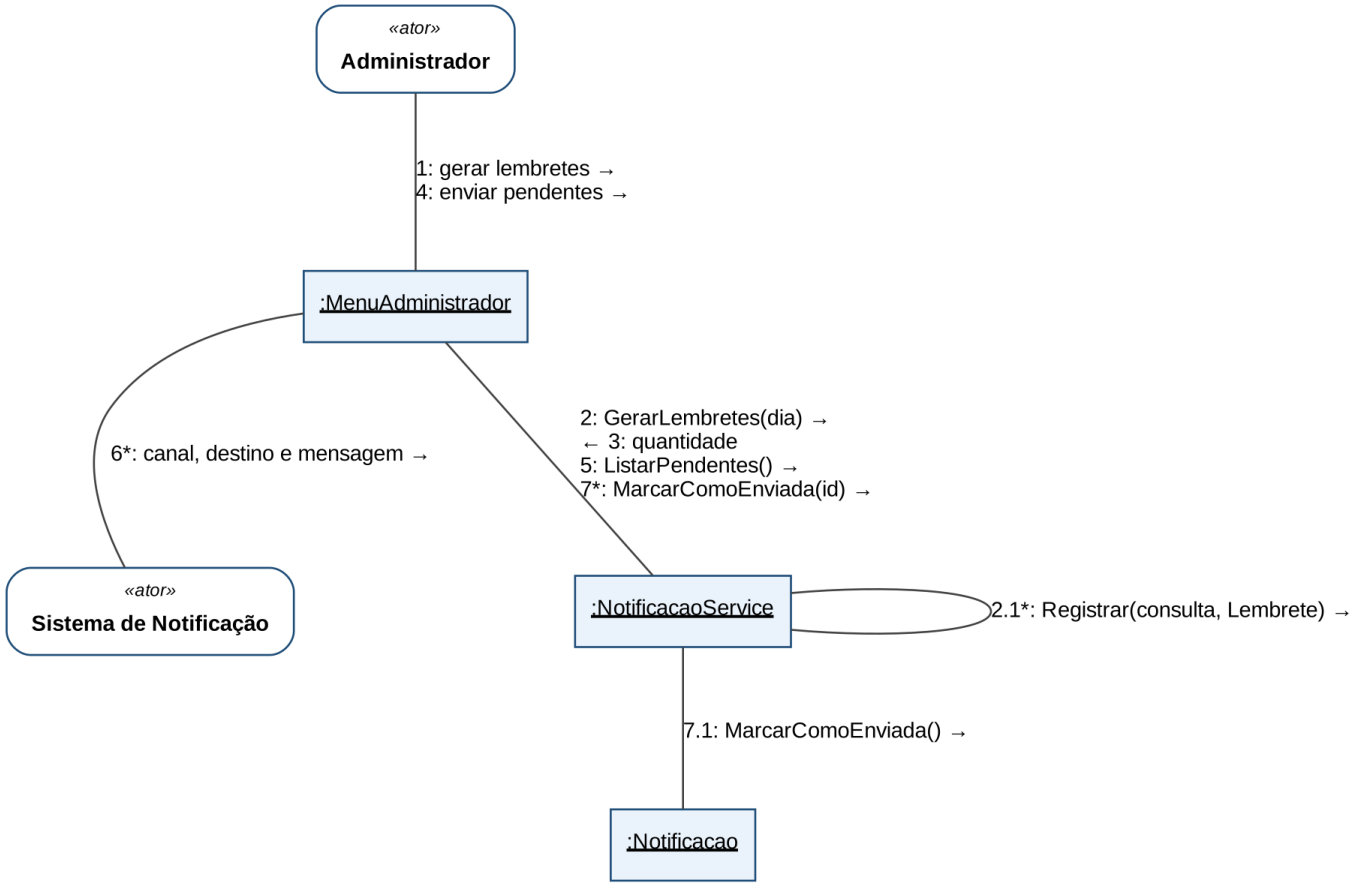


Figura 23 — UC14 — Enviar notificações (comunicação).

UC15 — Consultar histórico do paciente (comunicação)

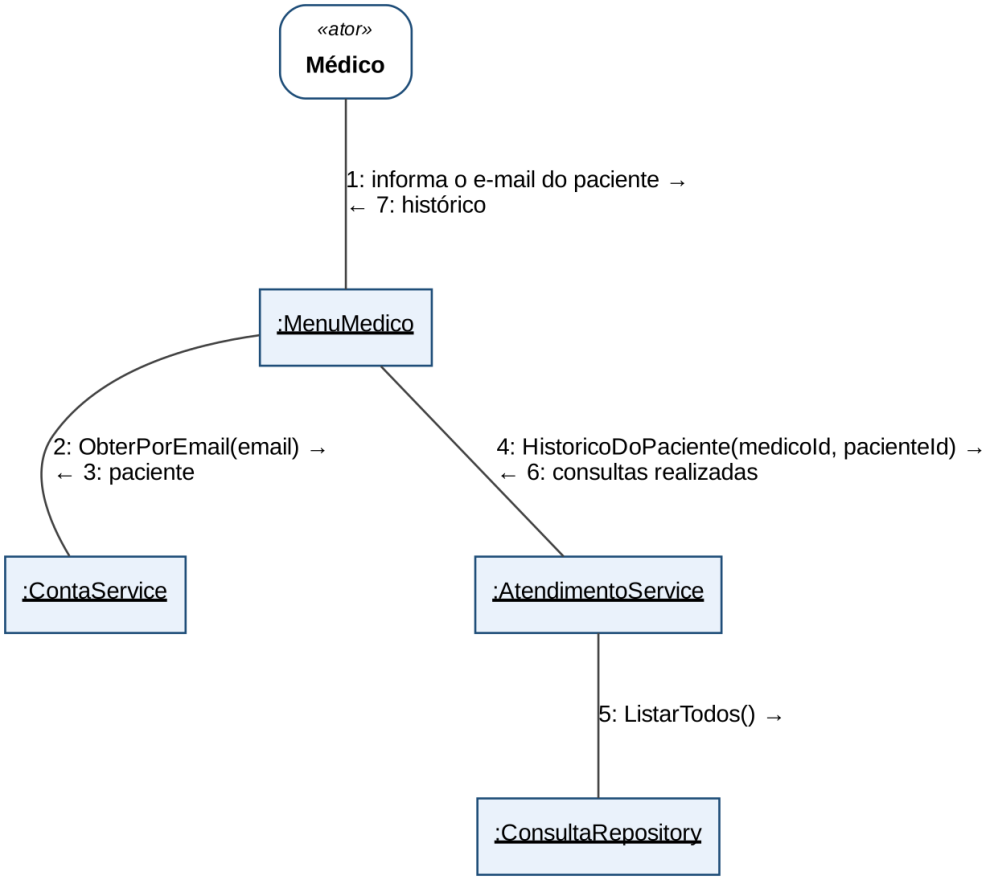


Figura 24 — UC15 — Consultar histórico do paciente (comunicação).

5.3 Métodos das classes

A partir dos diagramas, foram definidos os métodos das classes de entidade e das classes de serviço, que concentram as regras de cada grupo de casos de uso.

Classes de entidade

Usuario

Cabeçalho do método	Finalidade
`void DefinirSenha(string senha)`	Gera e guarda o hash da senha.
`bool VerificarSenha(string senha)`	Compara a senha informada; sempre falso para contas externas.
`void Ativar()`	Reativa a conta (UC12).
`void Desativar()`	Impede o acesso da conta (UC12).

HorarioDisponivel

Cabeçalho do método	Finalidade
`void Reservar()`	Marca o horário como ocupado; erro se já estiver reservado.
`void Liberar()`	Torna o horário disponível novamente.
`bool ConflitaCom(HorarioDisponivel outro)`	Verifica sobreposição com outro horário do mesmo médico no mesmo dia.

Consulta

Cancelar, Reagendar e Concluir só são aceitos quando a consulta está agendada.

Cabeçalho do método	Finalidade
`Consulta(int pacienteId, int horarioId)`	Cria a consulta com status Agendada.
`void Cancelar()`	Altera o status para Cancelada.
`void Reagendar(int novoHorarioId)`	Troca o horário da consulta.
`void Concluir()`	Altera o status para Realizada.

Notificacao

Cabeçalho do método	Finalidade
`Notificacao(int consultaId, TipoNotificacao tipo, CanalNotificacao canal, string destino, string mensagem)`	Cria a notificação com status Pendente.
`void MarcarComoEnviada()`	Registra o envio e a data; erro se já tiver sido enviada.

Paciente, Medico, Recepcionista, Administrador, Especialidade e LocalAtendimento possuem apenas atributos.

Classes de serviço

ContaService

Cabeçalho do método	Finalidade
`Paciente CadastrarPaciente(string nome, string email, string senha, string telefone, DateOnly dataNascimento, CanalNotificacao canal)`	UC01 para pacientes.
`Medico CadastrarMedico(string nome, string email, string senha, string crm, string telefone, int especialidadeId)`	UC01 para médicos.
`Usuario Autenticar(string email, string senha)`	UC02.
`void AtualizarPerfil(Paciente paciente, string telefone, CanalNotificacao canal)`	UC08 — paciente.
`void AtualizarPerfil(Medico medico, string telefone)`	UC08 — médico.
`Usuario? ObterPorEmail(string email)`	Localiza um usuário de qualquer tipo.
`IEnumerable<Usuario> ListarUsuarios()`	UC12.
`void AlterarSituacao(string email, bool ativo)`	UC12.

AgendamentoService

Cabeçalho do método	Finalidade
`IEnumerable<Medico> PesquisarMedicos(int? especialidadeId)`	UC03.
`IEnumerable<HorarioDisponivel> HorariosDisponiveis(int medicoId)`	UC04.
`IEnumerable<Consulta> ConsultasDoPaciente(int pacienteId)`	Consultas de um paciente.
`Consulta Agendar(int pacienteId, int horarioId)`	UC05.
`void Cancelar(int consultaId)`	UC06.
`void Reagendar(int consultaId, int novoHorarioId)`	UC07.
`IEnumerable<Consulta> ListarAgendamentos(StatusConsulta? status)`	UC13 — lista com filtro opcional.

DisponibilidadeService

Cabeçalho do método	Finalidade
`IEnumerable<LocalAtendimento> LocaisDoMedico(int medicoId)`	Locais para horários presenciais.
`IEnumerable<HorarioDisponivel> HorariosDoMedico(int medicoId)`	Todos os horários do médico.
`HorarioDisponivel CadastrarHorario(int medicoId, DateOnly data, TimeOnly inicio, TimeOnly fim, Modalidade modalidade, int? localId)`	UC09 — cadastro com validações.
`void RemoverHorario(int medicoId, int horarioId)`	UC09 — remoção de horário não utilizado.

AtendimentoService

Cabeçalho do método	Finalidade
`IEnumerable<Consulta> Agenda(int medicoId)`	UC10.
`void ConcluirConsulta(int medicoId, int consultaId)`	UC10 — registra atendimento.
`IEnumerable<Consulta> HistoricoDoPaciente(int medicoId, int pacienteId)`	UC15.
`IEnumerable<Consulta> TeleconsultasAgendadas(Usuario usuario)`	UC11 — teleconsultas do usuário.
`Consulta AcessarTeleconsulta(Usuario usuario, int consultaId, DateTime agora)`	UC11 — libera o acesso.

NotificacaoService

Cabeçalho do método	Finalidade
`Notificacao Registrar(Consulta consulta, TipoNotificacao tipo)`	Cria a notificação no canal do paciente.
`int GerarLembretes(DateOnly dia)`	UC14 — lembretes das consultas do dia.
`IEnumerable<Notificacao> ListarPendentes()`	UC14 — notificações a enviar.
`void MarcarComoEnviada(int notificacaoId)`	UC14 — confirma a entrega.

As classes de persistência (IRepositorio, CsvRepository, EfRepositorio e MediCareDbContext) estão descritas na seção 6.2.

---

6. Implementação

6.1 Arquitetura

A solução foi desenvolvida em C# com .NET 8 e organizada em quatro projetos. As regras de negócio ficam separadas da interface e da persistência; por isso foi possível evoluir de um programa de console com arquivos CSV (TP3 e TP4) para uma aplicação web com banco de dados (TP5) sem reescrever os serviços.

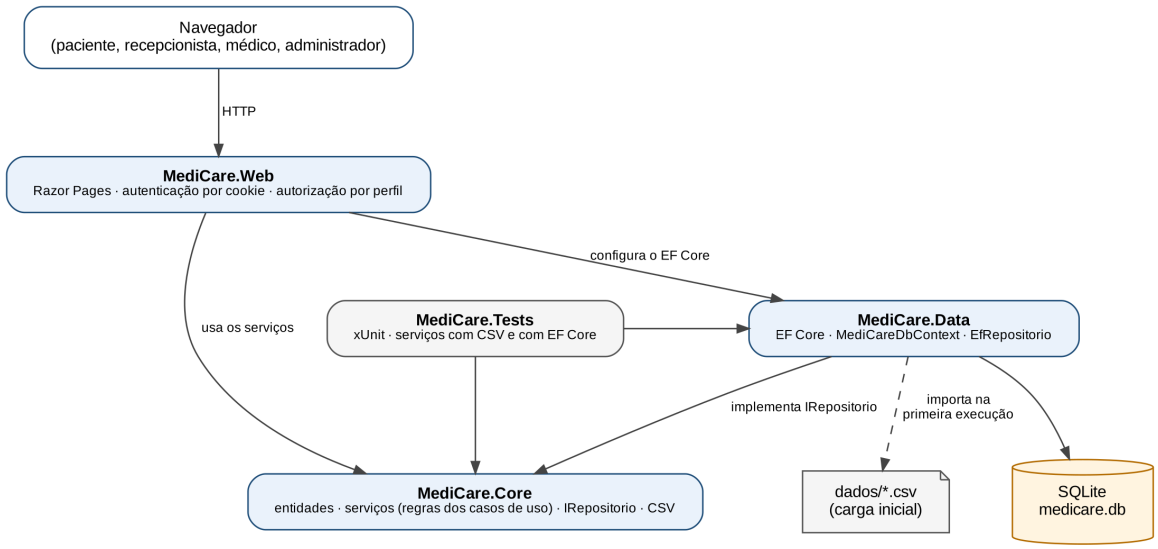


Figura 25 — Projetos da solução e dependências.

Projeto	Responsabilidade
MediCare.Core	Entidades, serviços com as regras dos casos de uso, interface IRepositorio e leitura e escrita de CSV.
MediCare.Data	Persistência com EF Core: MediCareDbContext, EfRepositorio e criação do banco.
MediCare.Web	Aplicação Razor Pages: telas, autenticação e autorização por perfil.
MediCare.Tests	Testes automatizados com xUnit.

6.2 Persistência

Na TP3 foi implementada a leitura e escrita dos dados em arquivos CSV, no formato RFC 4180, com um arquivo por entidade e validação do cabeçalho e dos valores. Na TP5, a persistência foi reimplementada com o **EF Core 8** e o provedor **SQLite**. Os serviços dependem apenas da interface IRepositorio, que tem as duas implementações; os arquivos CSV passaram a ser usados para preencher o banco na primeira execução e nos testes.

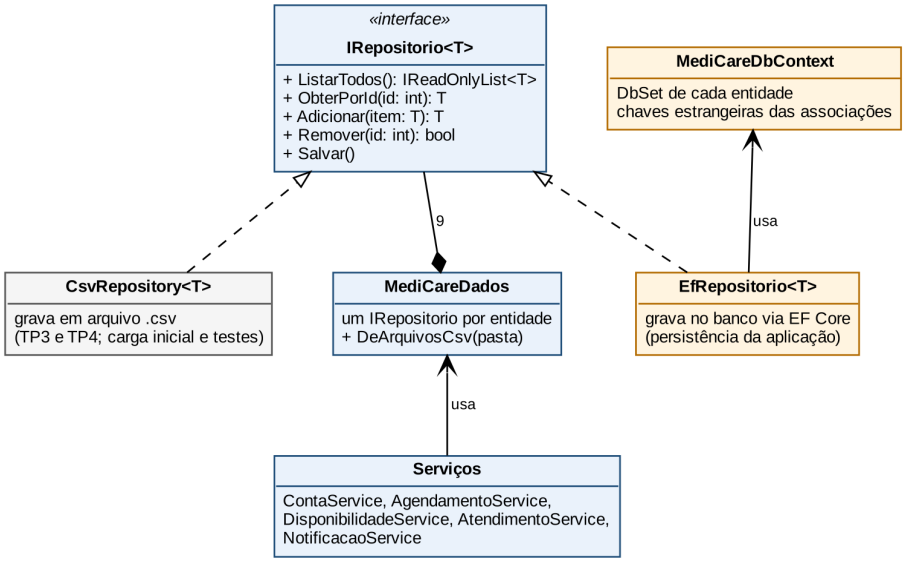


Figura 26 — Classes da camada de persistência.

Entidade	Tabela	Chaves estrangeiras
Especialidade	Especialidades	—
Paciente / Recepcionista / Administrador	Pacientes / Recepcionistas / Administradores	—
Medico	Medicos	EspecialidadeId → Especialidades
LocalAtendimento	LocaisAtendimento	MedicoId → Medicos
HorarioDisponivel	Horarios	MedicoId → Medicos; LocalId → LocaisAtendimento (opcional)
Consulta	Consultas	PacienteId → Pacientes; HorarioId → Horarios
Notificacao	Notificacoes	ConsultaId → Consultas

Mapeamento no contexto do banco:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Especialidade>().ToTable("Especialidades");
    modelBuilder.Entity<Paciente>().ToTable("Pacientes");
    modelBuilder.Entity<Recepcionista>().ToTable("Recepcionistas");
    modelBuilder.Entity<Administrador>().ToTable("Administradores");

    modelBuilder.Entity<Medico>().ToTable("Medicos")
        .HasOne<Especialidade>().WithMany().HasForeignKey(m => m.EspecialidadeId);

    modelBuilder.Entity<LocalAtendimento>().ToTable("LocaisAtendimento")
        .HasOne<Medico>().WithMany().HasForeignKey(l => l.MedicoId);

    var horario = modelBuilder.Entity<HorarioDisponivel>().ToTable("Horarios");
    horario.HasOne<Medico>().WithMany().HasForeignKey(h => h.MedicoId);
    horario.HasOne<LocalAtendimento>().WithMany().HasForeignKey(h => h.LocalId);

    var consulta = modelBuilder.Entity<Consulta>().ToTable("Consultas");
    consulta.HasOne<Paciente>().WithMany().HasForeignKey(c => c.PacienteId);
    consulta.HasOne<HorarioDisponivel>().WithMany().HasForeignKey(c => c.HorarioId);

    modelBuilder.Entity<Notificacao>().ToTable("Notificacoes")
        .HasOne<Consulta>().WithMany().HasForeignKey(n => n.ConsultaId);
}
```

Repositório EF Core usado pelos serviços:

```
public class EfRepositorio<T>(MediCareDbContext contexto) : IRepositorio<T> where T : class, IEntidade
{
    private readonly DbSet<T> _conjunto = contexto.Set<T>();

    public IReadOnlyList<T> ListarTodos()
    {
        // Local inclui entidades adicionadas e ainda não gravadas, como no repositório CSV.
        _conjunto.Load();
        return _conjunto.Local.OrderBy(i => i.Id).ToList();
    }

    public T? ObterPorId(int id) => _conjunto.Find(id);

    public T Adicionar(T item)
    {
        _conjunto.Add(item);
        return item;
    }

    public bool Remover(int id)
    {
        var item = ObterPorId(id);
        if (item is null)
            return false;
        _conjunto.Remove(item);
        return true;
    }

    public void Salvar() => contexto.SaveChanges();
}
```

6.3 Interface com Razor Pages

O usuário entra com e-mail e senha e recebe um cookie de autenticação com o seu perfil. Cada pasta de páginas só pode ser acessada pelos perfis autorizados.

Caso de uso	Página	Perfis
UC01 — Criar conta	Conta/CriarConta, Conta/CriarContaMedico	Visitante
UC02 — Realizar login	Conta/Entrar	Visitante
UC03 — Pesquisar médicos	Agendamento/Medicos	Paciente, Recepcionista
UC04 e UC05 — Horários e agendamento	Agendamento/Horarios	Paciente, Recepcionista
UC06 — Cancelar consulta	Agendamento/Consultas	Paciente, Recepcionista
UC07 — Reagendar consulta	Agendamento/Reagendar	Paciente, Recepcionista
UC08 — Gerenciar perfil	AreaPaciente/Perfil, AreaMedico/Perfil	Paciente, Médico
UC09 — Gerenciar disponibilidade	AreaMedico/Horarios	Médico
UC10 — Gerenciar agenda	AreaMedico/Index	Médico
UC11 — Realizar teleconsulta	Teleconsulta/Acessar	Paciente, Médico
UC12 — Gerenciar usuários	Admin/Usuarios	Administrador
UC13 — Gerenciar agendamentos	Admin/Agendamentos	Administrador
UC14 — Enviar notificações	Admin/Notificacoes	Administrador
UC15 — Consultar histórico	AreaMedico/Historico	Médico

A recepcionista escolhe o paciente atendido na tela da recepção e usa as mesmas páginas de agendamento do paciente. As telas a seguir foram capturadas da aplicação em funcionamento.



Figura 27 — Página inicial.



Figura 28 — Entrar (UC02).

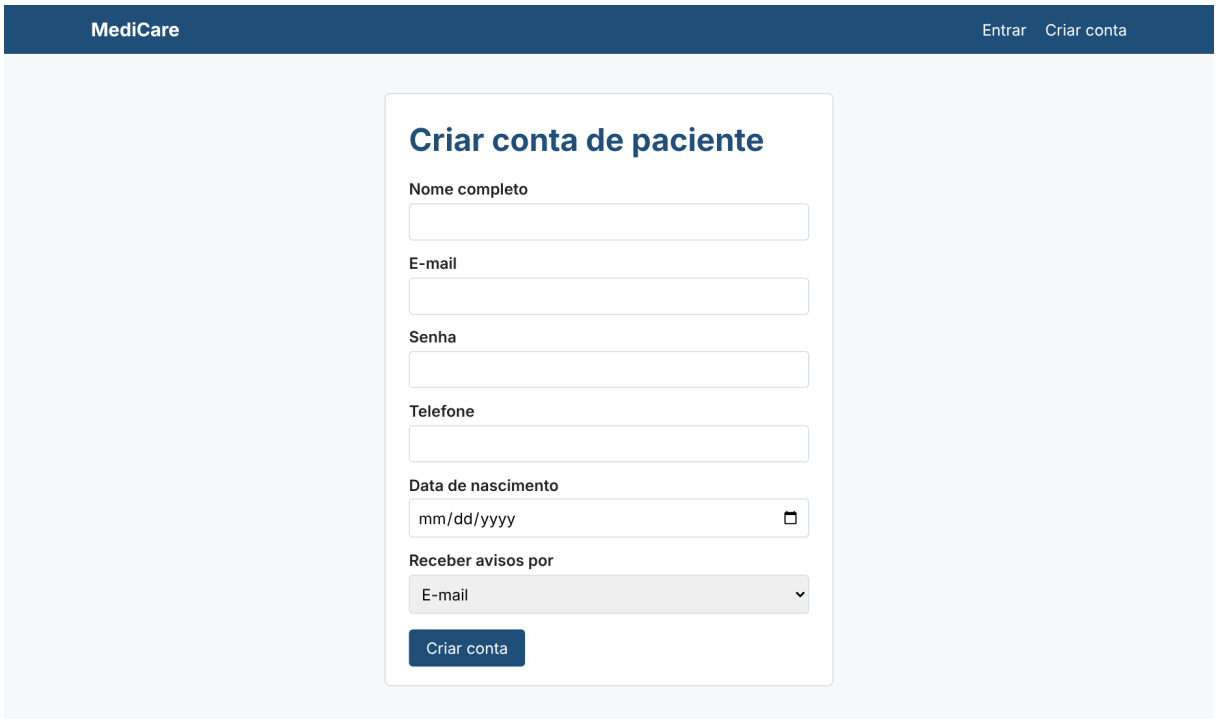


Figura 29 — Criar conta de paciente (UC01).



Figura 30 — Pesquisa de médicos por especialidade (UC03).



Figura 31 — Horários disponíveis de um médico (UC04).



Figura 32 — Consultas do paciente após o agendamento (UC05 e UC06).



Figura 33 — Reagendamento de consulta (UC07).

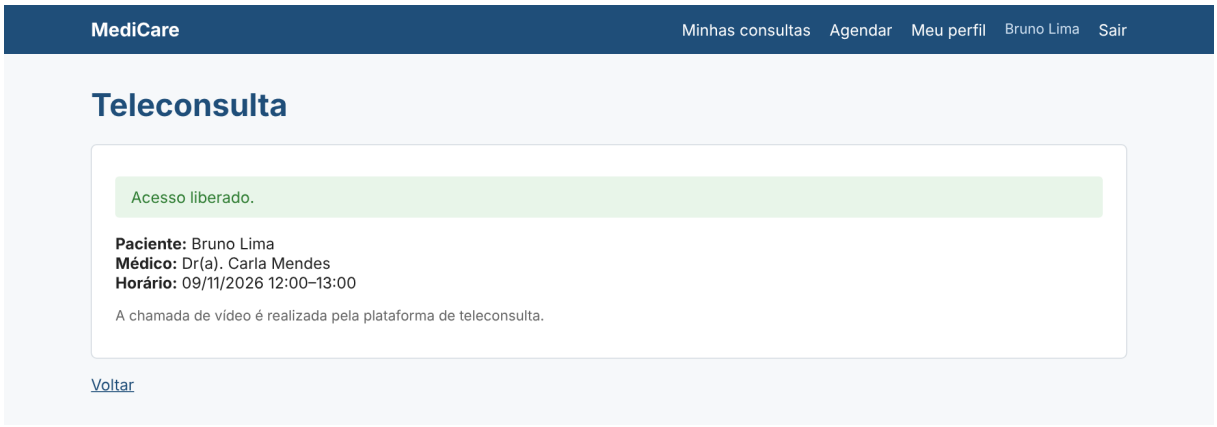


Figura 34 — Acesso à teleconsulta no horário agendado (UC11).

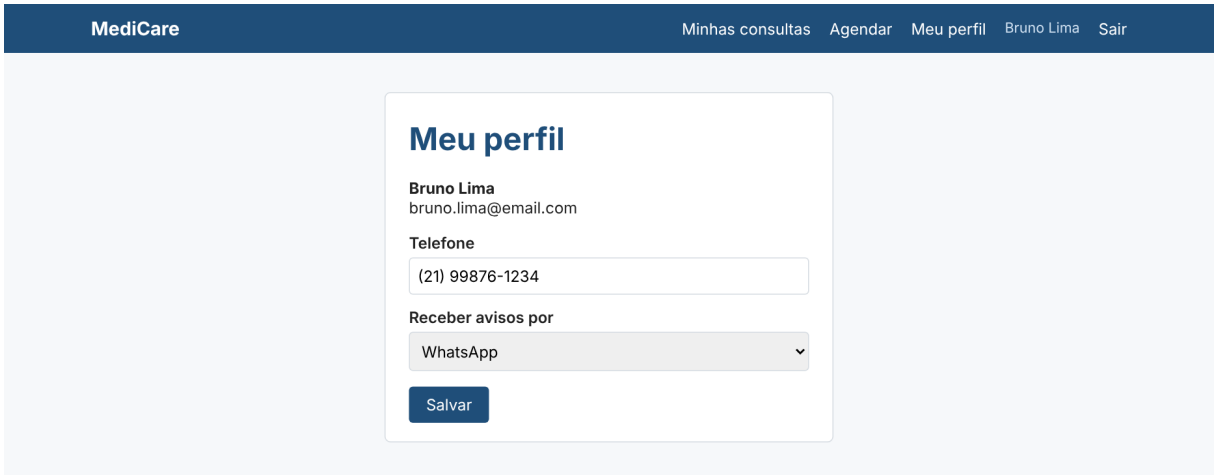


Figura 35 — Perfil do paciente (UC08).



Figura 36 — Recepção: escolha do paciente atendido.



Figura 37 — Recepcionista agendando para uma paciente (UC05).

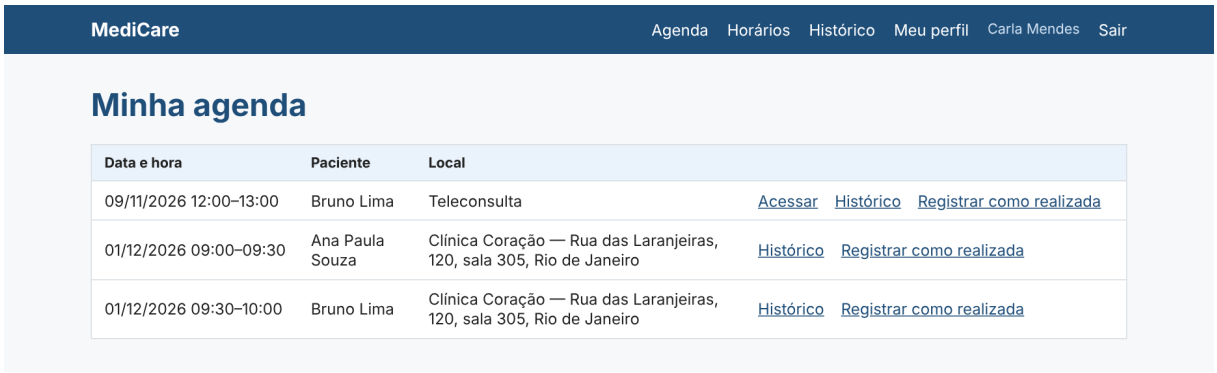


Figura 38 — Agenda do médico (UC10).

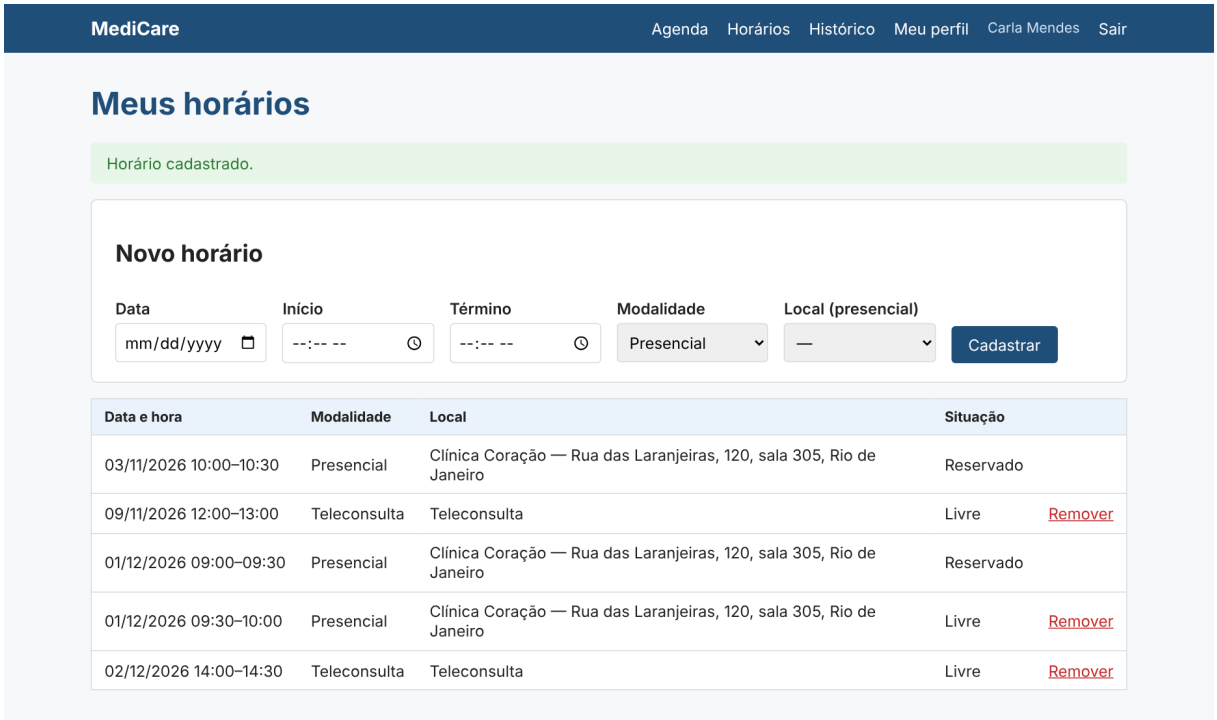


Figura 39 — Cadastro e lista de horários do médico (UC09).

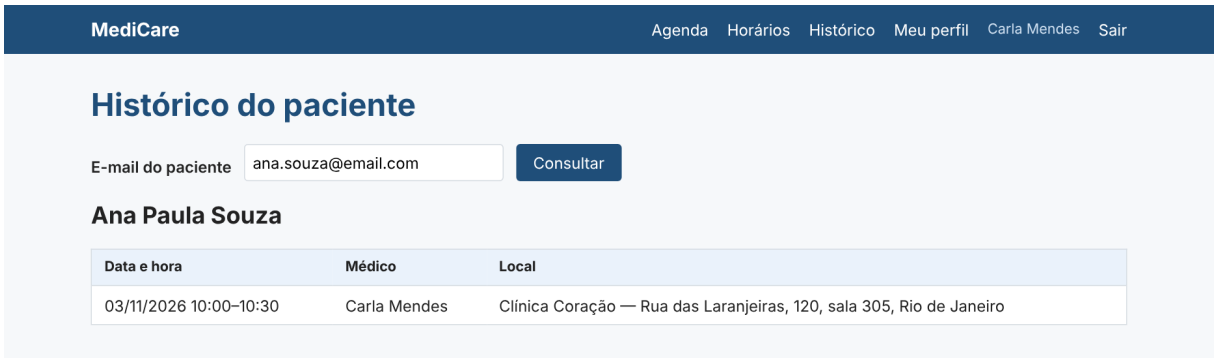


Figura 40 — Histórico de consultas da paciente (UC15).

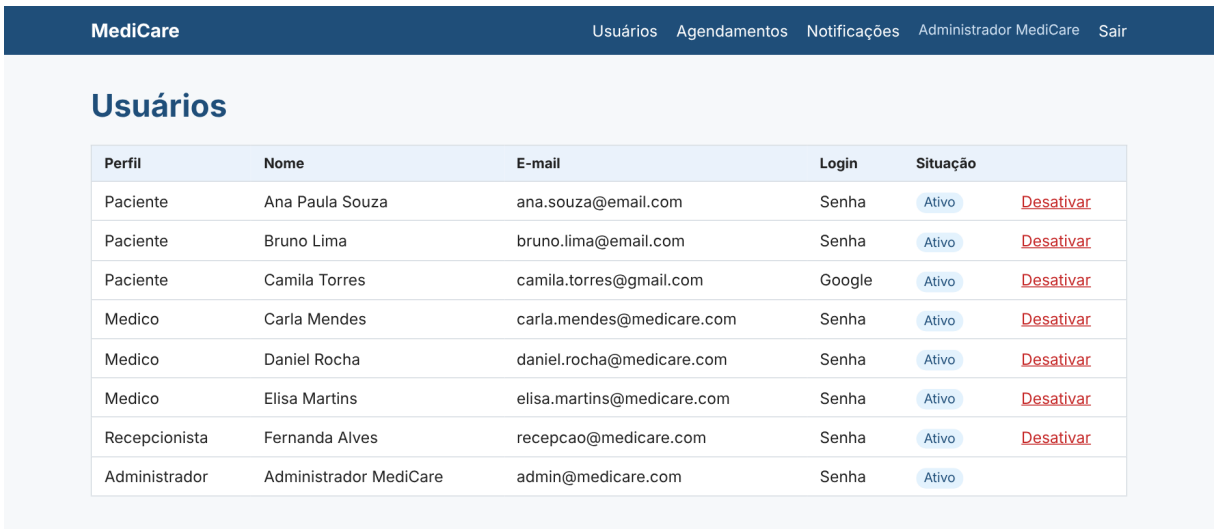


Figura 41 — Gerenciamento de usuários (UC12).

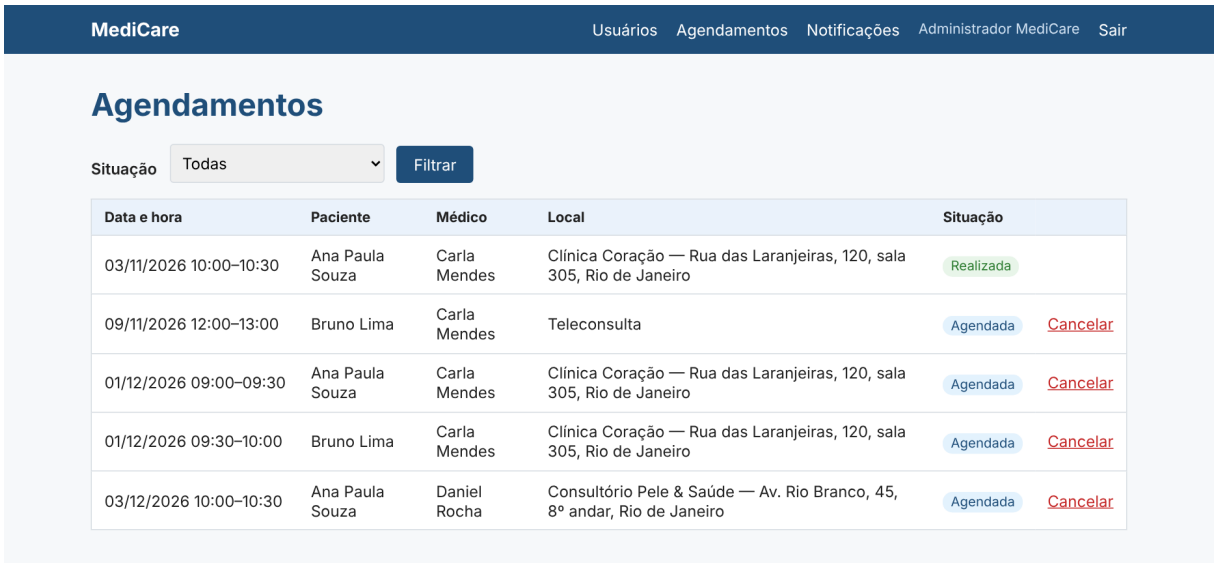


Figura 42 — Gerenciamento de agendamentos (UC13).

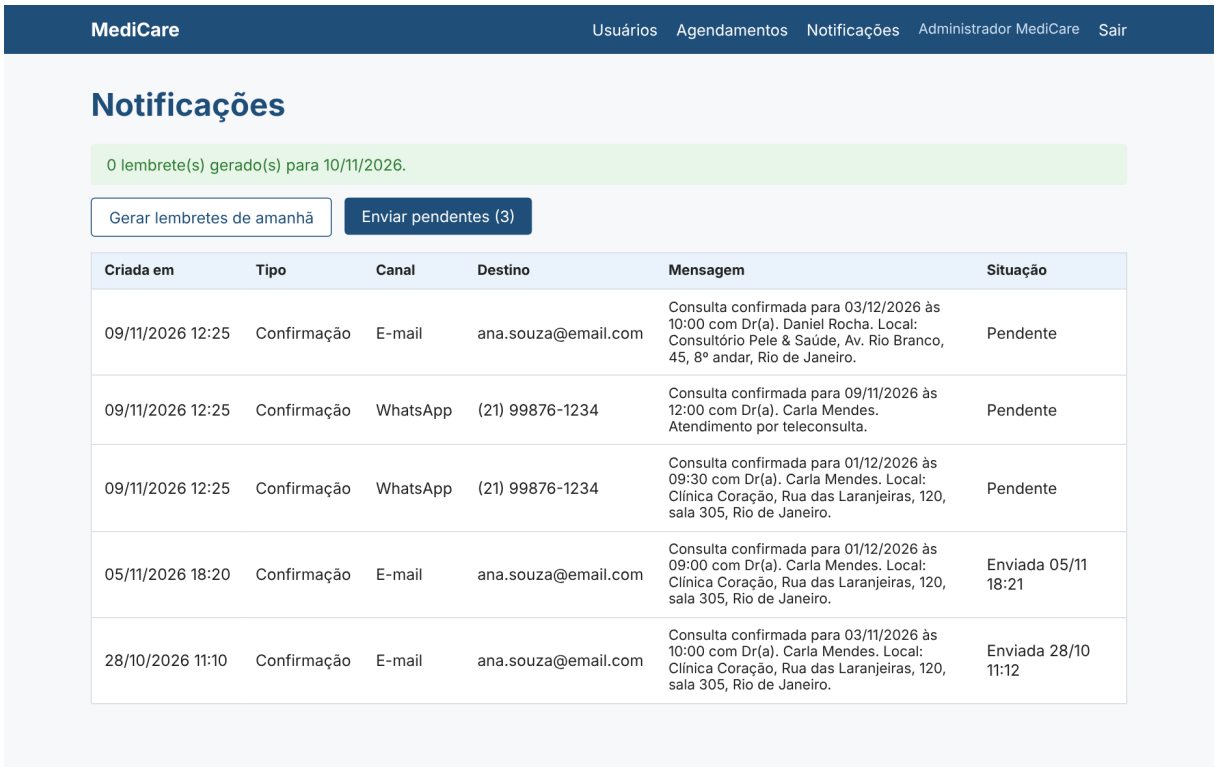


Figura 43 — Notificações de consulta (UC14).



Figura 44 — Acesso negado a uma página de outro perfil.

---

7. Testes

O projeto MediCare.Tests usa o xUnit. Os testes dos serviços são executados duas vezes, com a persistência em CSV e com o EF Core sobre um banco SQLite temporário, e a gravação é conferida reabrindo os dados depois de cada operação.

Classe de teste	Testes	O que verifica
ModeloTests	6	Senha, login externo, reserva e conflito de horários, regras de consulta e notificação.
CsvTests	6	Campos com vírgula, aspas e quebra de linha; geração de Id; dados inválidos.
ContaServiceTests	4 × 2	E-mail duplicado, login, conta desativada e atualização de perfil.
AgendamentoServiceTests	8 × 2	Agendamento, notificações, cancelamento, reagendamento e filtros.
DisponibilidadeServiceTests	5 × 2	Datas, intervalos, local do médico, conflito e remoção.

Classe de teste	Testes	O que verifica
AtendimentoServiceTests	6 × 2	Agenda, conclusão, histórico e acesso à teleconsulta.
NotificacaoServiceTests	2 × 2	Lembretes sem duplicidade e envio das pendentes.
PersistenciaTests	2 × 2	Gravação e leitura de consultas, horários, notificações e usuários.

Resultado da execução com `dotnet test`: 66 testes, todos aprovados.

```
Restore complete (7,9s)
  MediCare.Core net8.0 succeeded (2,1s) → src/MediCare.Core/bin/Debug/net8.0/Med
iCare.Core.dll
  MediCare.Data net8.0 succeeded (0,3s) → src/MediCare.Data/bin/Debug/net8.0/Med
iCare.Data.dll
  MediCare.Tests net8.0 succeeded (0,5s) → tests/MediCare.Tests/bin/Debug/net8.0
/MediCare.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445a1a (64-bit .NET
8.0.33)
[xUnit.net 00:00:00.09]   Discovering: MediCare.Tests
[xUnit.net 00:00:00.19]   Discovered:  MediCare.Tests
[xUnit.net 00:00:00.19]   Starting:    MediCare.Tests
[xUnit.net 00:00:01.68]   Finished:    MediCare.Tests
  MediCare.Tests test net8.0 succeeded (3,3s)

Test summary: total: 66, failed: 0, succeeded: 66, skipped: 0, duration: 3,3s
Build succeeded in 14,4s
>
```

Figura 45 — Resultado da execução dos testes.

A aplicação também foi verificada em funcionamento, simulando o uso pelo navegador em 48 pontos: login de cada perfil e casos de recusa, permissões entre perfis, agendamento, reagendamento e cancelamento, tentativa de cancelar consulta de outro paciente, cadastro e conflito de horários, histórico, acesso à teleconsulta, gestão de usuários e de agendamentos, notificações e criação de contas.

---

8. Execução e Artefatos

Requisito: .NET 8 SDK. Na pasta do projeto:

```
dotnet test                                 # executa os testes
dotnet run --project src/MediCare.Web       # inicia a aplicação em http://localhost:5080
```

Na primeira execução é criado o banco src/MediCare.Web/medicare.db com os dados de exemplo. Usuários de exemplo, todos com a senha senha123: ana.souza@email.com e bruno.lima@email.com (pacientes), recepcao@medicare.com (recepcionista), carla.mendes@medicare.com (médica) e admin@medicare.com (administrador).

Artefatos no repositório https://github.com/CodeByGiovanna/MediCare:

Artefato	Local
Código-fonte, dados de exemplo e testes	src/, dados/, tests/
Documentos de cada entrega (TP1 a TP5)	docs/
Instruções de execução	README.md
Documento final consolidado	este documento

---

9. Conclusão

O MediCare foi construído em cinco entregas, partindo da descrição do sistema e dos requisitos até uma aplicação web completa. Os casos de uso orientaram a modelagem de classes, e os diagramas de sequência e de comunicação definiram os métodos que depois foram implementados e testados.

A mudança de requisitos na quarta entrega mostrou na prática o valor do desenvolvimento iterativo do RUP: a recepcionista, as notificações, o histórico do paciente e o login externo foram incorporados ao modelo e ao código sem retrabalho nas partes já prontas, porque as regras de negócio estavam isoladas em serviços. A mesma separação permitiu trocar o programa de console pela interface Razor Pages e os arquivos CSV pelo EF Core na entrega final.

A versão entregue atende aos quinze casos de uso especificados, com permissões por perfil, dados em banco relacional e 66 testes automatizados aprovados. Os pontos listados na seção 3.6 são o caminho natural para a evolução do sistema, a partir da validação com os usuários.

Anexo — Código-fonte

Código completo da solução, organizado por projeto.

MediCare.Core/Csv/CsvParser.cs

```
using System.Text;

namespace MediCare.Core.Csv;

/// <summary>
/// Leitura e escrita de CSV no formato RFC 4180: campos separados por vírgula,
/// e campos com vírgula, aspas ou quebra de linha ficam entre aspas duplas.
/// </summary>
public static class CsvParser
{
    public const char Separador = ',';

    public static List<List<string>> Ler(TextReader leitor)
    {
        var registros = new List<List<string>>();
        var registro = new List<string>();
        var campo = new StringBuilder();
        var entreAspas = false;
        int c;

        while ((c = leitor.Read()) != -1)
        {
            var ch = (char)c;

            if (entreAspas)
            {
                if (ch == '"')
                {
                    if (leitor.Peek() == '"')
                    {
                        campo.Append('"');
                        leitor.Read();
                    }
                    else
                    {
                        entreAspas = false;
                    }
                }
                else
                {
                    campo.Append(ch);
                }
                continue;
            }

            switch (ch)
            {
                case '"':
                    entreAspas = true;
                    break;
                case Separador:
                    registro.Add(campo.ToString());
                    campo.Clear();
                    break;
                case '\r':
                    break;
                case '\n':
                    registro.Add(campo.ToString());
                    campo.Clear();
                    AdicionarSeNaoVazio(registros, registro);
                    registro = new List<string>();
                    break;
                default:
                    campo.Append(ch);
                    break;
            }
        }

        if (entreAspas)
            throw new FormatException("Arquivo CSV inválido: aspas não fechadas.");

        if (campo.Length > 0 || registro.Count > 0)
        {
            registro.Add(campo.ToString());
            AdicionarSeNaoVazio(registros, registro);
        }

        return registros;
    }
    public static string FormatarLinha(IEnumerable<string> campos) =>
```

```
        string.Join(Separador, campos.Select(Escapar));

    private static string Escapar(string valor)
    {
        if (valor.IndexOfAny(new[] { Separador, '"', '\n', '\r' }) < 0)
            return valor;
        return $"\"{valor.Replace("\"", "\"\"")}\"";
    }

    private static void AdicionarSeNaoVazio(List<List<string>> registros, List<string> registro)
    {
        var linhaEmBranco = registro.Count == 1 && registro[0].Length == 0;
        if (!linhaEmBranco)
            registros.Add(registro);
    }
}
```

MediCare.Core/Csv/CsvValor.cs

```
using System.Globalization;

namespace MediCare.Core.Csv;

/// <summary>
/// Conversões entre texto do CSV e tipos .NET, sempre em formato invariável
/// (datas ISO, ponto como separador decimal), para o arquivo não depender
/// da configuração regional da máquina.
/// </summary>
public static class CsvValor
{
    private const string FormatoData = "yyyy-MM-dd";
    private const string FormatoHora = "HH:mm";
    private const string FormatoDataHora = "yyyy-MM-ddTHH:mm:ss";
    private static readonly CultureInfo Cultura = CultureInfo.InvariantCulture;

    public static string De(int valor) => valor.ToString(Cultura);
    public static string De(int? valor) => valor?.ToString(Cultura) ?? "";
    public static string De(bool valor) => valor ? "true" : "false";
    public static string De(DateOnly valor) => valor.ToString(FormatoData, Cultura);
    public static string De(TimeOnly valor) => valor.ToString(FormatoHora, Cultura);
    public static string De(DateTime valor) => valor.ToString(FormatoDataHora, Cultura);
    public static string De(DateTime? valor) => valor is null ? "" : De(valor.Value);
    public static string De<TEnum>(TEnum valor) where TEnum : struct, Enum => valor.ToString();

    public static int ParaInt(string texto) => int.Parse(texto, Cultura);
    public static int? ParaIntOpcional(string texto) =>
        string.IsNullOrWhiteSpace(texto) ? null : int.Parse(texto, Cultura);
    public static bool ParaBool(string texto) => bool.Parse(texto);
    public static DateOnly ParaData(string texto) => DateOnly.ParseExact(texto, FormatoData, Cultura);
    public static TimeOnly ParaHora(string texto) => TimeOnly.ParseExact(texto, FormatoHora, Cultura);
    public static DateTime ParaDataHora(string texto) => DateTime.ParseExact(texto, FormatoDataHora, Cultura);
    public static DateTime? ParaDataHoraOpcional(string texto) =>
        string.IsNullOrWhiteSpace(texto) ? null : ParaDataHora(texto);
    public static TEnum ParaEnum<TEnum>(string texto) where TEnum : struct, Enum =>
        Enum.Parse<TEnum>(texto, ignoreCase: true);
}
```

MediCare.Core/Models/Administrador.cs

```
namespace MediCare.Core.Models;

public class Administrador : Usuario
{
}
```

MediCare.Core/Models/Consulta.cs

```
namespace MediCare.Core.Models;

public class Consulta : IEntidade
{
    public int Id { get; set; }
    public int PacienteId { get; set; }
    public int HorarioId { get; private set; }
    public StatusConsulta Status { get; private set; } = StatusConsulta.Agendada;
    public DateTime DataAgendamento { get; private set; }

    // Usado pelo EF Core ao materializar a entidade.
    private Consulta() { }

    public Consulta(int pacienteId, int horarioId)
    {
        PacienteId = pacienteId;
        HorarioId = horarioId;
        DataAgendamento = DateTime.Now;
    }

    internal Consulta(int id, int pacienteId, int horarioId, StatusConsulta status, DateTime dataAgendamento)
    {
```

```
        Id = id;
        PacienteId = pacienteId;
        HorarioId = horarioId;
        Status = status;
        DataAgendamento = dataAgendamento;
    }

    public void Cancelar()
    {
        GarantirAgendada();
        Status = StatusConsulta.Cancelada;
    }

    public void Reagendar(int novoHorarioId)
    {
        GarantirAgendada();
        HorarioId = novoHorarioId;
    }

    public void Concluir()
    {
        GarantirAgendada();
        Status = StatusConsulta.Realizada;
    }

    private void GarantirAgendada()
    {
        if (Status != StatusConsulta.Agendada)
            throw new InvalidOperationException(
                $"A consulta está {Status.ToString().ToLower()} e não pode ser alterada.");
    }
}
```

MediCare.Core/Models/Enums.cs

```
using System.ComponentModel.DataAnnotations;

namespace MediCare.Core.Models;

public enum Modalidade
{
    Presencial,
    Teleconsulta
}

public enum StatusConsulta
{
    Agendada,
    Cancelada,
    Realizada
}

public enum ProvedorAutenticacao
{
    [Display(Name = "Senha")] Local,
    Google,
    Microsoft,
    Meta
}

public enum CanalNotificacao
{
    [Display(Name = "E-mail")] Email,
    [Display(Name = "SMS")] Sms,
    WhatsApp
}

public enum TipoNotificacao
{
    [Display(Name = "Confirmação")] Confirmacao,
    Reagendamento,
    Cancelamento,
    Lembrete
}

public enum StatusNotificacao
{
    Pendente,
    Enviada
}
```

MediCare.Core/Models/Especialidade.cs

```
namespace MediCare.Core.Models;

public class Especialidade : IEntidade
{
    public int Id { get; set; }
    public string Nome { get; set; } = "";
}
```

MediCare.Core/Models/HorarioDisponivel.cs

```
namespace MediCare.Core.Models;

public class HorarioDisponivel : IEntidade
{
    public int Id { get; set; }
    public int MedicoId { get; set; }
    public DateOnly Data { get; set; }
    public TimeOnly HoraInicio { get; set; }
    public TimeOnly HoraFim { get; set; }
    public Modalidade Modalidade { get; set; }

    // Preenchido somente quando a modalidade é presencial.
    public int? LocalId { get; set; }

    public bool Disponivel { get; private set; } = true;

    public void Reservar()
    {
        if (!Disponivel)
            throw new InvalidOperationException("Este horário já está reservado.");
        Disponivel = false;
    }

    public void Liberar() => Disponivel = true;

    public bool ConflitaCom(HorarioDisponivel outro) =>
        MedicoId == outro.MedicoId &&
        Data == outro.Data &&
        HoraInicio < outro.HoraFim &&
        outro.HoraInicio < HoraFim;

    internal void Restaurar(bool disponivel) => Disponivel = disponivel;
}
```

MediCare.Core/Models/IEntidade.cs

```
namespace MediCare.Core.Models;

public interface IEntidade
{
    int Id { get; set; }
}
```

MediCare.Core/Models/LocalAtendimento.cs

```
namespace MediCare.Core.Models;

public class LocalAtendimento : IEntidade
{
    public int Id { get; set; }
    public int MedicoId { get; set; }
    public string Nome { get; set; } = "";
    public string Endereco { get; set; } = "";
    public string Cidade { get; set; } = "";
}
```

MediCare.Core/Models/Medico.cs

```
namespace MediCare.Core.Models;

public class Medico : Usuario
{
    public string Crm { get; set; } = "";
    public string Telefone { get; set; } = "";
    public int EspecialidadeId { get; set; }
}
```

MediCare.Core/Models/Notificacao.cs

```
namespace MediCare.Core.Models;

/// <summary>
/// Mensagem gerada pelo sistema para ser entregue ao paciente pelo sistema de notificação
/// (e-mail, SMS ou WhatsApp).
/// </summary>
public class Notificacao : IEntidade
{
    public int Id { get; set; }
    public int ConsultaId { get; private set; }
    public TipoNotificacao Tipo { get; private set; }
    public CanalNotificacao Canal { get; private set; }
    public string Destino { get; private set; }
    public string Mensagem { get; private set; }
    public DateTime DataCriacao { get; private set; }
    public StatusNotificacao Status { get; private set; } = StatusNotificacao.Pendente;
```

```
    public DateTime? DataEnvio { get; private set; }

    // Usado pelo EF Core ao materializar a entidade.
    private Notificacao()
    {
        Destino = "";
        Mensagem = "";
    }

    public Notificacao(int consultaId, TipoNotificacao tipo, CanalNotificacao canal, string destino, string mensagem)
    {
        ConsultaId = consultaId;
        Tipo = tipo;
        Canal = canal;
        Destino = destino;
        Mensagem = mensagem;
        DataCriacao = DateTime.Now;
    }

    internal Notificacao(int id, int consultaId, TipoNotificacao tipo, CanalNotificacao canal, string destino,
        string mensagem, DateTime dataCriacao, StatusNotificacao status, DateTime? dataEnvio)
        : this(consultaId, tipo, canal, destino, mensagem)
    {
        Id = id;
        DataCriacao = dataCriacao;
        Status = status;
        DataEnvio = dataEnvio;
    }

    public void MarcarComoEnviada()
    {
        if (Status == StatusNotificacao.Enviada)
            throw new InvalidOperationException("A notificação já foi enviada.");
        Status = StatusNotificacao.Enviada;
        DataEnvio = DateTime.Now;
    }
}
```

MediCare.Core/Models/Paciente.cs

```
namespace MediCare.Core.Models;

public class Paciente : Usuario
{
    public string Telefone { get; set; } = "";
    public DateOnly DataNascimento { get; set; }
    public CanalNotificacao CanalPreferido { get; set; } = CanalNotificacao.Email;
}
```

MediCare.Core/Models/Recepcionista.cs

```
namespace MediCare.Core.Models;

public class Recepcionista : Usuario
{
}
```

MediCare.Core/Models/Usuario.cs

```
using System.Security.Cryptography;
using System.Text;

namespace MediCare.Core.Models;

public abstract class Usuario : IEntidade
{
    public int Id { get; set; }
    public string Nome { get; set; } = "";
    public string Email { get; set; } = "";
    public string SenhaHash { get; protected set; } = "";
    public ProvedorAutenticacao Provedor { get; set; } = ProvedorAutenticacao.Local;
    public bool Ativo { get; protected set; } = true;
    public DateTime DataCadastro { get; set; } = DateTime.Now;

    public void DefinirSenha(string senha)
    {
        if (string.IsNullOrWhiteSpace(senha))
            throw new ArgumentException("A senha não pode ser vazia.");
        SenhaHash = GerarHash(senha);
    }

    public bool VerificarSenha(string senha) =>
        Provedor == ProvedorAutenticacao.Local && SenhaHash == GerarHash(senha);

    public void Desativar() => Ativo = false;
    public void Ativar() => Ativo = true;

    // Usado apenas ao reconstruir o objeto a partir do CSV.
    internal void Restaurar(string senhaHash, bool ativo)
```

```
    {
        SenhaHash = senhaHash;
        Ativo = ativo;
    }

    private static string GerarHash(string senha) =>
        Convert.ToHexString(SHA256.HashData(Encoding.UTF8.GetBytes(senha)));
}
```

MediCare.Core/Repositories/CsvRepository.cs

```
using System.Text;
using MediCare.Core.Csv;
using MediCare.Core.Models;

namespace MediCare.Core.Repositories;

/// <summary>
/// Repositório genérico que mantém uma entidade em um arquivo CSV.
/// Os registros são carregados na criação do repositório e gravados em Salvar().
/// Cada classe filha define o cabeçalho e como converter uma linha em objeto e vice-versa.
/// </summary>
public abstract class CsvRepository<T> : IRepositorio<T> where T : class, IEntidade
{
    private readonly string _caminho;
    private readonly List<T> _itens = new();

    protected CsvRepository(string caminho)
    {
        _caminho = caminho;
        Carregar();
    }

    protected abstract string[] Cabecalho { get; }
    protected abstract T DeCampos(IReadOnlyList<string> campos);
    protected abstract IEnumerable<string> ParaCampos(T item);

    public IReadOnlyList<T> ListarTodos() => _itens;

    public T? ObterPorId(int id) => _itens.FirstOrDefault(i => i.Id == id);

    public T Adicionar(T item)
    {
        item.Id = _itens.Count == 0 ? 1 : _itens.Max(i => i.Id) + 1;
        _itens.Add(item);
        return item;
    }

    public bool Remover(int id) => _itens.RemoveAll(i => i.Id == id) > 0;

    public void Salvar()
    {
        Directory.CreateDirectory(Path.GetDirectoryName(Path.GetFullPath(_caminho))!);

        var conteudo = new StringBuilder();
        conteudo.AppendLine(CsvParser.FormatarLinha(Cabecalho));
        foreach (var item in _itens.OrderBy(i => i.Id))
            conteudo.AppendLine(CsvParser.FormatarLinha(ParaCampos(item)));

        // Grava primeiro em um arquivo temporário para não corromper o original em caso de falha.
        var temporario = _caminho + ".tmp";
        File.WriteAllText(temporario, conteudo.ToString(), new UTF8Encoding(false));
        File.Move(temporario, _caminho, overwrite: true);
    }

    private void Carregar()
    {
        if (!File.Exists(_caminho))
            return;

        using var leitor = new StreamReader(_caminho, Encoding.UTF8);
        var registros = CsvParser.Ler(leitor);
        if (registros.Count == 0)
            return;

        ValidarCabecalho(registros[0]);

        for (var i = 1; i < registros.Count; i++)
        {
            var campos = registros[i];
            if (campos.Count != Cabecalho.Length)
                throw Erro(i, $"esperados {Cabecalho.Length} campos, encontrados {campos.Count}");

            try
            {
                _itens.Add(DeCampos(campos));
            }
            catch (FormatException ex)
            {
                throw Erro(i, ex.Message);
            }
```

```
        }
    }

    private void ValidarCabecalho(IReadOnlyList<string> cabecalhoArquivo)
    {
        if (!cabecalhoArquivo.SequenceEqual(Cabecalho, StringComparer.OrdinalIgnoreCase))
            throw new InvalidDataException(
                $"Cabeçalho inválido em {Path.GetFileName(_caminho)}. Esperado: {string.Join(",", Cabecalho)}");
    }

    private InvalidDataException Erro(int registro, string detalhe) =>
        new($"{Path.GetFileName(_caminho)}, registro {registro}: {detalhe}");
}
```

MediCare.Core/Repositories/IRepositorio.cs

```
using MediCare.Core.Models;

namespace MediCare.Core.Repositories;

/// <summary>
/// Operações de persistência usadas pelos serviços. Há duas implementações:
/// CsvRepository (arquivos CSV) e EfRepositorio (EF Core, no projeto MediCare.Data).
/// </summary>
public interface IRepositorio<T> where T : class, IEntidade
{
    IReadOnlyList<T> ListarTodos();
    T? ObterPorId(int id);
    T Adicionar(T item);
    bool Remover(int id);
    void Salvar();
}
```

MediCare.Core/Repositories/MediCareDados.cs

```
using MediCare.Core.Models;

namespace MediCare.Core.Repositories;

/// <summary>
/// Reúne os repositórios de todas as entidades. A forma de persistência (CSV ou EF Core)
/// é escolhida por quem cria o objeto; os serviços não dependem dela.
/// </summary>
public class MediCareDados
{
    public required IRepositorio<Especialidade> Especialidades { get; init; }
    public required IRepositorio<Paciente> Pacientes { get; init; }
    public required IRepositorio<Medico> Medicos { get; init; }
    public required IRepositorio<Recepcionista> Recepcionistas { get; init; }
    public required IRepositorio<Administrador> Administradores { get; init; }
    public required IRepositorio<LocalAtendimento> Locais { get; init; }
    public required IRepositorio<HorarioDisponivel> Horarios { get; init; }
    public required IRepositorio<Consulta> Consultas { get; init; }
    public required IRepositorio<Notificacao> Notificacoes { get; init; }

    public static MediCareDados DeArquivosCsv(string pasta) => new()
    {
        Especialidades = new EspecialidadeRepository(Path.Combine(pasta, "especialidades.csv")),
        Pacientes = new PacienteRepository(Path.Combine(pasta, "pacientes.csv")),
        Medicos = new MedicoRepository(Path.Combine(pasta, "medicos.csv")),
        Recepcionistas = new RecepcionistaRepository(Path.Combine(pasta, "recepcionistas.csv")),
        Administradores = new AdministradorRepository(Path.Combine(pasta, "administradores.csv")),
        Locais = new LocalAtendimentoRepository(Path.Combine(pasta, "locais_atendimento.csv")),
        Horarios = new HorarioRepository(Path.Combine(pasta, "horarios.csv")),
        Consultas = new ConsultaRepository(Path.Combine(pasta, "consultas.csv")),
        Notificacoes = new NotificacaoRepository(Path.Combine(pasta, "notificacoes.csv"))
    };

    public IEnumerable<Usuario> TodosOsUsuarios() =>
        Pacientes.ListarTodos().Cast<Usuario>()
            .Concat(Medicos.ListarTodos())
            .Concat(Recepcionistas.ListarTodos())
            .Concat(Administradores.ListarTodos());

    public void SalvarUsuarios()
    {
        Pacientes.Salvar();
        Medicos.Salvar();
        Recepcionistas.Salvar();
        Administradores.Salvar();
    }
}
```

MediCare.Core/Repositories/Repositorios.cs

```
using MediCare.Core.Csv;
using MediCare.Core.Models;

namespace MediCare.Core.Repositories;
```

```
public class PacienteRepository(string caminho) : UsuarioRepository<Paciente>(caminho)
{
    protected override string[] ColunasProprias => ["Telefone", "DataNascimento", "CanalPreferido"];

    protected override void LerCamposProprios(Paciente p, IReadOnlyList<string> c)
    {
        p.Telefone = c[0];
        p.DataNascimento = CsvValor.ParaData(c[1]);
        p.CanalPreferido = CsvValor.ParaEnum<CanalNotificacao>(c[2]);
    }

    protected override IEnumerable<string> CamposProprios(Paciente p) =>
        [p.Telefone, CsvValor.De(p.DataNascimento), CsvValor.De(p.CanalPreferido)];
}

public class MedicoRepository(string caminho) : UsuarioRepository<Medico>(caminho)
{
    protected override string[] ColunasProprias => ["Crm", "Telefone", "EspecialidadeId"];

    protected override void LerCamposProprios(Medico m, IReadOnlyList<string> c)
    {
        m.Crm = c[0];
        m.Telefone = c[1];
        m.EspecialidadeId = CsvValor.ParaInt(c[2]);
    }

    protected override IEnumerable<string> CamposProprios(Medico m) =>
        [m.Crm, m.Telefone, CsvValor.De(m.EspecialidadeId)];
}

public class RecepcionistaRepository(string caminho) : UsuarioRepository<Recepcionista>(caminho);

public class AdministradorRepository(string caminho) : UsuarioRepository<Administrador>(caminho);

public class EspecialidadeRepository(string caminho) : CsvRepository<Especialidade>(caminho)
{
    protected override string[] Cabecalho => ["Id", "Nome"];

    protected override Especialidade DeCampos(IReadOnlyList<string> c) => new()
    {
        Id = CsvValor.ParaInt(c[0]),
        Nome = c[1]
    };

    protected override IEnumerable<string> ParaCampos(Especialidade e) => [CsvValor.De(e.Id), e.Nome];
}

public class LocalAtendimentoRepository(string caminho) : CsvRepository<LocalAtendimento>(caminho)
{
    protected override string[] Cabecalho => ["Id", "MedicoId", "Nome", "Endereco", "Cidade"];

    protected override LocalAtendimento DeCampos(IReadOnlyList<string> c) => new()
    {
        Id = CsvValor.ParaInt(c[0]),
        MedicoId = CsvValor.ParaInt(c[1]),
        Nome = c[2],
        Endereco = c[3],
        Cidade = c[4]
    };

    protected override IEnumerable<string> ParaCampos(LocalAtendimento l) =>
        [CsvValor.De(l.Id), CsvValor.De(l.MedicoId), l.Nome, l.Endereco, l.Cidade];
}

public class HorarioRepository(string caminho) : CsvRepository<HorarioDisponivel>(caminho)
{
    protected override string[] Cabecalho =>
        ["Id", "MedicoId", "Data", "HoraInicio", "HoraFim", "Modalidade", "LocalId", "Disponivel"];

    protected override HorarioDisponivel DeCampos(IReadOnlyList<string> c)
    {
        var horario = new HorarioDisponivel
        {
            Id = CsvValor.ParaInt(c[0]),
            MedicoId = CsvValor.ParaInt(c[1]),
            Data = CsvValor.ParaData(c[2]),
            HoraInicio = CsvValor.ParaHora(c[3]),
            HoraFim = CsvValor.ParaHora(c[4]),
            Modalidade = CsvValor.ParaEnum<Modalidade>(c[5]),
            LocalId = CsvValor.ParaIntOpcional(c[6])
        };
        horario.Restaurar(CsvValor.ParaBool(c[7]));
        return horario;
    }

    protected override IEnumerable<string> ParaCampos(HorarioDisponivel h) =>
    [
        CsvValor.De(h.Id), CsvValor.De(h.MedicoId), CsvValor.De(h.Data), CsvValor.De(h.HoraInicio),
        CsvValor.De(h.HoraFim), CsvValor.De(h.Modalidade), CsvValor.De(h.LocalId), CsvValor.De(h.Disponivel)
    ];
}
public class ConsultaRepository(string caminho) : CsvRepository<Consulta>(caminho)
```

```
{
    protected override string[] Cabecalho => ["Id", "PacienteId", "HorarioId", "Status", "DataAgendamento"];

    protected override Consulta DeCampos(IReadOnlyList<string> c) => new(
        id: CsvValor.ParaInt(c[0]),
        pacienteId: CsvValor.ParaInt(c[1]),
        horarioId: CsvValor.ParaInt(c[2]),
        status: CsvValor.ParaEnum<StatusConsulta>(c[3]),
        dataAgendamento: CsvValor.ParaDataHora(c[4]));

    protected override IEnumerable<string> ParaCampos(Consulta c) =>
    [
        CsvValor.De(c.Id), CsvValor.De(c.PacienteId), CsvValor.De(c.HorarioId),
        CsvValor.De(c.Status), CsvValor.De(c.DataAgendamento)
    ];
}

public class NotificacaoRepository(string caminho) : CsvRepository<Notificacao>(caminho)
{
    protected override string[] Cabecalho =>
        ["Id", "ConsultaId", "Tipo", "Canal", "Destino", "Mensagem", "DataCriacao", "Status", "DataEnvio"];

    protected override Notificacao DeCampos(IReadOnlyList<string> c) => new(
        id: CsvValor.ParaInt(c[0]),
        consultaId: CsvValor.ParaInt(c[1]),
        tipo: CsvValor.ParaEnum<TipoNotificacao>(c[2]),
        canal: CsvValor.ParaEnum<CanalNotificacao>(c[3]),
        destino: c[4],
        mensagem: c[5],
        dataCriacao: CsvValor.ParaDataHora(c[6]),
        status: CsvValor.ParaEnum<StatusNotificacao>(c[7]),
        dataEnvio: CsvValor.ParaDataHoraOpcional(c[8]));

    protected override IEnumerable<string> ParaCampos(Notificacao n) =>
    [
        CsvValor.De(n.Id), CsvValor.De(n.ConsultaId), CsvValor.De(n.Tipo), CsvValor.De(n.Canal), n.Destino,
        n.Mensagem, CsvValor.De(n.DataCriacao), CsvValor.De(n.Status), CsvValor.De(n.DataEnvio)
    ];
}
```

MediCare.Core/Repositories/UsuarioRepository.cs

```
using MediCare.Core.Csv;
using MediCare.Core.Models;

namespace MediCare.Core.Repositories;

/// <summary>
/// Base dos repositórios de usuários. Trata as colunas herdadas de Usuario;
/// as classes filhas acrescentam apenas as colunas próprias de cada tipo.
/// </summary>
public abstract class UsuarioRepository<T>(string caminho) : CsvRepository<T>(caminho)
    where T : Usuario, new()
{
    private static readonly string[] ColunasUsuario =
        ["Id", "Nome", "Email", "SenhaHash", "Provedor", "Ativo", "DataCadastro"];

    protected virtual string[] ColunasProprias => [];

    protected override string[] Cabecalho => [.. ColunasUsuario, .. ColunasProprias];

    protected virtual void LerCamposProprios(T usuario, IReadOnlyList<string> campos) { }

    protected virtual IEnumerable<string> CamposProprios(T usuario) => [];

    protected override T DeCampos(IReadOnlyList<string> c)
    {
        var usuario = new T
        {
            Id = CsvValor.ParaInt(c[0]),
            Nome = c[1],
            Email = c[2],
            Provedor = CsvValor.ParaEnum<ProvedorAutenticacao>(c[4]),
            DataCadastro = CsvValor.ParaDataHora(c[6])
        };
        usuario.Restaurar(c[3], CsvValor.ParaBool(c[5]));
        LerCamposProprios(usuario, c.Skip(ColunasUsuario.Length).ToList());
        return usuario;
    }

    protected override IEnumerable<string> ParaCampos(T u) =>
    [
        CsvValor.De(u.Id), u.Nome, u.Email, u.SenhaHash, CsvValor.De(u.Provedor),
        CsvValor.De(u.Ativo), CsvValor.De(u.DataCadastro), .. CamposProprios(u)
    ];
}
```

MediCare.Core/Services/AgendamentoService.cs

```
using MediCare.Core.Models;
```

```
using MediCare.Core.Repositories;

namespace MediCare.Core.Services;

/// <summary>
/// Casos de uso UC03 a UC07, usados pelo paciente e pela recepcionista
/// (pesquisar médicos, consultar horários, agendar, cancelar e reagendar consultas),
/// e UC13, usado pelo administrador para acompanhar e cancelar agendamentos.
/// </summary>
public class AgendamentoService(MediCareDados dados, NotificacaoService notificacoes)
{
    public IEnumerable<Medico> PesquisarMedicos(int? especialidadeId = null) =>
        dados.Medicos.ListarTodos()
            .Where(m => m.Ativo && (especialidadeId is null || m.EspecialidadeId == especialidadeId));

    public IEnumerable<HorarioDisponivel> HorariosDisponiveis(int medicoId) =>
        dados.Horarios.ListarTodos()
            .Where(h => h.MedicoId == medicoId && h.Disponivel)
            .OrderBy(h => h.Data).ThenBy(h => h.HoraInicio);

    public IEnumerable<Consulta> ConsultasDoPaciente(int pacienteId) =>
        dados.Consultas.ListarTodos().Where(c => c.PacienteId == pacienteId);

    public IEnumerable<Consulta> ListarAgendamentos(StatusConsulta? status = null) =>
        dados.Consultas.ListarTodos()
            .Where(c => status is null || c.Status == status)
            .OrderBy(c => ObterHorario(c.HorarioId).Data)
            .ThenBy(c => ObterHorario(c.HorarioId).HoraInicio);

    public Consulta Agendar(int pacienteId, int horarioId)
    {
        var paciente = dados.Pacientes.ObterPorId(pacienteId)
            ?? throw new InvalidOperationException("Paciente não encontrado.");
        if (!paciente.Ativo)
            throw new InvalidOperationException("Paciente desativado não pode agendar consultas.");

        var horario = ObterHorario(horarioId);
        horario.Reservar();

        var consulta = dados.Consultas.Adicionar(new Consulta(pacienteId, horarioId));
        Salvar();
        notificacoes.Registrar(consulta, TipoNotificacao.Confirmacao);
        return consulta;
    }

    public void Cancelar(int consultaId)
    {
        var consulta = ObterConsulta(consultaId);
        consulta.Cancelar();
        ObterHorario(consulta.HorarioId).Liberar();
        Salvar();
        notificacoes.Registrar(consulta, TipoNotificacao.Cancelamento);
    }

    public void Reagendar(int consultaId, int novoHorarioId)
    {
        var consulta = ObterConsulta(consultaId);
        var horarioAtual = ObterHorario(consulta.HorarioId);
        var novoHorario = ObterHorario(novoHorarioId);

        if (novoHorario.MedicoId != horarioAtual.MedicoId)
            throw new InvalidOperationException("O novo horário deve ser do mesmo médico.");
        if (!novoHorario.Disponivel)
            throw new InvalidOperationException("O novo horário não está disponível.");

        consulta.Reagendar(novoHorarioId);
        novoHorario.Reservar();
        horarioAtual.Liberar();
        Salvar();
        notificacoes.Registrar(consulta, TipoNotificacao.Reagendamento);
    }

    private Consulta ObterConsulta(int id) =>
        dados.Consultas.ObterPorId(id) ?? throw new InvalidOperationException("Consulta não encontrada.");

    private HorarioDisponivel ObterHorario(int id) =>
        dados.Horarios.ObterPorId(id) ?? throw new InvalidOperationException("Horário não encontrado.");

    private void Salvar()
    {
        dados.Horarios.Salvar();
        dados.Consultas.Salvar();
    }
}
```

MediCare.Core/Services/AtendimentoService.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;

namespace MediCare.Core.Services;
```

```
/// <summary>
/// Casos de uso de atendimento: UC10 (gerenciar agenda), UC11 (realizar teleconsulta)
/// e UC15 (consultar histórico do paciente).
/// </summary>
public class AtendimentoService(MediCareDados dados)
{
    public IEnumerable<Consulta> Agenda(int medicoId) =>
        dados.Consultas.ListarTodos()
            .Where(c => c.Status == StatusConsulta.Agendada && Horario(c).MedicoId == medicoId)
            .OrderBy(c => Horario(c).Data).ThenBy(c => Horario(c).HoraInicio);

    public void ConcluirConsulta(int medicoId, int consultaId)
    {
        var consulta = dados.Consultas.ObterPorId(consultaId);
        if (consulta is null || Horario(consulta).MedicoId != medicoId)
            throw new InvalidOperationException("Consulta não encontrada na sua agenda.");

        consulta.Concluir();
        dados.Consultas.Salvar();
    }

    public IEnumerable<Consulta> HistoricoDoPaciente(int medicoId, int pacienteId)
    {
        var consultasDoPaciente = dados.Consultas.ListarTodos()
            .Where(c => c.PacienteId == pacienteId)
            .ToList();

        if (!consultasDoPaciente.Any(c => Horario(c).MedicoId == medicoId))
            throw new InvalidOperationException(
                "Só é possível ver o histórico de pacientes atendidos por você.");

        return consultasDoPaciente
            .Where(c => c.Status == StatusConsulta.Realizada)
            .OrderByDescending(c => Horario(c).Data).ThenByDescending(c => Horario(c).HoraInicio);
    }

    public Consulta AcessarTeleconsulta(Usuario usuario, int consultaId, DateTime agora)
    {
        var consulta = dados.Consultas.ObterPorId(consultaId)
            ?? throw new InvalidOperationException("Consulta não encontrada.");
        var horario = Horario(consulta);

        var participa = usuario switch
        {
            Paciente p => p.Id == consulta.PacienteId,
            Medico m => m.Id == horario.MedicoId,
            _ => false
        };
        if (!participa)
            throw new InvalidOperationException("Você não participa desta consulta.");
        if (horario.Modalidade != Modalidade.Teleconsulta)
            throw new InvalidOperationException("Esta consulta é presencial.");
        if (consulta.Status != StatusConsulta.Agendada)
            throw new InvalidOperationException("A consulta não está agendada.");

        var inicio = horario.Data.ToDateTime(horario.HoraInicio);
        var fim = horario.Data.ToDateTime(horario.HoraFim);
        if (agora < inicio || agora > fim)
            throw new InvalidOperationException(
                $"A teleconsulta só pode ser acessada no horário agendado ({inicio:dd/MM/yyyy HH:mm} a {fim:HH:mm}).");

        return consulta;
    }

    public IEnumerable<Consulta> TeleconsultasAgendadas(Usuario usuario) =>
        dados.Consultas.ListarTodos()
            .Where(c => c.Status == StatusConsulta.Agendada && Horario(c).Modalidade == Modalidade.Teleconsulta)
            .Where(c => usuario switch
            {
                Paciente p => c.PacienteId == p.Id,
                Medico m => Horario(c).MedicoId == m.Id,
                _ => false
            });

    private HorarioDisponivel Horario(Consulta consulta) => dados.Horarios.ObterPorId(consulta.HorarioId)!;
}
```

MediCare.Core/Services/ContaService.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;

namespace MediCare.Core.Services;

/// <summary>
/// Contas de usuário: UC01 (criar conta), UC02 (login), UC08 (gerenciar perfil) e UC12 (gerenciar usuários).
/// </summary>
public class ContaService(MediCareDados dados)
{
    public Paciente CadastrarPaciente(string nome, string email, string senha, string telefone,
        DateOnly dataNascimento, CanalNotificacao canal)
```

```
    {
        GarantirEmailLivre(email);

        var paciente = new Paciente
        {
            Nome = nome,
            Email = email,
            Telefone = telefone,
            DataNascimento = dataNascimento,
            CanalPreferido = canal
        };
        paciente.DefinirSenha(senha);

        dados.Pacientes.Adicionar(paciente);
        dados.Pacientes.Salvar();
        return paciente;
    }

    public Medico CadastrarMedico(string nome, string email, string senha, string crm, string telefone,
        int especialidadeId)
    {
        GarantirEmailLivre(email);
        if (dados.Especialidades.ObterPorId(especialidadeId) is null)
            throw new ArgumentException("Especialidade não encontrada.");

        var medico = new Medico
        {
            Nome = nome,
            Email = email,
            Crm = crm,
            Telefone = telefone,
            EspecialidadeId = especialidadeId
        };
        medico.DefinirSenha(senha);

        dados.Medicos.Adicionar(medico);
        dados.Medicos.Salvar();
        return medico;
    }

    public Usuario Autenticar(string email, string senha)
    {
        var usuario = ObterPorEmail(email)
            ?? throw new InvalidOperationException("E-mail ou senha inválidos.");

        if (usuario.Provedor != ProvedorAutenticacao.Local)
            throw new InvalidOperationException($"Esta conta utiliza login pelo {usuario.Provedor}.");
        if (!usuario.VerificarSenha(senha))
            throw new InvalidOperationException("E-mail ou senha inválidos.");
        if (!usuario.Ativo)
            throw new InvalidOperationException("Conta desativada. Procure o administrador.");

        return usuario;
    }

    public void AtualizarPerfil(Paciente paciente, string telefone, CanalNotificacao canal)
    {
        paciente.Telefone = telefone;
        paciente.CanalPreferido = canal;
        dados.Pacientes.Salvar();
    }

    public void AtualizarPerfil(Medico medico, string telefone)
    {
        medico.Telefone = telefone;
        dados.Medicos.Salvar();
    }

    public Usuario? ObterPorEmail(string email) =>
        dados.TodosOsUsuarios().FirstOrDefault(u => u.Email.Equals(email, StringComparison.OrdinalIgnoreCase));

    public IEnumerable<Usuario> ListarUsuarios() => dados.TodosOsUsuarios();

    public void AlterarSituacao(string email, bool ativo)
    {
        var usuario = ObterPorEmail(email) ?? throw new InvalidOperationException("Usuário não encontrado.");
        if (ativo) usuario.Ativar();
        else usuario.Desativar();
        dados.SalvarUsuarios();
    }

    private void GarantirEmailLivre(string email)
    {
        if (ObterPorEmail(email) is not null)
            throw new InvalidOperationException("Já existe uma conta com este e-mail.");
    }
}
```

MediCare.Core/Services/DisponibilidadeService.cs

```
using MediCare.Core.Models;
```

```
using MediCare.Core.Repositories;

namespace MediCare.Core.Services;

/// <summary>
/// Caso de uso UC09: o médico cadastra e remove seus horários de atendimento.
/// </summary>
public class DisponibilidadeService(MediCareDados dados)
{
    public IEnumerable<LocalAtendimento> LocaisDoMedico(int medicoId) =>
        dados.Locais.ListarTodos().Where(l => l.MedicoId == medicoId);

    public IEnumerable<HorarioDisponivel> HorariosDoMedico(int medicoId) =>
        dados.Horarios.ListarTodos()
            .Where(h => h.MedicoId == medicoId)
            .OrderBy(h => h.Data).ThenBy(h => h.HoraInicio);

    public HorarioDisponivel CadastrarHorario(int medicoId, DateOnly data, TimeOnly inicio, TimeOnly fim,
        Modalidade modalidade, int? localId)
    {
        if (data < DateOnly.FromDateTime(DateTime.Today))
            throw new ArgumentException("Não é possível cadastrar horários em datas passadas.");
        if (fim <= inicio)
            throw new ArgumentException("A hora de término deve ser posterior à hora de início.");

        if (modalidade == Modalidade.Presencial)
        {
            var local = localId is int id ? dados.Locais.ObterPorId(id) : null;
            if (local is null || local.MedicoId != medicoId)
                throw new ArgumentException("Informe um local de atendimento cadastrado para este médico.");
        }
        else
        {
            localId = null;
        }

        var horario = new HorarioDisponivel
        {
            MedicoId = medicoId,
            Data = data,
            HoraInicio = inicio,
            HoraFim = fim,
            Modalidade = modalidade,
            LocalId = localId
        };

        if (HorariosDoMedico(medicoId).Any(h => h.ConflitaCom(horario)))
            throw new InvalidOperationException("O horário conflita com outro já cadastrado.");

        dados.Horarios.Adicionar(horario);
        dados.Horarios.Salvar();
        return horario;
    }

    public void RemoverHorario(int medicoId, int horarioId)
    {
        var horario = dados.Horarios.ObterPorId(horarioId);
        if (horario is null || horario.MedicoId != medicoId)
            throw new InvalidOperationException("Horário não encontrado.");
        if (dados.Consultas.ListarTodos().Any(c => c.HorarioId == horarioId))
            throw new InvalidOperationException("O horário possui consultas registradas e não pode ser removido.");

        dados.Horarios.Remover(horarioId);
        dados.Horarios.Salvar();
    }
}
```

MediCare.Core/Services/NotificacaoService.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;

namespace MediCare.Core.Services;

/// <summary>
/// Gera as notificações de consulta (UC14) e controla o que já foi entregue ao sistema de notificação.
/// </summary>
public class NotificacaoService(MediCareDados dados)
{
    public Notificacao Registrar(Consulta consulta, TipoNotificacao tipo)
    {
        var paciente = dados.Pacientes.ObterPorId(consulta.PacienteId)!;
        var canal = paciente.CanalPreferido;
        var destino = canal == CanalNotificacao.Email ? paciente.Email : paciente.Telefone;

        var notificacao = new Notificacao(consulta.Id, tipo, canal, destino, MontarMensagem(consulta, tipo));
        dados.Notificacoes.Adicionar(notificacao);
        dados.Notificacoes.Salvar();
        return notificacao;
    }
    public int GerarLembretes(DateOnly dia)
```

```
    {
        var jaLembradas = dados.Notificacoes.ListarTodos()
            .Where(n => n.Tipo == TipoNotificacao.Lembrete)
            .Select(n => n.ConsultaId)
            .ToHashSet();

        var consultas = dados.Consultas.ListarTodos()
            .Where(c => c.Status == StatusConsulta.Agendada
                        && !jaLembradas.Contains(c.Id)
                        && dados.Horarios.ObterPorId(c.HorarioId)!.Data == dia)
            .ToList();

        foreach (var consulta in consultas)
            Registrar(consulta, TipoNotificacao.Lembrete);

        return consultas.Count;
    }

    public IEnumerable<Notificacao> ListarPendentes() =>
        dados.Notificacoes.ListarTodos().Where(n => n.Status == StatusNotificacao.Pendente);

    public void MarcarComoEnviada(int notificacaoId)
    {
        var notificacao = dados.Notificacoes.ObterPorId(notificacaoId)
            ?? throw new InvalidOperationException("Notificação não encontrada.");
        notificacao.MarcarComoEnviada();
        dados.Notificacoes.Salvar();
    }

    private string MontarMensagem(Consulta consulta, TipoNotificacao tipo)
    {
        var horario = dados.Horarios.ObterPorId(consulta.HorarioId)!;
        var medico = dados.Medicos.ObterPorId(horario.MedicoId)!;
        var quando = $"{horario.Data:dd/MM/yyyy} às {horario.HoraInicio:HH:mm} com Dr(a). {medico.Nome}";

        var onde = horario.LocalId is int localId && dados.Locais.ObterPorId(localId) is { } local
            ? $" Local: {local.Nome}, {local.Endereco}, {local.Cidade}."
            : " Atendimento por teleconsulta.";

        return tipo switch
        {
            TipoNotificacao.Confirmacao => $"Consulta confirmada para {quando}.{onde}",
            TipoNotificacao.Reagendamento => $"Consulta reagendada para {quando}.{onde}",
            TipoNotificacao.Cancelamento => $"A consulta de {quando} foi cancelada.",
            TipoNotificacao.Lembrete => $"Lembrete: consulta em {quando}.{onde}",
            _ => throw new ArgumentOutOfRangeException(nameof(tipo))
        };
    }
}
```

MediCare.Data/DadosEfCore.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;

namespace MediCare.Data;

public static class DadosEfCore
{
    public static MediCareDados Criar(MediCareDbContext contexto) => new()
    {
        Especialidades = new EfRepositorio<Especialidade>(contexto),
        Pacientes = new EfRepositorio<Paciente>(contexto),
        Medicos = new EfRepositorio<Medico>(contexto),
        Recepcionistas = new EfRepositorio<Recepcionista>(contexto),
        Administradores = new EfRepositorio<Administrador>(contexto),
        Locais = new EfRepositorio<LocalAtendimento>(contexto),
        Horarios = new EfRepositorio<HorarioDisponivel>(contexto),
        Consultas = new EfRepositorio<Consulta>(contexto),
        Notificacoes = new EfRepositorio<Notificacao>(contexto)
    };

    /// <summary>
    /// Cria o banco, se ainda não existir, e o preenche com os dados de exemplo
    /// dos arquivos CSV usados nas etapas anteriores do projeto.
    /// </summary>
    public static void PrepararBanco(MediCareDbContext contexto, string pastaCsv)
    {
        contexto.Database.EnsureCreated();
        if (contexto.Especialidades.Any())
            return;

        var csv = MediCareDados.DeArquivosCsv(pastaCsv);
        contexto.Especialidades.AddRange(csv.Especialidades.ListarTodos());
        contexto.Pacientes.AddRange(csv.Pacientes.ListarTodos());
        contexto.Medicos.AddRange(csv.Medicos.ListarTodos());
        contexto.Recepcionistas.AddRange(csv.Recepcionistas.ListarTodos());
        contexto.Administradores.AddRange(csv.Administradores.ListarTodos());
        contexto.Locais.AddRange(csv.Locais.ListarTodos());
        contexto.Horarios.AddRange(csv.Horarios.ListarTodos());
        contexto.Consultas.AddRange(csv.Consultas.ListarTodos());
```

```
        contexto.Notificacoes.AddRange(csv.Notificacoes.ListarTodos());
        contexto.SaveChanges();
    }
}
```

MediCare.Data/EfRepositorio.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;
using Microsoft.EntityFrameworkCore;

namespace MediCare.Data;

/// <summary>
/// Implementação de IRepositorio com EF Core. As entidades ficam rastreadas pelo contexto,
/// então as alterações feitas pelos serviços são gravadas em Salvar().
/// </summary>
public class EfRepositorio<T>(MediCareDbContext contexto) : IRepositorio<T> where T : class, IEntidade
{
    private readonly DbSet<T> _conjunto = contexto.Set<T>();

    public IReadOnlyList<T> ListarTodos()
    {
        // Local inclui entidades adicionadas e ainda não gravadas, como no repositório CSV.
        _conjunto.Load();
        return _conjunto.Local.OrderBy(i => i.Id).ToList();
    }

    public T? ObterPorId(int id) => _conjunto.Find(id);

    public T Adicionar(T item)
    {
        _conjunto.Add(item);
        return item;
    }

    public bool Remover(int id)
    {
        var item = ObterPorId(id);
        if (item is null)
            return false;
        _conjunto.Remove(item);
        return true;
    }

    public void Salvar() => contexto.SaveChanges();
}
```

MediCare.Data/MediCareDbContext.cs

```
using MediCare.Core.Models;
using Microsoft.EntityFrameworkCore;

namespace MediCare.Data;

/// <summary>
/// Mapeamento das entidades do modelo para o banco SQLite.
/// Cada tipo de usuário tem a sua própria tabela; as associações do diagrama de classes
/// viram chaves estrangeiras.
/// </summary>
public class MediCareDbContext(DbContextOptions<MediCareDbContext> options) : DbContext(options)
{
    public DbSet<Especialidade> Especialidades => Set<Especialidade>();
    public DbSet<Paciente> Pacientes => Set<Paciente>();
    public DbSet<Medico> Medicos => Set<Medico>();
    public DbSet<Recepcionista> Recepcionistas => Set<Recepcionista>();
    public DbSet<Administrador> Administradores => Set<Administrador>();
    public DbSet<LocalAtendimento> Locais => Set<LocalAtendimento>();
    public DbSet<HorarioDisponivel> Horarios => Set<HorarioDisponivel>();
    public DbSet<Consulta> Consultas => Set<Consulta>();
    public DbSet<Notificacao> Notificacoes => Set<Notificacao>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Especialidade>().ToTable("Especialidades");
        modelBuilder.Entity<Paciente>().ToTable("Pacientes");
        modelBuilder.Entity<Recepcionista>().ToTable("Recepcionistas");
        modelBuilder.Entity<Administrador>().ToTable("Administradores");

        modelBuilder.Entity<Medico>().ToTable("Medicos")
            .HasOne<Especialidade>().WithMany().HasForeignKey(m => m.EspecialidadeId);

        modelBuilder.Entity<LocalAtendimento>().ToTable("LocaisAtendimento")
            .HasOne<Medico>().WithMany().HasForeignKey(l => l.MedicoId);

        var horario = modelBuilder.Entity<HorarioDisponivel>().ToTable("Horarios");
        horario.HasOne<Medico>().WithMany().HasForeignKey(h => h.MedicoId);
        horario.HasOne<LocalAtendimento>().WithMany().HasForeignKey(h => h.LocalId);

        var consulta = modelBuilder.Entity<Consulta>().ToTable("Consultas");
```

```
        consulta.HasOne<Paciente>().WithMany().HasForeignKey(c => c.PacienteId);
        consulta.HasOne<HorarioDisponivel>().WithMany().HasForeignKey(c => c.HorarioId);

        modelBuilder.Entity<Notificacao>().ToTable("Notificacoes")
            .HasOne<Consulta>().WithMany().HasForeignKey(n => n.ConsultaId);
    }
}
```

MediCare.Web/Program.cs

```
using MediCare.Core.Repositories;
using MediCare.Core.Services;
using MediCare.Data;
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Authentication.Cookies;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Pasta dos arquivos CSV, relativa ao projeto. Na persistência com EF Core,
// esses arquivos são usados apenas para preencher o banco na primeira execução.
var pastaDados = CaminhoDoProjeto(builder.Configuration["PastaDados"] ?? "dados");
var usarCsv = builder.Configuration["Persistencia"] == "Csv";

if (usarCsv)
{
    builder.Services.AddSingleton(MediCareDados.DeArquivosCsv(pastaDados));
}
else
{
    var banco = CaminhoDoProjeto(builder.Configuration["BancoDeDados"] ?? "medicare.db");
    builder.Services.AddDbContext<MediCareDbContext>(opcoes => opcoes.UseSqlite($"Data Source={banco}"));
    builder.Services.AddScoped(sp => DadosEfCore.Criar(sp.GetRequiredService<MediCareDbContext>()));
}

builder.Services.AddScoped<NotificacaoService>();
builder.Services.AddScoped<ContaService>();
builder.Services.AddScoped<AgendamentoService>();
builder.Services.AddScoped<DisponibilidadeService>();
builder.Services.AddScoped<AtendimentoService>();
builder.Services.AddScoped<UsuarioLogado>();
builder.Services.AddScoped<Apresentacao>();

builder.Services
    .AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(opcoes =>
    {
        opcoes.LoginPath = "/Conta/Entrar";
        opcoes.LogoutPath = "/Conta/Sair";
        opcoes.AccessDeniedPath = "/Conta/AcessoNegado";
    });

builder.Services.AddAuthorization(opcoes =>
{
    opcoes.AddPolicy(Perfis.Agendamento, p => p.RequireRole(Perfis.Paciente, Perfis.Recepcionista));
    opcoes.AddPolicy(Perfis.Teleconsulta, p => p.RequireRole(Perfis.Paciente, Perfis.Medico));
    opcoes.AddPolicy(Perfis.Paciente, p => p.RequireRole(Perfis.Paciente));
    opcoes.AddPolicy(Perfis.Recepcionista, p => p.RequireRole(Perfis.Recepcionista));
    opcoes.AddPolicy(Perfis.Medico, p => p.RequireRole(Perfis.Medico));
    opcoes.AddPolicy(Perfis.Administrador, p => p.RequireRole(Perfis.Administrador));
});

builder.Services
    .AddRazorPages(opcoes =>
    {
        opcoes.Conventions.AuthorizeFolder("/Agendamento", Perfis.Agendamento);
        opcoes.Conventions.AuthorizeFolder("/Teleconsulta", Perfis.Teleconsulta);
        opcoes.Conventions.AuthorizeFolder("/AreaPaciente", Perfis.Paciente);
        opcoes.Conventions.AuthorizeFolder("/Recepcao", Perfis.Recepcionista);
        opcoes.Conventions.AuthorizeFolder("/AreaMedico", Perfis.Medico);
        opcoes.Conventions.AuthorizeFolder("/Admin", Perfis.Administrador);
    })
    .AddMvcOptions(opcoes => opcoes.Filters.Add<AcessoExclusivoFilter>());

var app = builder.Build();

if (!usarCsv)
{
    using var escopo = app.Services.CreateScope();
    DadosEfCore.PrepararBanco(escopo.ServiceProvider.GetRequiredService<MediCareDbContext>(), pastaDados);
}

if (!app.Environment.IsDevelopment())
    app.UseExceptionHandler("/Erro");

app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapRazorPages();
app.Run();
```

```
string CaminhoDoProjeto(string caminho) =>
    Path.GetFullPath(Path.Combine(builder.Environment.ContentRootPath, caminho));
```

MediCare.Web/Infraestrutura/AcessoExclusivoFilter.cs

```
using Microsoft.AspNetCore.Mvc.Filters;

namespace MediCare.Web.Infraestrutura;

/// <summary>
/// Processa uma requisição por vez. Assim, duas pessoas não conseguem reservar o mesmo
/// horário ao mesmo tempo, e a opção de persistência em CSV, que não suporta gravações
/// simultâneas, também fica protegida.
/// </summary>
public class AcessoExclusivoFilter : IAsyncPageFilter
{
    private static readonly SemaphoreSlim Trava = new(1, 1);

    public Task OnPageHandlerSelectionAsync(PageHandlerSelectedContext context) => Task.CompletedTask;

    public async Task OnPageHandlerExecutionAsync(PageHandlerExecutingContext context, PageHandlerExecutionDelegate next)
    {
        await Trava.WaitAsync();
        try
        {
            await next();
        }
        finally
        {
            Trava.Release();
        }
    }
}
```

MediCare.Web/Infraestrutura/Apresentacao.cs

```
using System.ComponentModel.DataAnnotations;
using System.Reflection;
using MediCare.Core.Models;
using MediCare.Core.Repositories;

namespace MediCare.Web.Infraestrutura;

/// <summary>
/// Textos exibidos nas telas a partir dos identificadores guardados nas entidades.
/// </summary>
public class Apresentacao(MediCareDados dados)
{
    public HorarioDisponivel Horario(Consulta consulta) => dados.Horarios.ObterPorId(consulta.HorarioId)!;

    public string Paciente(Consulta consulta) => dados.Pacientes.ObterPorId(consulta.PacienteId)!.Nome;

    public string Medico(int medicoId) => dados.Medicos.ObterPorId(medicoId)!.Nome;

    public string Medico(Consulta consulta) => Medico(Horario(consulta).MedicoId);

    public string Especialidade(int especialidadeId) =>
        dados.Especialidades.ObterPorId(especialidadeId)?.Nome ?? "";

    public string DataHora(HorarioDisponivel h) =>
        $"{h.Data:dd/MM/yyyy} {h.HoraInicio:HH:mm}–{h.HoraFim:HH:mm}";

    public string Local(HorarioDisponivel h) =>
        h.LocalId is int id && dados.Locais.ObterPorId(id) is { } local
            ? $"{local.Nome} — {local.Endereco}, {local.Cidade}"
            : "Teleconsulta";

    /// <summary>Nome de exibição de um valor de enumeração (atributo Display, quando houver).</summary>
    public static string Nome(Enum valor) =>
        valor.GetType().GetField(valor.ToString())?.GetCustomAttribute<DisplayAttribute>()?.Name ?? valor.ToString();
}
```

MediCare.Web/Infraestrutura/PaginaAgendamento.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;
using Microsoft.AspNetCore.Mvc;

namespace MediCare.Web.Infraestrutura;

/// <summary>
/// Base das páginas de agendamento, usadas pelo paciente e pela recepcionista.
/// O paciente sempre age sobre a própria conta; a recepcionista informa o paciente que está atendendo.
/// </summary>
public abstract class PaginaAgendamento(MediCareDados dados) : PaginaBase
{
    protected MediCareDados Dados { get; } = dados;

    [BindProperty(SupportsGet = true)] public int? PacienteId { get; set; }
```

```
    public Paciente Paciente { get; private set; } = null!;

    public bool AtendimentoPelaRecepcao => User.IsInRole(Perfis.Recepcionista);

    protected bool CarregarPaciente()
    {
        var id = User.IsInRole(Perfis.Paciente) ? UsuarioLogado.Id(User) : PacienteId;
        var paciente = id is int i ? Dados.Pacientes.ObterPorId(i) : null;
        if (paciente is null)
            return false;

        Paciente = paciente;
        PacienteId = paciente.Id;
        return true;
    }

    protected IActionResult SemPaciente()
    {
        Erro = "Selecione o paciente que está sendo atendido.";
        return RedirectToPage("/Recepcao/Index");
    }
}
```

MediCare.Web/Infraestrutura/PaginaBase.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace MediCare.Web.Infraestrutura;

/// <summary>
/// Base das páginas: guarda as mensagens de sucesso e de erro exibidas ao usuário.
/// </summary>
public abstract class PaginaBase : PageModel
{
    [TempData] public string? Mensagem { get; set; }
    [TempData] public string? Erro { get; set; }

    /// <summary>
    /// Executa uma operação e, se uma regra de negócio for violada, mostra o motivo na tela.
    /// </summary>
    protected bool Tentar(Action operacao, string? sucesso = null)
    {
        try
        {
            operacao();
            Mensagem = sucesso;
            return true;
        }
        catch (Exception ex) when (ex is InvalidOperationException or ArgumentException)
        {
            Erro = ex.Message;
            return false;
        }
    }
}
```

MediCare.Web/Infraestrutura/Perfis.cs

```
namespace MediCare.Web.Infraestrutura;

/// <summary>
/// Nomes dos perfis de acesso (roles) e das políticas de autorização.
/// </summary>
public static class Perfis
{
    public const string Paciente = "Paciente";
    public const string Recepcionista = "Recepcionista";
    public const string Medico = "Medico";
    public const string Administrador = "Administrador";

    // Políticas que reúnem mais de um perfil.
    public const string Agendamento = "Agendamento";
    public const string Teleconsulta = "Teleconsulta";
}
```

MediCare.Web/Infraestrutura/UsuarioLogado.cs

```
using System.Security.Claims;
using MediCare.Core.Models;
using MediCare.Core.Repositories;
using Microsoft.AspNetCore.Authentication.Cookies;

namespace MediCare.Web.Infraestrutura;

/// <summary>
/// Converte o usuário autenticado (cookie) no objeto do modelo e vice-versa.
/// </summary>
public class UsuarioLogado(MediCareDados dados)
```

```
{
    public static ClaimsPrincipal CriarIdentidade(Usuario usuario)
    {
        var claims = new List<Claim>
        {
            new(ClaimTypes.NameIdentifier, usuario.Id.ToString()),
            new(ClaimTypes.Name, usuario.Nome),
            new(ClaimTypes.Email, usuario.Email),
            new(ClaimTypes.Role, Perfil(usuario))
        };
        return new ClaimsPrincipal(new ClaimsIdentity(claims, CookieAuthenticationDefaults.AuthenticationScheme));
    }

    public static string Perfil(Usuario usuario) => usuario switch
    {
        Paciente => Perfis.Paciente,
        Recepcionista => Perfis.Recepcionista,
        Medico => Perfis.Medico,
        Administrador => Perfis.Administrador,
        _ => throw new ArgumentOutOfRangeException(nameof(usuario))
    };

    public static int Id(ClaimsPrincipal principal) =>
        int.Parse(principal.FindFirstValue(ClaimTypes.NameIdentifier)!);

    public Usuario Obter(ClaimsPrincipal principal)
    {
        var id = Id(principal);
        Usuario? usuario =
            principal.IsInRole(Perfis.Paciente) ? dados.Pacientes.ObterPorId(id) :
            principal.IsInRole(Perfis.Recepcionista) ? dados.Recepcionistas.ObterPorId(id) :
            principal.IsInRole(Perfis.Medico) ? dados.Medicos.ObterPorId(id) :
            principal.IsInRole(Perfis.Administrador) ? dados.Administradores.ObterPorId(id) :
            null;
        return usuario ?? throw new InvalidOperationException("Usuário não encontrado.");
    }

    public Paciente ObterPaciente(ClaimsPrincipal principal) => (Paciente)Obter(principal);

    public Medico ObterMedico(ClaimsPrincipal principal) => (Medico)Obter(principal);
}
```

MediCare.Web/Pages/Erro.cshtml

```
@page
@{
    ViewData["Title"] = "Erro";
}
<h1>Ocorreu um erro</h1>
<p>Não foi possível concluir a operação. Tente novamente.</p>
<p><a href="/">Voltar ao início</a></p>
```

MediCare.Web/Pages/Index.cshtml

```
@page
@model IndexModel
@{
    ViewData["Title"] = "Início";
}
<section class="destaque">
    <h1>MediCare</h1>
    <p>Encontre médicos por especialidade, veja os horários disponíveis e agende consultas
       presenciais ou por teleconsulta.</p>
    <p>
        <a class="botao" asp-page="/Conta/Entrar">Entrar</a>
        <a class="botao secundario" asp-page="/Conta/CriarConta">Criar conta de paciente</a>
    </p>
    <p class="discreto">É médico? <a asp-page="/Conta/CriarContaMedico">Cadastre-se aqui</a>.</p>
</section>
```

MediCare.Web/Pages/Index.cshtml.cs

```
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace MediCare.Web.Pages;

public class IndexModel : PageModel
{
    // Usuário autenticado vai direto para a tela inicial do seu perfil.
    public IActionResult OnGet()
    {
        if (User.IsInRole(Perfis.Paciente)) return RedirectToPage("/Agendamento/Consultas");
        if (User.IsInRole(Perfis.Recepcionista)) return RedirectToPage("/Recepcao/Index");
        if (User.IsInRole(Perfis.Medico)) return RedirectToPage("/AreaMedico/Index");
        if (User.IsInRole(Perfis.Administrador)) return RedirectToPage("/Admin/Usuarios");
        return Page();
    }
```

```
}
```

MediCare.Web/Pages/_ViewImports.cshtml

```
@using MediCare.Core.Models
@using MediCare.Web.Infraestrutura
@namespace MediCare.Web.Pages
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

MediCare.Web/Pages/_ViewStart.cshtml

```
@{
    Layout = "_Layout";
}
```

MediCare.Web/Pages/Admin/Agendamentos.cshtml

```
@page
@model AgendamentosModel
@inject Apresentacao Texto
@{
    ViewData["Title"] = "Agendamentos";
}
<h1>Agendamentos</h1>
<partial name="_Mensagens" model="Model" />

<form method="get" class="filtro">
    <label asp-for="Status">Situação</label>
    <select asp-for="Status" asp-items="Html.GetEnumSelectList<StatusConsulta>()">
        <option value="">Todas</option>
    </select>
    <button type="submit" class="botao">Filtrar</button>
</form>

<table>
    <thead><tr><th>Data e hora</th><th>Paciente</th><th>Médico</th><th>Local</th><th>Situação</th><th></th></tr></thead>
    <tbody>
    @foreach (var c in Model.Consultas)
    {
        var h = Texto.Horario(c);
        <tr>
            <td>@Texto.DataHora(h)</td>
            <td>@Texto.Paciente(c)</td>
            <td>@Texto.Medico(c)</td>
            <td>@Texto.Local(h)</td>
            <td><span class="situacao @c.Status.ToString().ToLower()">@Apresentacao.Nome(c.Status)</span></td>
            <td>
                @if (c.Status == StatusConsulta.Agendada)
                {
                    <form method="post" class="inline" onsubmit="return confirm('Cancelar esta consulta?');">
                        <input type="hidden" name="consultaId" value="@c.Id" />
                        <button type="submit" class="link perigo">Cancelar</button>
                    </form>
                }
            </td>
        </tr>
    }
    </tbody>
</table>
```

MediCare.Web/Pages/Admin/Agendamentos.cshtml.cs

```
using MediCare.Core.Models;
using MediCare.Core.Services;
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Mvc;

namespace MediCare.Web.Pages.Admin;

/// <summary>
/// UC13 — Gerenciar agendamentos.
/// </summary>
public class AgendamentosModel(AgendamentoService agendamento) : PaginaBase
{
    [BindProperty(SupportsGet = true)]
    public StatusConsulta? Status { get; set; }

    public IReadOnlyList<Consulta> Consultas { get; private set; } = [];

    public void OnGet() => Consultas = agendamento.ListarAgendamentos(Status).ToList();

    public IActionResult OnPost(int consultaId)
    {
        Tentar(() => agendamento.Cancelar(consultaId), "Consulta cancelada e horário liberado.");
        return RedirectToPage(new { Status });
    }
}
```

MediCare.Web/Pages/Admin/Notificacoes.cshtml

```
@page
@model NotificacoesModel
@{
    ViewData["Title"] = "Notificações";
}
<h1>Notificações</h1>
<partial name="_Mensagens" model="Model" />

<div class="botoes">
    <form method="post" asp-page-handler="GerarLembretes" class="inline">
        <button type="submit" class="botao secundario">Gerar lembretes de amanhã</button>
    </form>
    <form method="post" asp-page-handler="Enviar" class="inline">
        <button type="submit" class="botao">Enviar pendentes (@Model.Pendentes)</button>
    </form>
</div>

<table>
    <thead><tr><th>Criada em</th><th>Tipo</th><th>Canal</th><th>Destino</th><th>Mensagem</th><th>Situação</th></tr></thead>
    <tbody>
    @foreach (var n in Model.Notificacoes)
    {
        <tr>
            <td>@n.DataCriacao.ToString("dd/MM/yyyy HH:mm")</td>
            <td>@Apresentacao.Nome(n.Tipo)</td>
            <td>@Apresentacao.Nome(n.Canal)</td>
            <td>@n.Destino</td>
            <td class="texto-longo">@n.Mensagem</td>
            <td>@(n.Status == StatusNotificacao.Enviada ? $"Enviada {n.DataEnvio:dd/MM HH:mm}" : "Pendente")</td>
        </tr>
    }
    </tbody>
</table>
```

MediCare.Web/Pages/Admin/Notificacoes.cshtml.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;
using MediCare.Core.Services;
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Mvc;

namespace MediCare.Web.Pages.Admin;

/// <summary>
/// UC14 — Enviar notificações. A entrega real depende do serviço externo (e-mail, SMS ou WhatsApp);
/// aqui as notificações pendentes são registradas como entregues a esse serviço.
/// </summary>
public class NotificacoesModel(MediCareDados dados, NotificacaoService notificacoes) : PaginaBase
{
    public IReadOnlyList<Notificacao> Notificacoes { get; private set; } = [];
    public int Pendentes { get; private set; }

    public void OnGet()
    {
        Notificacoes = dados.Notificacoes.ListarTodos().OrderByDescending(n => n.DataCriacao).ToList();
        Pendentes = Notificacoes.Count(n => n.Status == StatusNotificacao.Pendente);
    }

    public IActionResult OnPostGerarLembretes()
    {
        var amanha = DateOnly.FromDateTime(DateTime.Today.AddDays(1));
        var quantidade = notificacoes.GerarLembretes(amanha);
        Mensagem = $"{quantidade} lembrete(s) gerado(s) para {amanha:dd/MM/yyyy}.";
        return RedirectToPage();
    }

    public IActionResult OnPostEnviar()
    {
        var pendentes = notificacoes.ListarPendentes().ToList();
        foreach (var n in pendentes)
            notificacoes.MarcarComoEnviada(n.Id);
        Mensagem = $"{pendentes.Count} notificação(ões) enviada(s).";
        return RedirectToPage();
    }
}
```

MediCare.Web/Pages/Admin/Usuarios.cshtml

```
@page
@model UsuariosModel
@{
    ViewData["Title"] = "Usuários";
}
<h1>Usuários</h1>
<partial name="_Mensagens" model="Model" />
<table>
```

```
    <thead><tr><th>Perfil</th><th>Nome</th><th>E-mail</th><th>Login</th><th>Situação</th><th></th></tr></thead>
    <tbody>
    @foreach (var u in Model.Usuarios)
    {
        <tr>
            <td>@UsuarioLogado.Perfil(u)</td>
            <td>@u.Nome</td>
            <td>@u.Email</td>
            <td>@Apresentacao.Nome(u.Provedor)</td>
            <td><span class="situacao @(u.Ativo ? "agendada" : "cancelada")">@(u.Ativo ? "Ativo" : "Desativado")</span></td>
            <td>
                @if (!(u is Administrador && u.Id == Model.AdministradorId))
                {
                    <form method="post" class="inline">
                        <input type="hidden" name="email" value="@u.Email" />
                        <input type="hidden" name="ativo" value="@(!u.Ativo)" />
                        <button type="submit" class="link @(u.Ativo ? "perigo" : "")">
                            @(u.Ativo ? "Desativar" : "Ativar")
                        </button>
                    </form>
                }
            </td>
        </tr>
    }
    </tbody>
</table>
```

MediCare.Web/Pages/Admin/Usuarios.cshtml.cs

```
using MediCare.Core.Models;
using MediCare.Core.Services;
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Mvc;

namespace MediCare.Web.Pages.Admin;

/// <summary>
/// UC12 — Gerenciar usuários.
/// </summary>
public class UsuariosModel(ContaService contas) : PaginaBase
{
    public IReadOnlyList<Usuario> Usuarios { get; private set; } = [];

    // O administrador logado não pode desativar a própria conta.
    public int AdministradorId => UsuarioLogado.Id(User);

    public void OnGet() => Usuarios = contas.ListarUsuarios().ToList();

    public IActionResult OnPost(string email, bool ativo)
    {
        Tentar(() => contas.AlterarSituacao(email, ativo), ativo ? "Usuário ativado." : "Usuário desativado.");
        return RedirectToPage();
    }
}
```

MediCare.Web/Pages/Agendamento/Consultas.cshtml

```
@page
@model ConsultasModel
@inject Apresentacao Texto
@{
    ViewData["Title"] = "Consultas";
}
<partial name="_Paciente" model="Model" />
<h1>@(Model.AtendimentoPelaRecepcao ? "Consultas do paciente" : "Minhas consultas")</h1>
<partial name="_Mensagens" model="Model" />

<p><a class="botao" asp-page="Medicos" asp-route-pacienteId="@Model.PacienteId">Agendar nova consulta</a></p>

@if (!Model.Consultas.Any())
{
    <p>Nenhuma consulta encontrada.</p>
}
else
{
    <table>
        <thead><tr><th>Data e hora</th><th>Médico</th><th>Local</th><th>Situação</th><th></th></tr></thead>
        <tbody>
        @foreach (var c in Model.Consultas)
        {
            var h = Texto.Horario(c);
            <tr>
                <td>@Texto.DataHora(h)</td>
                <td>@Texto.Medico(c)</td>
                <td>@Texto.Local(h)</td>
                <td><span class="situacao @c.Status.ToString().ToLower()">@Apresentacao.Nome(c.Status)</span></td>
                <td class="acoes">
                    @if (c.Status == StatusConsulta.Agendada)
                    {
                        @if (h.Modalidade == Modalidade.Teleconsulta && !Model.AtendimentoPelaRecepcao)
```

```
                        {
                            <a asp-page="/Teleconsulta/Acessar" asp-route-consultaId="@c.Id">Acessar</a>
                        }
                        <a asp-page="Reagendar" asp-route-consultaId="@c.Id"
                           asp-route-pacienteId="@Model.PacienteId">Reagendar</a>
                        <form method="post" asp-page-handler="Cancelar" class="inline"
                              onsubmit="return confirm('Cancelar esta consulta?');">
                            <input type="hidden" name="consultaId" value="@c.Id" />
                            <input type="hidden" asp-for="PacienteId" />
                            <button type="submit" class="link perigo">Cancelar</button>
                        </form>
                    }
                </td>
            </tr>
        }
        </tbody>
    </table>
}
```

MediCare.Web/Pages/Agendamento/Consultas.cshtml.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;
using MediCare.Core.Services;
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Mvc;

namespace MediCare.Web.Pages.Agendamento;

/// <summary>
/// Consultas do paciente e UC06 — Cancelar consulta.
/// </summary>
public class ConsultasModel(MediCareDados dados, AgendamentoService agendamento, Apresentacao texto)
    : PaginaAgendamento(dados)
{
    public IReadOnlyList<Consulta> Consultas { get; private set; } = [];

    public IActionResult OnGet()
    {
        if (!CarregarPaciente())
            return SemPaciente();

        Consultas = agendamento.ConsultasDoPaciente(Paciente.Id)
            .OrderByDescending(c => texto.Horario(c).Data)
            .ThenByDescending(c => texto.Horario(c).HoraInicio)
            .ToList();
        return Page();
    }

    public IActionResult OnPostCancelar(int consultaId)
    {
        if (!CarregarPaciente())
            return SemPaciente();

        var consulta = Dados.Consultas.ObterPorId(consultaId);
        if (consulta is null || consulta.PacienteId != Paciente.Id)
            return NotFound();

        Tentar(() => agendamento.Cancelar(consultaId), "Consulta cancelada e horário liberado.");
        return RedirectToPage(new { pacienteId = PacienteId });
    }
}
```

MediCare.Web/Pages/Agendamento/Horarios.cshtml

```
@page
@model HorariosModel
@inject Apresentacao Texto
@{
    ViewData["Title"] = "Horários disponíveis";
}
<partial name="_Paciente" model="Model" />
<h1>Horários de @Model.Medico.Nome</h1>
<p class="discreto">@Texto.Especialidade(Model.Medico.EspecialidadeId) · @Model.Medico.Crm</p>
<partial name="_Mensagens" model="Model" />

@if (!Model.Horarios.Any())
{
    <p>Não há horários disponíveis para este médico.</p>
}
else
{
    <table>
        <thead><tr><th>Data e hora</th><th>Modalidade</th><th>Local</th><th></th></tr></thead>
        <tbody>
        @foreach (var h in Model.Horarios)
        {
            <tr>
                <td>@Texto.DataHora(h)</td>
                <td>@h.Modalidade</td>
```

```
                <td>@Texto.Local(h)</td>
                <td>
                    <form method="post">
                        <input type="hidden" name="horarioId" value="@h.Id" />
                        <input type="hidden" asp-for="PacienteId" />
                        <input type="hidden" name="medicoId" value="@Model.Medico.Id" />
                        <button type="submit" class="botao pequeno">Agendar</button>
                    </form>
                </td>
            </tr>
        }
        </tbody>
    </table>
}
<p><a asp-page="Medicos" asp-route-pacienteId="@Model.PacienteId">Voltar à pesquisa</a></p>
```

MediCare.Web/Pages/Agendamento/Horarios.cshtml.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;
using MediCare.Core.Services;
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Mvc;

namespace MediCare.Web.Pages.Agendamento;

/// <summary>
/// UC04 — Consultar horários disponíveis e UC05 — Agendar consulta.
/// </summary>
public class HorariosModel(MediCareDados dados, AgendamentoService agendamento) : PaginaAgendamento(dados)
{
    public Medico Medico { get; private set; } = null!;
    public IReadOnlyList<HorarioDisponivel> Horarios { get; private set; } = [];

    public IActionResult OnGet(int medicoId)
    {
        if (!CarregarPaciente())
            return SemPaciente();
        if (Dados.Medicos.ObterPorId(medicoId) is not { } medico)
            return NotFound();

        Medico = medico;
        Horarios = agendamento.HorariosDisponiveis(medicoId).ToList();
        return Page();
    }

    public IActionResult OnPost(int medicoId, int horarioId)
    {
        if (!CarregarPaciente())
            return SemPaciente();

        var sucesso = $"Consulta agendada. A confirmação foi enviada por {Apresentacao.Nome(Paciente.CanalPreferido)}.";
        if (!Tentar(() => agendamento.Agendar(Paciente.Id, horarioId), sucesso))
            return RedirectToPage(new { medicoId, pacienteId = PacienteId });

        return RedirectToPage("Consultas", new { pacienteId = PacienteId });
    }
}
```

MediCare.Web/Pages/Agendamento/Medicos.cshtml

```
@page
@model MedicosModel
@inject Apresentacao Texto
@{
    ViewData["Title"] = "Pesquisar médicos";
}
<partial name="_Paciente" model="Model" />
<h1>Pesquisar médicos</h1>
<partial name="_Mensagens" model="Model" />

<form method="get" class="filtro">
    <input type="hidden" asp-for="PacienteId" />
    <label for="especialidadeId">Especialidade</label>
    <select id="especialidadeId" name="especialidadeId" asp-items="Model.Especialidades">
        <option value="">Todas</option>
    </select>
    <button type="submit" class="botao">Pesquisar</button>
</form>

@if (!Model.Medicos.Any())
{
    <p>Nenhum médico encontrado.</p>
}
else
{
    <table>
        <thead><tr><th>Médico</th><th>Especialidade</th><th>CRM</th><th></th></tr></thead>
        <tbody>
        @foreach (var m in Model.Medicos)
```

```
        {
            <tr>
                <td>@m.Nome</td>
                <td>@Texto.Especialidade(m.EspecialidadeId)</td>
                <td>@m.Crm</td>
                <td>
                    <a asp-page="Horarios" asp-route-medicoId="@m.Id" asp-route-pacienteId="@Model.PacienteId">
                        Ver horários
                    </a>
                </td>
            </tr>
        }
        </tbody>
    </table>
}
```

MediCare.Web/Pages/Agendamento/Medicos.cshtml.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;
using MediCare.Core.Services;
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;

namespace MediCare.Web.Pages.Agendamento;

/// <summary>
/// UC03 — Pesquisar médicos.
/// </summary>
public class MedicosModel(MediCareDados dados, AgendamentoService agendamento) : PaginaAgendamento(dados)
{
    public IEnumerable<SelectListItem> Especialidades { get; private set; } = [];
    public IReadOnlyList<Medico> Medicos { get; private set; } = [];

    public IActionResult OnGet(int? especialidadeId)
    {
        if (!CarregarPaciente())
            return SemPaciente();

        Especialidades = Dados.Especialidades.ListarTodos()
            .Select(e => new SelectListItem(e.Nome, e.Id.ToString(), e.Id == especialidadeId));
        Medicos = agendamento.PesquisarMedicos(especialidadeId).ToList();
        return Page();
    }
}
```

MediCare.Web/Pages/Agendamento/Reagendar.cshtml

```
@page
@model ReagendarModel
@inject Apresentacao Texto
@{
    ViewData["Title"] = "Reagendar consulta";
    var atual = Texto.Horario(Model.Consulta);
}
<partial name="_Paciente" model="Model" />
<h1>Reagendar consulta</h1>
<partial name="_Mensagens" model="Model" />
<p>Consulta atual: <strong>@Texto.DataHora(atual)</strong> com Dr(a). @Texto.Medico(Model.Consulta)
   — @Texto.Local(atual)</p>

@if (!Model.Horarios.Any())
{
    <p>Não há outros horários disponíveis com este médico.</p>
}
else
{
    <table>
        <thead><tr><th>Novo horário</th><th>Modalidade</th><th>Local</th><th></th></tr></thead>
        <tbody>
        @foreach (var h in Model.Horarios)
        {
            <tr>
                <td>@Texto.DataHora(h)</td>
                <td>@h.Modalidade</td>
                <td>@Texto.Local(h)</td>
                <td>
                    <form method="post">
                        <input type="hidden" name="consultaId" value="@Model.Consulta.Id" />
                        <input type="hidden" name="novoHorarioId" value="@h.Id" />
                        <input type="hidden" asp-for="PacienteId" />
                        <button type="submit" class="botao pequeno">Escolher</button>
                    </form>
                </td>
            </tr>
        }
        </tbody>
    </table>
}
```

```
<p><a asp-page="Consultas" asp-route-pacienteId="@Model.PacienteId">Voltar às consultas</a></p>
```

MediCare.Web/Pages/Agendamento/Reagendar.cshtml.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;
using MediCare.Core.Services;
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Mvc;

namespace MediCare.Web.Pages.Agendamento;

/// <summary>
/// UC07 — Reagendar consulta.
/// </summary>
public class ReagendarModel(MediCareDados dados, AgendamentoService agendamento) : PaginaAgendamento(dados)
{
    public Consulta Consulta { get; private set; } = null!;
    public IReadOnlyList<HorarioDisponivel> Horarios { get; private set; } = [];

    public IActionResult OnGet(int consultaId)
    {
        if (!CarregarPaciente())
            return SemPaciente();
        if (!CarregarConsulta(consultaId))
            return NotFound();

        var medicoId = Dados.Horarios.ObterPorId(Consulta.HorarioId)!.MedicoId;
        Horarios = agendamento.HorariosDisponiveis(medicoId).ToList();
        return Page();
    }

    public IActionResult OnPost(int consultaId, int novoHorarioId)
    {
        if (!CarregarPaciente())
            return SemPaciente();
        if (!CarregarConsulta(consultaId))
            return NotFound();

        if (!Tentar(() => agendamento.Reagendar(consultaId, novoHorarioId), "Consulta reagendada."))
            return RedirectToPage(new { consultaId, pacienteId = PacienteId });

        return RedirectToPage("Consultas", new { pacienteId = PacienteId });
    }

    private bool CarregarConsulta(int consultaId)
    {
        var consulta = Dados.Consultas.ObterPorId(consultaId);
        if (consulta is null || consulta.PacienteId != Paciente.Id || consulta.Status != StatusConsulta.Agendada)
            return false;
        Consulta = consulta;
        return true;
    }
}
```

MediCare.Web/Pages/Agendamento/_Paciente.cshtml

```
@model PaginaAgendamento
@if (Model.AtendimentoPelaRecepcao)
{
    <p class="faixa">Atendendo: <strong>@Model.Paciente.Nome</strong> (@Model.Paciente.Email)
        — <a asp-page="/Recepcao/Index">trocar paciente</a></p>
}
```

MediCare.Web/Pages/AreaMedico/Historico.cshtml

```
@page
@model HistoricoModel
@inject Apresentacao Texto
@{
    ViewData["Title"] = "Histórico do paciente";
}
<h1>Histórico do paciente</h1>
<partial name="_Mensagens" model="Model" />

<form method="get" class="filtro">
    <label for="email">E-mail do paciente</label>
    <input id="email" name="email" value="@Model.Email" />
    <button type="submit" class="botao">Consultar</button>
</form>

@if (Model.Paciente is not null)
{
    <h2>@Model.Paciente.Nome</h2>
    @if (!Model.Consultas.Any())
    {
        <p>Nenhuma consulta realizada.</p>
    }
    else
```

```
    {
        <table>
            <thead><tr><th>Data e hora</th><th>Médico</th><th>Local</th></tr></thead>
            <tbody>
            @foreach (var c in Model.Consultas)
            {
                var h = Texto.Horario(c);
                <tr>
                    <td>@Texto.DataHora(h)</td>
                    <td>@Texto.Medico(c)</td>
                    <td>@Texto.Local(h)</td>
                </tr>
            }
            </tbody>
        </table>
    }
}
```

MediCare.Web/Pages/AreaMedico/Historico.cshtml.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;
using MediCare.Core.Services;
using MediCare.Web.Infraestrutura;

namespace MediCare.Web.Pages.AreaMedico;

/// <summary>
/// UC15 — Consultar histórico do paciente.
/// </summary>
public class HistoricoModel(MediCareDados dados, ContaService contas, AtendimentoService atendimento) : PaginaBase
{
    public string? Email { get; private set; }
    public Paciente? Paciente { get; private set; }
    public IReadOnlyList<Consulta> Consultas { get; private set; } = [];

    public void OnGet(int? pacienteId, string? email)
    {
        Email = email;
        var paciente = pacienteId is int id
            ? dados.Pacientes.ObterPorId(id)
            : string.IsNullOrWhiteSpace(email) ? null : contas.ObterPorEmail(email) as Paciente;

        if (paciente is null)
        {
            if (pacienteId is not null || !string.IsNullOrWhiteSpace(email))
                Erro = "Paciente não encontrado.";
            return;
        }

        Email = paciente.Email;
        if (Tentar(() => Consultas = atendimento.HistoricoDoPaciente(UsuarioLogado.Id(User), paciente.Id).ToList()))
            Paciente = paciente;
    }
}
```

MediCare.Web/Pages/AreaMedico/Horarios.cshtml

```
@page
@model HorariosModel
@inject Apresentacao Texto
@{
    ViewData["Title"] = "Meus horários";
}
<h1>Meus horários</h1>
<partial name="_Mensagens" model="Model" />

<div class="cartao">
    <h2>Novo horário</h2>
    <form method="post" asp-page-handler="Cadastrar" class="grade">
        <div>
            <label asp-for="Data">Data</label>
            <input asp-for="Data" type="date" />
            <span asp-validation-for="Data" class="erro-campo"></span>
        </div>
        <div>
            <label asp-for="Inicio">Início</label>
            <input asp-for="Inicio" type="time" />
            <span asp-validation-for="Inicio" class="erro-campo"></span>
        </div>
        <div>
            <label asp-for="Fim">Término</label>
            <input asp-for="Fim" type="time" />
            <span asp-validation-for="Fim" class="erro-campo"></span>
        </div>
        <div>
            <label asp-for="Modalidade">Modalidade</label>
            <select asp-for="Modalidade" asp-items="Html.GetEnumSelectList<Modalidade>()"></select>
        </div>
        <div>
```

```
            <label asp-for="LocalId">Local (presencial)</label>
            <select asp-for="LocalId" asp-items="Model.Locais">
                <option value="">—</option>
            </select>
        </div>
        <div class="alinhar-base">
            <button type="submit" class="botao">Cadastrar</button>
        </div>
    </form>
</div>

<table>
    <thead><tr><th>Data e hora</th><th>Modalidade</th><th>Local</th><th>Situação</th><th></th></tr></thead>
    <tbody>
    @foreach (var h in Model.Horarios)
    {
        <tr>
            <td>@Texto.DataHora(h)</td>
            <td>@h.Modalidade</td>
            <td>@Texto.Local(h)</td>
            <td>@(h.Disponivel ? "Livre" : "Reservado")</td>
            <td>
                @if (h.Disponivel)
                {
                    <form method="post" asp-page-handler="Remover" class="inline">
                        <input type="hidden" name="horarioId" value="@h.Id" />
                        <button type="submit" class="link perigo">Remover</button>
                    </form>
                }
            </td>
        </tr>
    }
    </tbody>
</table>
```

MediCare.Web/Pages/AreaMedico/Horarios.cshtml.cs

```
using System.ComponentModel.DataAnnotations;
using MediCare.Core.Models;
using MediCare.Core.Services;
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;

namespace MediCare.Web.Pages.AreaMedico;

/// <summary>
/// UC09 — Gerenciar disponibilidade.
/// </summary>
public class HorariosModel(DisponibilidadeService disponibilidade) : PaginaBase
{
    [BindProperty, Required(ErrorMessage = "Informe a data.")]
    public DateOnly? Data { get; set; }

    [BindProperty, Required(ErrorMessage = "Informe o início.")]
    public TimeOnly? Inicio { get; set; }

    [BindProperty, Required(ErrorMessage = "Informe o término.")]
    public TimeOnly? Fim { get; set; }

    [BindProperty]
    public Modalidade Modalidade { get; set; }

    [BindProperty]
    public int? LocalId { get; set; }

    public IEnumerable<SelectListItem> Locais { get; private set; } = [];
    public IReadOnlyList<HorarioDisponivel> Horarios { get; private set; } = [];

    private int MedicoId => UsuarioLogado.Id(User);

    public void OnGet() => Carregar();

    public IActionResult OnPostCadastrar()
    {
        if (!ModelState.IsValid)
        {
            Carregar();
            return Page();
        }

        if (!Tentar(() => disponibilidade.CadastrarHorario(
                MedicoId, Data!.Value, Inicio!.Value, Fim!.Value, Modalidade, LocalId), "Horário cadastrado."))
        {
            Carregar();
            return Page();
        }
        return RedirectToPage();
    }

    public IActionResult OnPostRemover(int horarioId)
```

```
    {
        Tentar(() => disponibilidade.RemoverHorario(MedicoId, horarioId), "Horário removido.");
        return RedirectToPage();
    }

    private void Carregar()
    {
        Locais = disponibilidade.LocaisDoMedico(MedicoId).Select(l => new SelectListItem(l.Nome, l.Id.ToString()));
        Horarios = disponibilidade.HorariosDoMedico(MedicoId).ToList();
    }
}
```

MediCare.Web/Pages/AreaMedico/Index.cshtml

```
@page
@model MediCare.Web.Pages.AreaMedico.IndexModel
@inject Apresentacao Texto
@{
    ViewData["Title"] = "Agenda";
}
<h1>Minha agenda</h1>
<partial name="_Mensagens" model="Model" />

@if (!Model.Consultas.Any())
{
    <p>Nenhuma consulta agendada.</p>
}
else
{
    <table>
        <thead><tr><th>Data e hora</th><th>Paciente</th><th>Local</th><th></th></tr></thead>
        <tbody>
        @foreach (var c in Model.Consultas)
        {
            var h = Texto.Horario(c);
            <tr>
                <td>@Texto.DataHora(h)</td>
                <td>@Texto.Paciente(c)</td>
                <td>@Texto.Local(h)</td>
                <td class="acoes">
                    @if (h.Modalidade == Modalidade.Teleconsulta)
                    {
                        <a asp-page="/Teleconsulta/Acessar" asp-route-consultaId="@c.Id">Acessar</a>
                    }
                    <a asp-page="Historico" asp-route-pacienteId="@c.PacienteId">Histórico</a>
                    <form method="post" asp-page-handler="Concluir" class="inline">
                        <input type="hidden" name="consultaId" value="@c.Id" />
                        <button type="submit" class="link">Registrar como realizada</button>
                    </form>
                </td>
            </tr>
        }
        </tbody>
    </table>
}
```

MediCare.Web/Pages/AreaMedico/Index.cshtml.cs

```
using MediCare.Core.Models;
using MediCare.Core.Services;
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Mvc;

namespace MediCare.Web.Pages.AreaMedico;

/// <summary>
/// UC10 — Gerenciar agenda.
/// </summary>
public class IndexModel(AtendimentoService atendimento) : PaginaBase
{
    public IReadOnlyList<Consulta> Consultas { get; private set; } = [];

    public void OnGet() => Consultas = atendimento.Agenda(UsuarioLogado.Id(User)).ToList();

    public IActionResult OnPostConcluir(int consultaId)
    {
        Tentar(() => atendimento.ConcluirConsulta(UsuarioLogado.Id(User), consultaId),
            "Consulta registrada como realizada.");
        return RedirectToPage();
    }
}
```

MediCare.Web/Pages/AreaMedico/Perfil.cshtml

```
@page
@model PerfilModel
@{
    ViewData["Title"] = "Meu perfil";
}
```

```
<div class="cartao estreito">
    <h1>Meu perfil</h1>
    <partial name="_Mensagens" model="Model" />
    <p><strong>@Model.Medico.Nome</strong><br />@Model.Medico.Email<br />@Model.Medico.Crm</p>
    <form method="post">
        <label asp-for="Telefone">Telefone</label>
        <input asp-for="Telefone" />
        <span asp-validation-for="Telefone" class="erro-campo"></span>
        <button type="submit" class="botao">Salvar</button>
    </form>
</div>
```

MediCare.Web/Pages/AreaMedico/Perfil.cshtml.cs

```
using System.ComponentModel.DataAnnotations;
using MediCare.Core.Models;
using MediCare.Core.Services;
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Mvc;

namespace MediCare.Web.Pages.AreaMedico;

/// <summary>
/// UC08 — Gerenciar perfil (médico).
/// </summary>
public class PerfilModel(ContaService contas, UsuarioLogado usuarioLogado) : PaginaBase
{
    public Medico Medico { get; private set; } = null!;

    [BindProperty, Required(ErrorMessage = "Informe o telefone.")]
    public string Telefone { get; set; } = "";

    public void OnGet()
    {
        Medico = usuarioLogado.ObterMedico(User);
        Telefone = Medico.Telefone;
    }

    public IActionResult OnPost()
    {
        Medico = usuarioLogado.ObterMedico(User);
        if (!ModelState.IsValid)
            return Page();

        contas.AtualizarPerfil(Medico, Telefone);
        Mensagem = "Dados atualizados.";
        return RedirectToPage();
    }
}
```

MediCare.Web/Pages/AreaPaciente/Perfil.cshtml

```
@page
@model PerfilModel
@{
    ViewData["Title"] = "Meu perfil";
}
<div class="cartao estreito">
    <h1>Meu perfil</h1>
    <partial name="_Mensagens" model="Model" />
    <p><strong>@Model.Paciente.Nome</strong><br />@Model.Paciente.Email</p>
    <form method="post">
        <label asp-for="Telefone">Telefone</label>
        <input asp-for="Telefone" />
        <span asp-validation-for="Telefone" class="erro-campo"></span>

        <label asp-for="Canal">Receber avisos por</label>
        <select asp-for="Canal" asp-items="Html.GetEnumSelectList<CanalNotificacao>()"></select>

        <button type="submit" class="botao">Salvar</button>
    </form>
</div>
```

MediCare.Web/Pages/AreaPaciente/Perfil.cshtml.cs

```
using System.ComponentModel.DataAnnotations;
using MediCare.Core.Models;
using MediCare.Core.Services;
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Mvc;

namespace MediCare.Web.Pages.AreaPaciente;

/// <summary>
/// UC08 — Gerenciar perfil (paciente).
/// </summary>
public class PerfilModel(ContaService contas, UsuarioLogado usuarioLogado) : PaginaBase
{
    public Paciente Paciente { get; private set; } = null!;
```

```
    [BindProperty, Required(ErrorMessage = "Informe o telefone.")]
    public string Telefone { get; set; } = "";

    [BindProperty]
    public CanalNotificacao Canal { get; set; }

    public void OnGet()
    {
        Paciente = usuarioLogado.ObterPaciente(User);
        Telefone = Paciente.Telefone;
        Canal = Paciente.CanalPreferido;
    }

    public IActionResult OnPost()
    {
        Paciente = usuarioLogado.ObterPaciente(User);
        if (!ModelState.IsValid)
            return Page();

        contas.AtualizarPerfil(Paciente, Telefone, Canal);
        Mensagem = "Dados atualizados.";
        return RedirectToPage();
    }
}
```

MediCare.Web/Pages/Conta/AcessoNegado.cshtml

```
@page
@{
    ViewData["Title"] = "Acesso negado";
}
<h1>Acesso negado</h1>
<p>Esta página não está disponível para o seu perfil.</p>
<p><a href="/">Voltar ao início</a></p>
```

MediCare.Web/Pages/Conta/CriarConta.cshtml

```
@page
@model CriarContaModel
@{
    ViewData["Title"] = "Criar conta";
}
<div class="cartao estreito">
    <h1>Criar conta de paciente</h1>
    <partial name="_Mensagens" model="Model" />
    <form method="post">
        <label asp-for="Nome">Nome completo</label>
        <input asp-for="Nome" />
        <span asp-validation-for="Nome" class="erro-campo"></span>

        <label asp-for="Email">E-mail</label>
        <input asp-for="Email" />
        <span asp-validation-for="Email" class="erro-campo"></span>

        <label asp-for="Senha">Senha</label>
        <input asp-for="Senha" />
        <span asp-validation-for="Senha" class="erro-campo"></span>

        <label asp-for="Telefone">Telefone</label>
        <input asp-for="Telefone" />
        <span asp-validation-for="Telefone" class="erro-campo"></span>

        <label asp-for="DataNascimento">Data de nascimento</label>
        <input asp-for="DataNascimento" type="date" />
        <span asp-validation-for="DataNascimento" class="erro-campo"></span>

        <label asp-for="Canal">Receber avisos por</label>
        <select asp-for="Canal" asp-items="Html.GetEnumSelectList<CanalNotificacao>()"></select>

        <button type="submit" class="botao">Criar conta</button>
    </form>
</div>
```

MediCare.Web/Pages/Conta/CriarConta.cshtml.cs

```
using System.ComponentModel.DataAnnotations;
using MediCare.Core.Models;
using MediCare.Core.Services;
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Mvc;

namespace MediCare.Web.Pages.Conta;

/// <summary>
/// UC01 — Criar conta (paciente).
/// </summary>
public class CriarContaModel(ContaService contas) : PaginaBase
{
```

```
    [BindProperty, Required(ErrorMessage = "Informe o nome.")]
    public string Nome { get; set; } = "";

    [BindProperty, Required(ErrorMessage = "Informe o e-mail."), EmailAddress(ErrorMessage = "E-mail inválido.")]
    public string Email { get; set; } = "";

    [BindProperty, Required(ErrorMessage = "Informe a senha."), MinLength(6, ErrorMessage = "Use ao menos 6 caracteres."),
     DataType(DataType.Password)]
    public string Senha { get; set; } = "";

    [BindProperty, Required(ErrorMessage = "Informe o telefone.")]
    public string Telefone { get; set; } = "";

    [BindProperty, Required(ErrorMessage = "Informe a data de nascimento.")]
    public DateOnly? DataNascimento { get; set; }

    [BindProperty]
    public CanalNotificacao Canal { get; set; } = CanalNotificacao.Email;

    public void OnGet() { }

    public async Task<IActionResult> OnPostAsync()
    {
        if (!ModelState.IsValid)
            return Page();

        Paciente? paciente = null;
        if (!Tentar(() => paciente = contas.CadastrarPaciente(Nome, Email, Senha, Telefone, DataNascimento!.Value, Canal)))
            return Page();

        await HttpContext.SignInAsync(UsuarioLogado.CriarIdentidade(paciente!));
        Mensagem = "Conta criada. Bem-vindo(a) ao MediCare!";
        return RedirectToPage("/Agendamento/Consultas");
    }
}
```

MediCare.Web/Pages/Conta/CriarContaMedico.cshtml

```
@page
@model CriarContaMedicoModel
@{
    ViewData["Title"] = "Cadastro de médico";
}
<div class="cartao estreito">
    <h1>Cadastro de médico</h1>
    <partial name="_Mensagens" model="Model" />
    <form method="post">
        <label asp-for="Nome">Nome completo</label>
        <input asp-for="Nome" />
        <span asp-validation-for="Nome" class="erro-campo"></span>

        <label asp-for="Email">E-mail</label>
        <input asp-for="Email" />
        <span asp-validation-for="Email" class="erro-campo"></span>

        <label asp-for="Senha">Senha</label>
        <input asp-for="Senha" />
        <span asp-validation-for="Senha" class="erro-campo"></span>

        <label asp-for="Crm">CRM</label>
        <input asp-for="Crm" placeholder="CRM-RJ 123456" />
        <span asp-validation-for="Crm" class="erro-campo"></span>

        <label asp-for="Telefone">Telefone</label>
        <input asp-for="Telefone" />
        <span asp-validation-for="Telefone" class="erro-campo"></span>

        <label asp-for="EspecialidadeId">Especialidade</label>
        <select asp-for="EspecialidadeId" asp-items="Model.Especialidades"></select>

        <button type="submit" class="botao">Cadastrar</button>
    </form>
</div>
```

MediCare.Web/Pages/Conta/CriarContaMedico.cshtml.cs

```
using System.ComponentModel.DataAnnotations;
using MediCare.Core.Models;
using MediCare.Core.Repositories;
using MediCare.Core.Services;
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;

namespace MediCare.Web.Pages.Conta;

/// <summary>
/// UC01 — Criar conta (médico).
/// </summary>
```

```
public class CriarContaMedicoModel(ContaService contas, MediCareDados dados) : PaginaBase
{
    [BindProperty, Required(ErrorMessage = "Informe o nome.")]
    public string Nome { get; set; } = "";

    [BindProperty, Required(ErrorMessage = "Informe o e-mail."), EmailAddress(ErrorMessage = "E-mail inválido.")]
    public string Email { get; set; } = "";

    [BindProperty, Required(ErrorMessage = "Informe a senha."), MinLength(6, ErrorMessage = "Use ao menos 6 caracteres."),
     DataType(DataType.Password)]
    public string Senha { get; set; } = "";

    [BindProperty, Required(ErrorMessage = "Informe o CRM.")]
    public string Crm { get; set; } = "";

    [BindProperty, Required(ErrorMessage = "Informe o telefone.")]
    public string Telefone { get; set; } = "";

    [BindProperty]
    public int EspecialidadeId { get; set; }

    public IEnumerable<SelectListItem> Especialidades =>
        dados.Especialidades.ListarTodos().Select(e => new SelectListItem(e.Nome, e.Id.ToString()));

    public void OnGet() { }

    public async Task<IActionResult> OnPostAsync()
    {
        if (!ModelState.IsValid)
            return Page();

        Medico? medico = null;
        if (!Tentar(() => medico = contas.CadastrarMedico(Nome, Email, Senha, Crm, Telefone, EspecialidadeId)))
            return Page();

        await HttpContext.SignInAsync(UsuarioLogado.CriarIdentidade(medico!));
        Mensagem = "Cadastro concluído. Cadastre seus horários de atendimento.";
        return RedirectToPage("/AreaMedico/Horarios");
    }
}
```

MediCare.Web/Pages/Conta/Entrar.cshtml

```
@page
@model EntrarModel
@{
    ViewData["Title"] = "Entrar";
}
<div class="cartao estreito">
    <h1>Entrar</h1>
    <partial name="_Mensagens" model="Model" />
    <form method="post">
        <label asp-for="Email">E-mail</label>
        <input asp-for="Email" autofocus />
        <span asp-validation-for="Email" class="erro-campo"></span>

        <label asp-for="Senha">Senha</label>
        <input asp-for="Senha" />
        <span asp-validation-for="Senha" class="erro-campo"></span>

        <button type="submit" class="botao">Entrar</button>
    </form>
    <p class="discreto">Não tem conta? <a asp-page="/Conta/CriarConta">Crie a sua</a>.</p>
</div>
```

MediCare.Web/Pages/Conta/Entrar.cshtml.cs

```
using System.ComponentModel.DataAnnotations;
using MediCare.Core.Services;
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Mvc;

namespace MediCare.Web.Pages.Conta;

/// <summary>
/// UC02 — Realizar login.
/// </summary>
public class EntrarModel(ContaService contas) : PaginaBase
{
    [BindProperty, Required(ErrorMessage = "Informe o e-mail.")]
    public string Email { get; set; } = "";

    [BindProperty, Required(ErrorMessage = "Informe a senha."), DataType(DataType.Password)]
    public string Senha { get; set; } = "";

    public void OnGet() { }

    public async Task<IActionResult> OnPostAsync(string? returnUrl)
    {
```

```
        if (!ModelState.IsValid)
            return Page();

        try
        {
            var usuario = contas.Autenticar(Email, Senha);
            await HttpContext.SignInAsync(UsuarioLogado.CriarIdentidade(usuario));
        }
        catch (InvalidOperationException ex)
        {
            Erro = ex.Message;
            return Page();
        }

        return LocalRedirect(Url.IsLocalUrl(returnUrl) ? returnUrl : "/");
    }
}
```

MediCare.Web/Pages/Conta/Sair.cshtml

```
@page
@model SairModel
```

MediCare.Web/Pages/Conta/Sair.cshtml.cs

```
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace MediCare.Web.Pages.Conta;

public class SairModel : PageModel
{
    public IActionResult OnGet() => RedirectToPage("/Index");

    public async Task<IActionResult> OnPostAsync()
    {
        await HttpContext.SignOutAsync();
        return RedirectToPage("/Index");
    }
}
```

MediCare.Web/Pages/Recepcao/Index.cshtml

```
@page
@model MediCare.Web.Pages.Recepcao.IndexModel
@{
    ViewData["Title"] = "Recepção";
}
<h1>Recepção</h1>
<partial name="_Mensagens" model="Model" />
<p>Escolha o paciente que está sendo atendido para agendar, reagendar ou cancelar consultas.</p>

<form method="get" class="filtro">
    <label asp-for="Busca">Nome ou e-mail</label>
    <input asp-for="Busca" />
    <button type="submit" class="botao">Buscar</button>
</form>

<table>
    <thead><tr><th>Paciente</th><th>E-mail</th><th>Telefone</th><th></th></tr></thead>
    <tbody>
    @foreach (var p in Model.Pacientes)
    {
        <tr>
            <td>@p.Nome</td>
            <td>@p.Email</td>
            <td>@p.Telefone</td>
            <td class="acoes">
                <a asp-page="/Agendamento/Medicos" asp-route-pacienteId="@p.Id">Agendar</a>
                <a asp-page="/Agendamento/Consultas" asp-route-pacienteId="@p.Id">Consultas</a>
            </td>
        </tr>
    }
    </tbody>
</table>
```

MediCare.Web/Pages/Recepcao/Index.cshtml.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;
using MediCare.Web.Infraestrutura;
using Microsoft.AspNetCore.Mvc;

namespace MediCare.Web.Pages.Recepcao;

/// <summary>
/// Tela inicial da recepcionista: escolha do paciente atendido no balcão.
```

```
/// </summary>
public class IndexModel(MediCareDados dados) : PaginaBase
{
    [BindProperty(SupportsGet = true)]
    public string? Busca { get; set; }

    public IReadOnlyList<Paciente> Pacientes { get; private set; } = [];

    public void OnGet()
    {
        Pacientes = dados.Pacientes.ListarTodos()
            .Where(p => p.Ativo)
            .Where(p => string.IsNullOrWhiteSpace(Busca)
                        || p.Nome.Contains(Busca, StringComparison.OrdinalIgnoreCase)
                        || p.Email.Contains(Busca, StringComparison.OrdinalIgnoreCase))
            .OrderBy(p => p.Nome)
            .ToList();
    }
}
```

MediCare.Web/Pages/Shared/_Layout.cshtml

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>@ViewData["Title"] — MediCare</title>
    <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
</head>
<body>
    <header>
        <nav>
            <a class="marca" href="/">MediCare</a>
            @if (User.Identity?.IsAuthenticated == true)
            {
                @if (User.IsInRole(Perfis.Paciente))
                {
                    <a asp-page="/Agendamento/Consultas">Minhas consultas</a>
                    <a asp-page="/Agendamento/Medicos">Agendar</a>
                    <a asp-page="/AreaPaciente/Perfil">Meu perfil</a>
                }
                @if (User.IsInRole(Perfis.Recepcionista))
                {
                    <a asp-page="/Recepcao/Index">Pacientes</a>
                }
                @if (User.IsInRole(Perfis.Medico))
                {
                    <a asp-page="/AreaMedico/Index">Agenda</a>
                    <a asp-page="/AreaMedico/Horarios">Horários</a>
                    <a asp-page="/AreaMedico/Historico">Histórico</a>
                    <a asp-page="/AreaMedico/Perfil">Meu perfil</a>
                }
                @if (User.IsInRole(Perfis.Administrador))
                {
                    <a asp-page="/Admin/Usuarios">Usuários</a>
                    <a asp-page="/Admin/Agendamentos">Agendamentos</a>
                    <a asp-page="/Admin/Notificacoes">Notificações</a>
                }
                <span class="usuario">@User.Identity.Name</span>
                <form method="post" asp-page="/Conta/Sair" class="inline">
                    <button type="submit" class="link">Sair</button>
                </form>
            }
            else
            {
                <a asp-page="/Conta/Entrar">Entrar</a>
                <a asp-page="/Conta/CriarConta">Criar conta</a>
            }
        </nav>
    </header>
    <main>
        @RenderBody()
    </main>
</body>
</html>
```

MediCare.Web/Pages/Shared/_Mensagens.cshtml

```
@model PaginaBase
@if (!string.IsNullOrEmpty(Model.Mensagem))
{
    <div class="aviso sucesso">@Model.Mensagem</div>
}
@if (!string.IsNullOrEmpty(Model.Erro))
{
    <div class="aviso erro">@Model.Erro</div>
}
<div asp-validation-summary="ModelOnly" class="aviso erro"></div>
```

MediCare.Web/Pages/Teleconsulta/Acessar.cshtml

```
@page
@model AcessarModel
@inject Apresentacao Texto
@{
    ViewData["Title"] = "Teleconsulta";
}
<h1>Teleconsulta</h1>
<partial name="_Mensagens" model="Model" />

@if (Model.Consulta is not null)
{
    var h = Texto.Horario(Model.Consulta);
    <div class="cartao">
        <p class="aviso sucesso">Acesso liberado.</p>
        <p><strong>Paciente:</strong> @Texto.Paciente(Model.Consulta)<br />
           <strong>Médico:</strong> Dr(a). @Texto.Medico(Model.Consulta)<br />
           <strong>Horário:</strong> @Texto.DataHora(h)</p>
        <p class="discreto">A chamada de vídeo é realizada pela plataforma de teleconsulta.</p>
    </div>
}
<p><a href="/">Voltar</a></p>
```

MediCare.Web/Pages/Teleconsulta/Acessar.cshtml.cs

```
using MediCare.Core.Models;
using MediCare.Core.Services;
using MediCare.Web.Infraestrutura;

namespace MediCare.Web.Pages.Teleconsulta;

/// <summary>
/// UC11 — Realizar teleconsulta. O acesso só é liberado ao paciente e ao médico da consulta,
/// dentro do horário agendado.
/// </summary>
public class AcessarModel(AtendimentoService atendimento, UsuarioLogado usuarioLogado) : PaginaBase
{
    public Consulta? Consulta { get; private set; }

    public void OnGet(int consultaId)
    {
        var usuario = usuarioLogado.Obter(User);
        Tentar(() => Consulta = atendimento.AcessarTeleconsulta(usuario, consultaId, DateTime.Now));
    }
}
```

MediCare.Web/appsettings.json

```
{
  "Persistencia": "EfCore",
  "BancoDeDados": "medicare.db",
  "PastaDados": "../../dados",
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

MediCare.Web/wwwroot/css/site.css

```
:root {
    --azul: #1f4e79;
    --azul-claro: #eaf2fb;
    --texto: #222;
    --discreto: #666;
    --borda: #d5dbe1;
    --verde: #2e7d32;
    --vermelho: #c62828;
}

* { box-sizing: border-box; }

body {
    margin: 0;
    font-family: system-ui, -apple-system, "Segoe UI", Roboto, Arial, sans-serif;
    color: var(--texto);
    background: #f6f8fa;
}

header { background: var(--azul); }

nav {
    max-width: 1100px;
```

```
    margin: 0 auto;
    padding: 0.8rem 1.2rem;
    display: flex;
    flex-wrap: wrap;
    align-items: center;
    gap: 1.2rem;
}

nav a, nav .link { color: #fff; text-decoration: none; }
nav a:hover, nav .link:hover { text-decoration: underline; }
nav .marca { font-weight: 700; font-size: 1.2rem; margin-right: auto; }
nav .usuario { color: #cfe0f1; font-size: 0.9rem; }

main { max-width: 1100px; margin: 0 auto; padding: 1.5rem 1.2rem 3rem; }

h1 { color: var(--azul); margin-top: 0.5rem; }

a { color: var(--azul); }

table {
    width: 100%;
    border-collapse: collapse;
    background: #fff;
    border: 1px solid var(--borda);
    margin: 1rem 0;
}

th, td { padding: 0.6rem 0.8rem; text-align: left; border-bottom: 1px solid var(--borda); vertical-align: middle; }
th { background: var(--azul-claro); font-size: 0.9rem; }
td:first-child { white-space: nowrap; }
td.acoes { white-space: nowrap; }
td.acoes > * { margin-right: 0.8rem; }
td.texto-longo { max-width: 380px; font-size: 0.9rem; }

label { display: block; font-weight: 600; margin: 0.8rem 0 0.3rem; }

input, select {
    width: 100%;
    padding: 0.5rem;
    border: 1px solid var(--borda);
    border-radius: 4px;
    font: inherit;
}

.botao {
    display: inline-block;
    padding: 0.55rem 1.1rem;
    margin-top: 1rem;
    border: none;
    border-radius: 4px;
    background: var(--azul);
    color: #fff;
    font: inherit;
    text-decoration: none;
    cursor: pointer;
}

.botao.secundario { background: #fff; color: var(--azul); border: 1px solid var(--azul); }
.botao.pequeno { margin: 0; padding: 0.35rem 0.8rem; }

.link { background: none; border: none; padding: 0; color: var(--azul); font: inherit; cursor: pointer; text-decoration: underline; }
.link.perigo { color: var(--vermelho); }

.inline { display: inline; }

.cartao { background: #fff; border: 1px solid var(--borda); border-radius: 6px; padding: 1.2rem 1.5rem; margin: 1rem 0; }
.cartao.estreito { max-width: 460px; margin: 1rem auto; }

.filtro { display: flex; align-items: flex-end; gap: 0.8rem; flex-wrap: wrap; }
.filtro label { margin: 0 0 0.3rem; }
.filtro input, .filtro select { width: auto; min-width: 220px; }
.filtro .botao { margin-top: 0; }

.grade { display: grid; grid-template-columns: repeat(auto-fit, minmax(150px, 1fr)); gap: 0 1rem; }
.alinhar-base { display: flex; align-items: flex-end; }

.botoes { display: flex; gap: 0.8rem; }
.botoes .botao { margin-top: 0; }

.aviso { padding: 0.7rem 1rem; border-radius: 4px; margin: 0.8rem 0; }
.aviso:empty { display: none; }
.aviso.sucesso { background: #e8f5e9; color: var(--verde); }
.aviso.erro { background: #ffebee; color: var(--vermelho); }
.aviso.erro ul { margin: 0; padding-left: 1.2rem; }
.erro-campo { color: var(--vermelho); font-size: 0.85rem; }

.situacao { padding: 0.15rem 0.5rem; border-radius: 10px; font-size: 0.85rem; }
.situacao.agendada { background: #e3f2fd; color: var(--azul); }
.situacao.cancelada { background: #ffebee; color: var(--vermelho); }
.situacao.realizada { background: #e8f5e9; color: var(--verde); }

.faixa { background: #fff8e1; border: 1px solid #f0d98c; padding: 0.6rem 1rem; border-radius: 4px; }
```

```
.destaque { text-align: center; padding: 3rem 1rem; }
.destaque h1 { font-size: 2.4rem; }
.destaque .botao { margin: 0.5rem; }

.discreto { color: var(--discreto); font-size: 0.9rem; }
```

MediCare.Tests/AgendamentoServiceTests.cs

```
using MediCare.Core.Models;

namespace MediCare.Tests;

public class AgendamentoServiceTests
{
    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Agendar_reserva_o_horario_e_gera_confirmacao_no_canal_do_paciente(string persistencia)
    {
        using var a = new Ambiente(persistencia);

        var consulta = a.Agendamento.Agendar(a.PacienteSms.Id, a.PresencialA.Id);

        Assert.Equal(StatusConsulta.Agendada, consulta.Status);
        Assert.DoesNotContain(a.Agendamento.HorariosDisponiveis(a.MedicoA.Id), h => h.Id == a.PresencialA.Id);

        var notificacao = Assert.Single(a.Notificacoes.ListarPendentes());
        Assert.Equal(TipoNotificacao.Confirmacao, notificacao.Tipo);
        Assert.Equal(CanalNotificacao.Sms, notificacao.Canal);
        Assert.Equal("21922222222", notificacao.Destino);
        Assert.Contains("Clínica A", notificacao.Mensagem);
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Horario_ja_reservado_nao_pode_ser_agendado(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        a.Agendamento.Agendar(a.PacienteEmail.Id, a.PresencialA.Id);

        Assert.Throws<InvalidOperationException>(() => a.Agendamento.Agendar(a.PacienteSms.Id, a.PresencialA.Id));
        Assert.Single(a.Dados.Consultas.ListarTodos());
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Paciente_desativado_nao_agenda(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        a.Contas.AlterarSituacao(a.PacienteEmail.Email, ativo: false);

        Assert.Throws<InvalidOperationException>(() => a.Agendamento.Agendar(a.PacienteEmail.Id, a.PresencialA.Id));
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Cancelar_libera_o_horario_e_avisa_o_paciente(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        var consulta = a.Agendamento.Agendar(a.PacienteEmail.Id, a.PresencialA.Id);

        a.Agendamento.Cancelar(consulta.Id);

        Assert.Equal(StatusConsulta.Cancelada, consulta.Status);
        Assert.Contains(a.Agendamento.HorariosDisponiveis(a.MedicoA.Id), h => h.Id == a.PresencialA.Id);
        Assert.Contains(a.Notificacoes.ListarPendentes(), n => n.Tipo == TipoNotificacao.Cancelamento);
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Reagendar_troca_o_horario_e_libera_o_anterior(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        var consulta = a.Agendamento.Agendar(a.PacienteEmail.Id, a.PresencialA.Id);

        a.Agendamento.Reagendar(consulta.Id, a.TeleconsultaA.Id);

        Assert.Equal(a.TeleconsultaA.Id, consulta.HorarioId);
        Assert.True(a.Dados.Horarios.ObterPorId(a.PresencialA.Id)!.Disponivel);
        Assert.False(a.Dados.Horarios.ObterPorId(a.TeleconsultaA.Id)!.Disponivel);
        Assert.Contains(a.Notificacoes.ListarPendentes(), n => n.Tipo == TipoNotificacao.Reagendamento);
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Reagendar_para_outro_medico_e_recusado_sem_alterar_nada(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        var consulta = a.Agendamento.Agendar(a.PacienteEmail.Id, a.PresencialA.Id);

        Assert.Throws<InvalidOperationException>(() => a.Agendamento.Reagendar(consulta.Id, a.PresencialB.Id));
        Assert.Equal(a.PresencialA.Id, consulta.HorarioId);
        Assert.True(a.Dados.Horarios.ObterPorId(a.PresencialB.Id)!.Disponivel);
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Listagem_de_agendamentos_filtra_por_situacao(string persistencia)
    {
```

```
        using var a = new Ambiente(persistencia);
        var primeira = a.Agendamento.Agendar(a.PacienteEmail.Id, a.PresencialA.Id);
        a.Agendamento.Agendar(a.PacienteSms.Id, a.PresencialB.Id);
        a.Agendamento.Cancelar(primeira.Id);

        Assert.Equal(2, a.Agendamento.ListarAgendamentos().Count());
        Assert.Single(a.Agendamento.ListarAgendamentos(StatusConsulta.Cancelada));
        Assert.Single(a.Agendamento.ListarAgendamentos(StatusConsulta.Agendada));
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Pesquisa_ignora_medicos_desativados(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        a.Contas.AlterarSituacao(a.MedicoB.Email, ativo: false);

        var medico = Assert.Single(a.Agendamento.PesquisarMedicos());
        Assert.Equal(a.MedicoA.Id, medico.Id);
    }
}
```

MediCare.Tests/Ambiente.cs

```
using MediCare.Core.Models;
using MediCare.Core.Repositories;
using MediCare.Core.Services;
using MediCare.Data;
using Microsoft.EntityFrameworkCore;

namespace MediCare.Tests;

/// <summary>
/// Monta um cenário de teste isolado, numa pasta temporária, com a persistência indicada
/// ("Csv" ou "EfCore"). Os mesmos testes de serviço rodam com as duas implementações.
/// </summary>
public sealed class Ambiente : IDisposable
{
    public static TheoryData<string> Persistencias => new() { "Csv", "EfCore" };

    public static readonly DateOnly Dia = DateOnly.FromDateTime(DateTime.Today.AddDays(10));

    private readonly string _pasta = Path.Combine(Path.GetTempPath(), "medicare-teste-" + Guid.NewGuid());
    private readonly string _persistencia;
    private MediCareDbContext? _contexto;

    public MediCareDados Dados { get; private set; } = null!;
    public ContaService Contas { get; private set; } = null!;
    public NotificacaoService Notificacoes { get; private set; } = null!;
    public AgendamentoService Agendamento { get; private set; } = null!;
    public DisponibilidadeService Disponibilidade { get; private set; } = null!;
    public AtendimentoService Atendimento { get; private set; } = null!;

    // Cenário: dois médicos, dois pacientes e três horários livres.
    public Medico MedicoA { get; private set; } = null!;
    public Medico MedicoB { get; private set; } = null!;
    public Paciente PacienteEmail { get; private set; } = null!;
    public Paciente PacienteSms { get; private set; } = null!;
    public LocalAtendimento LocalA { get; private set; } = null!;
    public HorarioDisponivel PresencialA { get; private set; } = null!;
    public HorarioDisponivel TeleconsultaA { get; private set; } = null!;
    public HorarioDisponivel PresencialB { get; private set; } = null!;

    public Ambiente(string persistencia)
    {
        _persistencia = persistencia;
        Directory.CreateDirectory(_pasta);
        Abrir();
        Popular();
    }

    /// <summary>
    /// Descarta os objetos em memória e lê tudo de novo do armazenamento,
    /// para confirmar que as alterações foram de fato gravadas.
    /// </summary>
    public void Reabrir() => Abrir();

    private void Abrir()
    {
        if (_persistencia == "Csv")
        {
            Dados = MediCareDados.DeArquivosCsv(_pasta);
        }
        else
        {
            _contexto?.Dispose();
            var opcoes = new DbContextOptionsBuilder<MediCareDbContext>()
                .UseSqlite($"Data Source={Path.Combine(_pasta, "teste.db")}")
                .Options;
            _contexto = new MediCareDbContext(opcoes);
            _contexto.Database.EnsureCreated();
            Dados = DadosEfCore.Criar(_contexto);
```

```
        }

        Contas = new ContaService(Dados);
        Notificacoes = new NotificacaoService(Dados);
        Agendamento = new AgendamentoService(Dados, Notificacoes);
        Disponibilidade = new DisponibilidadeService(Dados);
        Atendimento = new AtendimentoService(Dados);
    }

    private void Popular()
    {
        var especialidade = Dados.Especialidades.Adicionar(new Especialidade { Nome = "Cardiologia" });
        Dados.Especialidades.Salvar();

        MedicoA = Contas.CadastrarMedico(
            "Carla Mendes", "carla@teste.com", "senha123", "CRM-RJ 1", "2133331111", especialidade.Id);
        MedicoB = Contas.CadastrarMedico(
            "Daniel Rocha", "daniel@teste.com", "senha123", "CRM-RJ 2", "2133332222", especialidade.Id);
        PacienteEmail = Contas.CadastrarPaciente("Ana Souza", "ana@teste.com", "senha123", "21911111111",
            new DateOnly(1990, 4, 12), CanalNotificacao.Email);
        PacienteSms = Contas.CadastrarPaciente("Bruno Lima", "bruno@teste.com", "senha123", "21922222222",
            new DateOnly(1985, 11, 30), CanalNotificacao.Sms);

        LocalA = Dados.Locais.Adicionar(new LocalAtendimento
            { MedicoId = MedicoA.Id, Nome = "Clínica A", Endereco = "Rua A, 10", Cidade = "Rio de Janeiro" });
        var localB = Dados.Locais.Adicionar(new LocalAtendimento
            { MedicoId = MedicoB.Id, Nome = "Clínica B", Endereco = "Rua B, 20", Cidade = "Rio de Janeiro" });
        Dados.Locais.Salvar();

        PresencialA = Disponibilidade.CadastrarHorario(
            MedicoA.Id, Dia, new(9, 0), new(9, 30), Modalidade.Presencial, LocalA.Id);
        TeleconsultaA = Disponibilidade.CadastrarHorario(
            MedicoA.Id, Dia, new(10, 0), new(10, 30), Modalidade.Teleconsulta, null);
        PresencialB = Disponibilidade.CadastrarHorario(
            MedicoB.Id, Dia, new(9, 0), new(9, 30), Modalidade.Presencial, localB.Id);
    }

    public void Dispose()
    {
        _contexto?.Dispose();
        try
        {
            Directory.Delete(_pasta, recursive: true);
        }
        catch (IOException)
        {
            // O SQLite pode manter o arquivo aberto por alguns instantes; a pasta é temporária.
        }
    }
}
```

MediCare.Tests/AtendimentoServiceTests.cs

```
using MediCare.Core.Models;

namespace MediCare.Tests;

public class AtendimentoServiceTests
{
    private static readonly DateTime InicioDaTeleconsulta = Ambiente.Dia.ToDateTime(new TimeOnly(10, 0));

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Agenda_mostra_apenas_consultas_agendadas_do_medico(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        a.Agendamento.Agendar(a.PacienteEmail.Id, a.TeleconsultaA.Id);
        a.Agendamento.Agendar(a.PacienteSms.Id, a.PresencialA.Id);
        var cancelada = a.Agendamento.Agendar(a.PacienteSms.Id, a.PresencialB.Id);
        a.Agendamento.Cancelar(cancelada.Id);

        var agenda = a.Atendimento.Agenda(a.MedicoA.Id).ToList();

        Assert.Equal(new[] { a.PresencialA.Id, a.TeleconsultaA.Id }, agenda.Select(c => c.HorarioId));
        Assert.Empty(a.Atendimento.Agenda(a.MedicoB.Id));
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Medico_so_conclui_consultas_da_propria_agenda(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        var consulta = a.Agendamento.Agendar(a.PacienteEmail.Id, a.PresencialA.Id);

        Assert.Throws<InvalidOperationException>(() => a.Atendimento.ConcluirConsulta(a.MedicoB.Id, consulta.Id));

        a.Atendimento.ConcluirConsulta(a.MedicoA.Id, consulta.Id);
        Assert.Equal(StatusConsulta.Realizada, consulta.Status);
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Historico_traz_consultas_realizadas_somente_para_medico_do_paciente(string persistencia)
    {
```

```
        using var a = new Ambiente(persistencia);
        var realizada = a.Agendamento.Agendar(a.PacienteEmail.Id, a.PresencialA.Id);
        a.Atendimento.ConcluirConsulta(a.MedicoA.Id, realizada.Id);
        a.Agendamento.Agendar(a.PacienteEmail.Id, a.TeleconsultaA.Id);

        var historico = a.Atendimento.HistoricoDoPaciente(a.MedicoA.Id, a.PacienteEmail.Id);

        Assert.Equal(realizada.Id, Assert.Single(historico).Id);
        Assert.Throws<InvalidOperationException>(() => a.Atendimento.HistoricoDoPaciente(a.MedicoB.Id, a.PacienteEmail.Id));
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Teleconsulta_e_liberada_aos_participantes_no_horario(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        var consulta = a.Agendamento.Agendar(a.PacienteEmail.Id, a.TeleconsultaA.Id);
        var duranteAConsulta = InicioDaTeleconsulta.AddMinutes(15);

        Assert.Equal(consulta.Id, a.Atendimento.AcessarTeleconsulta(a.PacienteEmail, consulta.Id, duranteAConsulta).Id);
        Assert.Equal(consulta.Id, a.Atendimento.AcessarTeleconsulta(a.MedicoA, consulta.Id, duranteAConsulta).Id);
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Teleconsulta_e_recusada_fora_do_horario_ou_para_quem_nao_participa(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        var consulta = a.Agendamento.Agendar(a.PacienteEmail.Id, a.TeleconsultaA.Id);
        var duranteAConsulta = InicioDaTeleconsulta.AddMinutes(15);

        Assert.Throws<InvalidOperationException>(() =>
            a.Atendimento.AcessarTeleconsulta(a.PacienteEmail, consulta.Id, InicioDaTeleconsulta.AddMinutes(-10)));
        Assert.Throws<InvalidOperationException>(() =>
            a.Atendimento.AcessarTeleconsulta(a.PacienteSms, consulta.Id, duranteAConsulta));
        Assert.Throws<InvalidOperationException>(() =>
            a.Atendimento.AcessarTeleconsulta(a.MedicoB, consulta.Id, duranteAConsulta));
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Consulta_presencial_nao_tem_acesso_remoto(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        var consulta = a.Agendamento.Agendar(a.PacienteEmail.Id, a.PresencialA.Id);

        var erro = Assert.Throws<InvalidOperationException>(() => a.Atendimento.AcessarTeleconsulta(
            a.PacienteEmail, consulta.Id, Ambiente.Dia.ToDateTime(new TimeOnly(9, 10))));
        Assert.Contains("presencial", erro.Message);
    }
}
```

MediCare.Tests/ContaServiceTests.cs

```
using MediCare.Core.Models;

namespace MediCare.Tests;

public class ContaServiceTests
{
    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Email_ja_usado_por_qualquer_perfil_e_recusado(string persistencia)
    {
        using var a = new Ambiente(persistencia);

        Assert.Throws<InvalidOperationException>(() => a.Contas.CadastrarPaciente(
            "Outra", "CARLA@teste.com", "senha123", "1", new DateOnly(2000, 1, 1), CanalNotificacao.Email));
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Login_com_senha_correta_retorna_o_usuario(string persistencia)
    {
        using var a = new Ambiente(persistencia);

        var usuario = a.Contas.Autenticar("ana@teste.com", "senha123");

        Assert.Equal(a.PacienteEmail.Id, Assert.IsType<Paciente>(usuario).Id);
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Login_e_recusado_com_senha_errada_ou_conta_desativada(string persistencia)
    {
        using var a = new Ambiente(persistencia);

        Assert.Throws<InvalidOperationException>(() => a.Contas.Autenticar("ana@teste.com", "errada"));

        a.Contas.AlterarSituacao("ana@teste.com", ativo: false);
        var erro = Assert.Throws<InvalidOperationException>(() => a.Contas.Autenticar("ana@teste.com", "senha123"));
        Assert.Contains("desativada", erro.Message);
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Perfil_atualizado_e_gravado(string persistencia)
    {
```

```
        using var a = new Ambiente(persistencia);

        a.Contas.AtualizarPerfil(a.PacienteEmail, "21900000000", CanalNotificacao.WhatsApp);
        a.Reabrir();

        var paciente = a.Dados.Pacientes.ObterPorId(a.PacienteEmail.Id)!;
        Assert.Equal("21900000000", paciente.Telefone);
        Assert.Equal(CanalNotificacao.WhatsApp, paciente.CanalPreferido);
        Assert.True(paciente.VerificarSenha("senha123"));
    }
}
```

MediCare.Tests/CsvTests.cs

```
using MediCare.Core.Csv;
using MediCare.Core.Models;
using MediCare.Core.Repositories;

namespace MediCare.Tests;

public class CsvTests : IDisposable
{
    private readonly string _pasta = Path.Combine(Path.GetTempPath(), "medicare-csv-" + Guid.NewGuid());

    public CsvTests() => Directory.CreateDirectory(_pasta);

    public void Dispose() => Directory.Delete(_pasta, recursive: true);

    [Fact]
    public void Campos_com_virgula_aspas_e_quebra_de_linha_voltam_iguais()
    {
        string[] campos = ["1", "Rua A, 10", "Espaço \"Crescer\"", "linha 1\nlinha 2", ""];

        var linha = CsvParser.FormatarLinha(campos);
        var lidos = CsvParser.Ler(new StringReader(linha));

        Assert.Equal(campos, Assert.Single(lidos));
    }

    [Fact]
    public void Valor_simples_nao_recebe_aspas()
    {
        Assert.Equal("1,Cardiologia,true", CsvParser.FormatarLinha(["1", "Cardiologia", "true"]));
    }

    [Fact]
    public void Aspas_nao_fechadas_geram_erro()
    {
        Assert.Throws<FormatException>(() => CsvParser.Ler(new StringReader("1,\"sem fim")));
    }

    [Fact]
    public void Repositorio_gera_ids_em_sequencia_e_regrava_o_arquivo()
    {
        var caminho = Path.Combine(_pasta, "especialidades.csv");
        var repositorio = new EspecialidadeRepository(caminho);

        var primeira = repositorio.Adicionar(new Especialidade { Nome = "Cardiologia" });
        var segunda = repositorio.Adicionar(new Especialidade { Nome = "Clínica Geral, adulto" });
        repositorio.Salvar();

        Assert.Equal(1, primeira.Id);
        Assert.Equal(2, segunda.Id);

        var relido = new EspecialidadeRepository(caminho);
        Assert.Equal(new[] { "Cardiologia", "Clínica Geral, adulto" }, relido.ListarTodos().Select(e => e.Nome));
    }

    [Fact]
    public void Cabecalho_diferente_do_esperado_e_recusado()
    {
        var caminho = Path.Combine(_pasta, "especialidades.csv");
        File.WriteAllText(caminho, "Codigo,Descricao\n1,Cardiologia\n");

        Assert.Throws<InvalidDataException>(() => new EspecialidadeRepository(caminho));
    }

    [Fact]
    public void Registro_com_data_invalida_informa_o_arquivo()
    {
        var caminho = Path.Combine(_pasta, "horarios.csv");
        File.WriteAllText(caminho,
            "Id,MedicoId,Data,HoraInicio,HoraFim,Modalidade,LocalId,Disponivel\n" +
            "1,1,2026-13-01,09:00,09:30,Presencial,1,true\n");

        var erro = Assert.Throws<InvalidDataException>(() => new HorarioRepository(caminho));
        Assert.Contains("horarios.csv", erro.Message);
    }
}
```

MediCare.Tests/DisponibilidadeServiceTests.cs

```
using MediCare.Core.Models;

namespace MediCare.Tests;

public class DisponibilidadeServiceTests
{
    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Recusa_data_passada_e_intervalo_invertido(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        var ontem = DateOnly.FromDateTime(DateTime.Today.AddDays(-1));

        Assert.Throws<ArgumentException>(() => a.Disponibilidade.CadastrarHorario(
            a.MedicoA.Id, ontem, new(9, 0), new(9, 30), Modalidade.Teleconsulta, null));
        Assert.Throws<ArgumentException>(() => a.Disponibilidade.CadastrarHorario(
            a.MedicoA.Id, Ambiente.Dia, new(15, 0), new(14, 0), Modalidade.Teleconsulta, null));
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Presencial_exige_local_do_proprio_medico(string persistencia)
    {
        using var a = new Ambiente(persistencia);

        Assert.Throws<ArgumentException>(() => a.Disponibilidade.CadastrarHorario(
            a.MedicoA.Id, Ambiente.Dia, new(14, 0), new(14, 30), Modalidade.Presencial, null));
        Assert.Throws<ArgumentException>(() => a.Disponibilidade.CadastrarHorario(
            a.MedicoB.Id, Ambiente.Dia, new(14, 0), new(14, 30), Modalidade.Presencial, a.LocalA.Id));
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Teleconsulta_nao_guarda_local(string persistencia)
    {
        using var a = new Ambiente(persistencia);

        var horario = a.Disponibilidade.CadastrarHorario(
            a.MedicoA.Id, Ambiente.Dia, new(14, 0), new(14, 30), Modalidade.Teleconsulta, a.LocalA.Id);

        Assert.Null(horario.LocalId);
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Recusa_horario_que_conflita_com_outro_do_mesmo_medico(string persistencia)
    {
        using var a = new Ambiente(persistencia);

        Assert.Throws<InvalidOperationException>(() => a.Disponibilidade.CadastrarHorario(
            a.MedicoA.Id, Ambiente.Dia, new(9, 15), new(9, 45), Modalidade.Teleconsulta, null));
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Remove_horario_livre_mas_nao_horario_ja_usado(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        a.Agendamento.Agendar(a.PacienteEmail.Id, a.PresencialA.Id);

        Assert.Throws<InvalidOperationException>(() => a.Disponibilidade.RemoverHorario(a.MedicoA.Id, a.PresencialA.Id));

        a.Disponibilidade.RemoverHorario(a.MedicoA.Id, a.TeleconsultaA.Id);
        Assert.Null(a.Dados.Horarios.ObterPorId(a.TeleconsultaA.Id));
    }
}
```

MediCare.Tests/ModeloTests.cs

```
using MediCare.Core.Models;

namespace MediCare.Tests;

public class ModeloTests
{
    [Fact]
    public void Senha_e_guardada_como_hash_e_verificada()
    {
        var paciente = new Paciente();
        paciente.DefinirSenha("segredo1");

        Assert.NotEqual("segredo1", paciente.SenhaHash);
        Assert.True(paciente.VerificarSenha("segredo1"));
        Assert.False(paciente.VerificarSenha("outra"));
    }

    [Fact]
    public void Conta_de_provedor_externo_nao_aceita_senha()
    {
        var paciente = new Paciente { Provedor = ProvedorAutenticacao.Google };
        paciente.DefinirSenha("segredo1");

        Assert.False(paciente.VerificarSenha("segredo1"));
```

```
    }

    [Fact]
    public void Horario_nao_pode_ser_reservado_duas_vezes()
    {
        var horario = new HorarioDisponivel();
        horario.Reservar();

        Assert.False(horario.Disponivel);
        Assert.Throws<InvalidOperationException>(horario.Reservar);
    }

    [Fact]
    public void Horarios_sobrepostos_conflitam_e_consecutivos_nao()
    {
        var dia = new DateOnly(2026, 12, 1);
        var nove = new HorarioDisponivel { MedicoId = 1, Data = dia, HoraInicio = new(9, 0), HoraFim = new(9, 30) };
        var noveEQuinze = new HorarioDisponivel { MedicoId = 1, Data = dia, HoraInicio = new(9, 15), HoraFim = new(9, 45) };
        var noveEMeia = new HorarioDisponivel { MedicoId = 1, Data = dia, HoraInicio = new(9, 30), HoraFim = new(10, 0) };
        var outroMedico = new HorarioDisponivel { MedicoId = 2, Data = dia, HoraInicio = new(9, 0), HoraFim = new(9, 30) };

        Assert.True(nove.ConflitaCom(noveEQuinze));
        Assert.False(nove.ConflitaCom(noveEMeia));
        Assert.False(nove.ConflitaCom(outroMedico));
    }

    [Fact]
    public void Consulta_cancelada_nao_pode_ser_alterada()
    {
        var consulta = new Consulta(pacienteId: 1, horarioId: 1);
        consulta.Cancelar();

        Assert.Equal(StatusConsulta.Cancelada, consulta.Status);
        Assert.Throws<InvalidOperationException>(consulta.Cancelar);
        Assert.Throws<InvalidOperationException>(() => consulta.Reagendar(2));
        Assert.Throws<InvalidOperationException>(consulta.Concluir);
    }

    [Fact]
    public void Notificacao_so_pode_ser_enviada_uma_vez()
    {
        var notificacao = new Notificacao(1, TipoNotificacao.Lembrete, CanalNotificacao.Email, "a@b.com", "texto");
        notificacao.MarcarComoEnviada();

        Assert.Equal(StatusNotificacao.Enviada, notificacao.Status);
        Assert.NotNull(notificacao.DataEnvio);
        Assert.Throws<InvalidOperationException>(notificacao.MarcarComoEnviada);
    }
}
```

MediCare.Tests/NotificacaoServiceTests.cs

```
using MediCare.Core.Models;

namespace MediCare.Tests;

public class NotificacaoServiceTests
{
    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Lembrete_e_gerado_uma_unica_vez_por_consulta(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        a.Agendamento.Agendar(a.PacienteEmail.Id, a.PresencialA.Id);
        a.Agendamento.Agendar(a.PacienteSms.Id, a.PresencialB.Id);

        Assert.Equal(2, a.Notificacoes.GerarLembretes(Ambiente.Dia));
        Assert.Equal(0, a.Notificacoes.GerarLembretes(Ambiente.Dia));
        Assert.Equal(0, a.Notificacoes.GerarLembretes(Ambiente.Dia.AddDays(1)));
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Notificacao_enviada_sai_da_lista_de_pendentes(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        a.Agendamento.Agendar(a.PacienteEmail.Id, a.PresencialA.Id);
        var pendente = Assert.Single(a.Notificacoes.ListarPendentes());

        a.Notificacoes.MarcarComoEnviada(pendente.Id);

        Assert.Empty(a.Notificacoes.ListarPendentes());
        Assert.Throws<InvalidOperationException>(() => a.Notificacoes.MarcarComoEnviada(pendente.Id));
    }
}
```

MediCare.Tests/PersistenciaTests.cs

```
using MediCare.Core.Models;

namespace MediCare.Tests;
/// <summary>
```

```
/// Confirma que o estado das entidades é gravado e lido de volta corretamente,
/// tanto nos arquivos CSV quanto no banco SQLite via EF Core.
/// </summary>
public class PersistenciaTests
{
    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Operacoes_de_agendamento_sao_gravadas(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        var reagendada = a.Agendamento.Agendar(a.PacienteEmail.Id, a.PresencialA.Id);
        a.Agendamento.Reagendar(reagendada.Id, a.TeleconsultaA.Id);
        var cancelada = a.Agendamento.Agendar(a.PacienteSms.Id, a.PresencialB.Id);
        a.Agendamento.Cancelar(cancelada.Id);
        var enviada = a.Notificacoes.ListarPendentes().First();
        a.Notificacoes.MarcarComoEnviada(enviada.Id);

        a.Reabrir();

        var consulta = a.Dados.Consultas.ObterPorId(reagendada.Id)!;
        Assert.Equal(a.TeleconsultaA.Id, consulta.HorarioId);
        Assert.Equal(StatusConsulta.Agendada, consulta.Status);
        Assert.Equal(StatusConsulta.Cancelada, a.Dados.Consultas.ObterPorId(cancelada.Id)!.Status);

        var horario = a.Dados.Horarios.ObterPorId(a.TeleconsultaA.Id)!;
        Assert.False(horario.Disponivel);
        Assert.Equal(Modalidade.Teleconsulta, horario.Modalidade);
        Assert.Equal(Ambiente.Dia, horario.Data);
        Assert.Equal(new TimeOnly(10, 0), horario.HoraInicio);
        Assert.Null(horario.LocalId);
        Assert.True(a.Dados.Horarios.ObterPorId(a.PresencialA.Id)!.Disponivel);

        var notificacao = a.Dados.Notificacoes.ObterPorId(enviada.Id)!;
        Assert.Equal(StatusNotificacao.Enviada, notificacao.Status);
        Assert.NotNull(notificacao.DataEnvio);
        Assert.Equal(4, a.Dados.Notificacoes.ListarTodos().Count);
    }

    [Theory, MemberData(nameof(Ambiente.Persistencias), MemberType = typeof(Ambiente))]
    public void Usuarios_sao_gravados_com_senha_e_situacao(string persistencia)
    {
        using var a = new Ambiente(persistencia);
        a.Contas.AlterarSituacao(a.MedicoB.Email, ativo: false);

        a.Reabrir();

        Assert.Equal(4, a.Dados.TodosOsUsuarios().Count());
        Assert.False(a.Dados.Medicos.ObterPorId(a.MedicoB.Id)!.Ativo);
        Assert.IsType<Medico>(a.Contas.Autenticar("carla@teste.com", "senha123"));
        Assert.Equal(CanalNotificacao.Sms, a.Dados.Pacientes.ObterPorId(a.PacienteSms.Id)!.CanalPreferido);
    }
}
```